

obj-prog
2.0

Sugeneruota Doxygen 1.17.0

1 Programos naudojimas:	1
1.1 Reikalavimai:	1
1.2 Programos diegimas:	1
1.2.1 main (paprastas programos veikimas):	1
1.2.2 test (programos testas):	1
1.2.3 clean (ištrina main bei tests sukompiliuotas programas):	1
1.3 Programos naudojimas:	2
1.3.1 main:	2
1.3.2 test:	2
1.4 v2.0:	2
1.4.1 Pridėta doxygen sukurta dokumentacija:	2
1.4.2 Pridėtas unit testing:	2
1.5 v1.5:	2
1.5.1 Pridėta bazinė klasė Zmogus:	2
1.6 v1.2:	3
1.6.1 Realizuota "Rule of Five":	3
1.6.1.1 Įvesties operatoriaus naudojimas (veikia su visais įvedimo srautais):	3
1.6.1.2 Išvesties operatoriaus naudojimas (veikia su visais išvesties srautais):	3
1.6.2 Studento klasės metodų testas.	3
1.7 v1.1 testavimas:	3
1.7.1 Spartos palyginimas:	4
1.7.1.1 Pastebėjimai	4
1.8 Relizų aprašas	5
2 Direktorijų hierarchija	7
2.1 Direktorijos	7
3 Hierarchijos Indeksas	9
3.1 Klasių hierarchija	9
4 Klasės Indeksas	11
4.1 Klasės	11
5 Failo Indeksas	13
5.1 Failai	13
6 Direktorijų dokumentacija	15
6.1 vector Direktorijos aprašas	15
7 Klasės Dokumentacija	17
7.1 Studentas Klasė	17
7.1.1 Smulkus aprašymas	19
7.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	20
7.1.2.1 Studentas() [1/5]	20

7.1.2.2 Studentas() [2/5]	20
7.1.2.3 Studentas() [3/5]	20
7.1.2.4 Studentas() [4/5]	20
7.1.2.5 Studentas() [5/5]	21
7.1.2.6 ~Studentas()	21
7.1.3 Metodų Dokumentacija	21
7.1.3.1 addNd()	21
7.1.3.2 egzaminoRezultatolvestis()	21
7.1.3.3 egzRandom()	21
7.1.3.4 galutinis()	21
7.1.3.5 genVardaPavarde()	22
7.1.3.6 getNd()	22
7.1.3.7 getRez()	22
7.1.3.8 namuDarbuRezultatulvestis()	22
7.1.3.9 ndRandom()	23
7.1.3.10 operator=() [1/2]	23
7.1.3.11 operator=() [2/2]	23
7.1.3.12 read()	23
7.1.3.13 readRandom()	24
7.1.3.14 readSemiRandom()	24
7.1.3.15 readStudentConsole()	24
7.1.3.16 readStudentStream()	24
7.1.3.17 resizeNd()	25
7.1.3.18 setNd()	25
7.1.3.19 setRez()	25
7.1.3.20 studentoVardoPavardesIvestis()	26
7.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija	26
7.1.4.1 operator<<	26
7.1.4.2 operator>>	26
7.1.5 Atributų Dokumentacija	26
7.1.5.1 nd	26
7.1.5.2 rez	27
7.2 Timer Klasė	27
7.2.1 Smulkus aprašymas	27
7.2.2 Tipo Aprašymo Dokumentacija	27
7.2.2.1 durationDouble	27
7.2.2.2 hrClock	28
7.2.3 Konstruktoriaus ir Destruktoriaus Dokumentacija	28
7.2.3.1 Timer()	28
7.2.4 Metodų Dokumentacija	28
7.2.4.1 elapsed()	28
7.2.4.2 reset()	28

7.2.5 Atributų Dokumentacija	28
7.2.5.1 start	28
7.3 Vector< T, Allocator > Klasė Šablonas	28
7.3.1 Smulkus aprašymas	32
7.3.2 Tipo Aprašymo Dokumentacija	32
7.3.2.1 allocator_type	32
7.3.2.2 const_iterator	32
7.3.2.3 const_pointer	32
7.3.2.4 const_reference	33
7.3.2.5 const_reverse_iterator	33
7.3.2.6 difference_type	33
7.3.2.7 iterator	33
7.3.2.8 pointer	33
7.3.2.9 reference	33
7.3.2.10 reverse_iterator	33
7.3.2.11 size_type	34
7.3.2.12 value_type	34
7.3.3 Konstruktoriaus ir Destruktoriaus Dokumentacija	34
7.3.3.1 Vector() [1/9]	34
7.3.3.2 Vector() [2/9]	34
7.3.3.3 Vector() [3/9]	34
7.3.3.4 Vector() [4/9]	35
7.3.3.5 Vector() [5/9]	35
7.3.3.6 Vector() [6/9]	36
7.3.3.7 Vector() [7/9]	36
7.3.3.8 Vector() [8/9]	36
7.3.3.9 Vector() [9/9]	37
7.3.3.10 ~Vector()	37
7.3.4 Metodų Dokumentacija	37
7.3.4.1 assign() [1/3]	37
7.3.4.2 assign() [2/3]	38
7.3.4.3 assign() [3/3]	38
7.3.4.4 at() [1/2]	38
7.3.4.5 at() [2/2]	39
7.3.4.6 back() [1/2]	39
7.3.4.7 back() [2/2]	40
7.3.4.8 begin() [1/2]	40
7.3.4.9 begin() [2/2]	40
7.3.4.10 capacity()	40
7.3.4.11 cbegin()	41
7.3.4.12 cend()	41
7.3.4.13 clear()	41

7.3.4.14 crbegin()	41
7.3.4.15 crend()	42
7.3.4.16 data() [1/2]	42
7.3.4.17 data() [2/2]	42
7.3.4.18 emplace()	42
7.3.4.19 emplace_back()	43
7.3.4.20 empty()	43
7.3.4.21 end() [1/2]	44
7.3.4.22 end() [2/2]	44
7.3.4.23 erase() [1/2]	44
7.3.4.24 erase() [2/2]	45
7.3.4.25 front() [1/2]	45
7.3.4.26 front() [2/2]	45
7.3.4.27 get_allocator()	46
7.3.4.28 insert() [1/5]	46
7.3.4.29 insert() [2/5]	46
7.3.4.30 insert() [3/5]	47
7.3.4.31 insert() [4/5]	47
7.3.4.32 insert() [5/5]	48
7.3.4.33 max_size()	48
7.3.4.34 operator=() [1/3]	48
7.3.4.35 operator=() [2/3]	49
7.3.4.36 operator=() [3/3]	49
7.3.4.37 operator[]() [1/2]	49
7.3.4.38 operator[]() [2/2]	50
7.3.4.39 pop_back()	50
7.3.4.40 push_back() [1/2]	50
7.3.4.41 push_back() [2/2]	51
7.3.4.42 rbegin() [1/2]	51
7.3.4.43 rbegin() [2/2]	51
7.3.4.44 reallocate()	51
7.3.4.45 rend() [1/2]	52
7.3.4.46 rend() [2/2]	52
7.3.4.47 reserve()	52
7.3.4.48 resize() [1/2]	53
7.3.4.49 resize() [2/2]	53
7.3.4.50 shift_left()	53
7.3.4.51 shift_right()	54
7.3.4.52 shrink_to_fit()	54
7.3.4.53 size()	54
7.3.4.54 swap()	54
7.3.5 Atributų Dokumentacija	55

7.3.5.1 alloc_	55
7.3.5.2 capacity_	55
7.3.5.3 data_	55
7.3.5.4 size_	55
7.4 Zmogus Klasė	55
7.4.1 Smulkus aprašymas	56
7.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija	56
7.4.2.1 Zmogus() [1/4]	56
7.4.2.2 Zmogus() [2/4]	56
7.4.2.3 Zmogus() [3/4]	57
7.4.2.4 Zmogus() [4/4]	57
7.4.2.5 ~Zmogus()	57
7.4.3 Metodų Dokumentacija	57
7.4.3.1 getPavarde()	57
7.4.3.2 getVardas()	58
7.4.3.3 operator=() [1/2]	58
7.4.3.4 operator=() [2/2]	58
7.4.3.5 setPavarde()	58
7.4.3.6 setVardas()	58
7.4.4 Atributų Dokumentacija	59
7.4.4.1 pavarde	59
7.4.4.2 vardas	59
8 Failo Dokumentacija	61
8.1 gtest.cpp Failo Nuoroda	61
8.1.1 Funkcijos Dokumentacija	61
8.1.1.1 TEST() [1/6]	61
8.1.1.2 TEST() [2/6]	61
8.1.1.3 TEST() [3/6]	62
8.1.1.4 TEST() [4/6]	62
8.1.1.5 TEST() [5/6]	62
8.1.1.6 TEST() [6/6]	62
8.2 gtest.cpp	62
8.3 README.md Failo Nuoroda	63
8.4 vector/apdorojimas.cpp Failo Nuoroda	63
8.4.1 Funkcijos Dokumentacija	64
8.4.1.1 failoGeneravimas()	64
8.4.1.2 setw()	64
8.5 apdorojimas.cpp	64
8.6 vector/apdorojimas.h Failo Nuoroda	65
8.6.1 Funkcijos Dokumentacija	65
8.6.1.1 failoGeneravimas()	65

8.6.1.2 setw()	66
8.7 apdorojimas.h	67
8.8 vector/apdorojimas.hpp Failo Nuoroda	67
8.8.1 Funkcijos Dokumentacija	68
8.8.1.1 rusiavimasPagal()	68
8.8.1.2 rusiavimasSkirstymas()	68
8.8.1.3 skirstymas()	69
8.8.1.4 skirstymasStrat1()	69
8.8.1.5 skirstymasStrat2()	70
8.8.1.6 skirstymasStrat3()	70
8.9 apdorojimas.hpp	71
8.10 vector/isvestis.cpp Failo Nuoroda	72
8.10.1 Funkcijos Dokumentacija	73
8.10.1.1 failoPasirinkimas()	73
8.10.1.2 failoUzklausa()	73
8.10.1.3 gautiTipaPasirinkima()	74
8.10.1.4 lietuviskosRaides()	74
8.10.1.5 medianosUzklausa()	74
8.10.1.6 menu()	74
8.10.1.7 ndPasirinkimas()	75
8.10.1.8 rusiavimoPasirinkimas()	75
8.10.1.9 strategijosPasirinkimas()	75
8.10.1.10 studentuPasirinkimas()	75
8.10.1.11 testavimoPasirinkimas()	76
8.11 isvestis.cpp	76
8.12 vector/isvestis.h Failo Nuoroda	78
8.12.1 Funkcijos Dokumentacija	78
8.12.1.1 failoPasirinkimas()	78
8.12.1.2 failoUzklausa()	79
8.12.1.3 gautiTipaPasirinkima()	79
8.12.1.4 lietuviskosRaides()	79
8.12.1.5 medianosUzklausa()	80
8.12.1.6 menu()	80
8.12.1.7 ndPasirinkimas()	80
8.12.1.8 rusiavimoPasirinkimas()	80
8.12.1.9 strategijosPasirinkimas()	81
8.12.1.10 studentuPasirinkimas()	81
8.12.1.11 testavimoPasirinkimas()	81
8.13 isvestis.h	82
8.14 vector/isvestis.hpp Failo Nuoroda	82
8.14.1 Funkcijos Dokumentacija	82
8.14.1.1 isvestis()	82

8.15 isvestis.hpp	84
8.16 vector/isvestis.cpp Failo Nuoroda	84
8.16.1 Funkcijos Dokumentacija	85
8.16.1.1 arTikSkaicius()	85
8.16.1.2 cinEOFgaudymas()	85
8.16.1.3 gautiPatvirtinima()	85
8.16.1.4 gautiSkaiciu()	86
8.16.1.5 ivestiStudentus()	86
8.17 isvestis.cpp	86
8.18 vector/isvestis.h Failo Nuoroda	88
8.18.1 Funkcijos Dokumentacija	88
8.18.1.1 arTikSkaicius()	88
8.18.1.2 cinEOFgaudymas()	89
8.18.1.3 egzaminoRezultatolvestis()	89
8.18.1.4 gautiPatvirtinima()	89
8.18.1.5 gautiSkaiciu()	89
8.18.1.6 ivestiStudentus()	90
8.18.1.7 namuDarbuRezultatulvestis()	90
8.18.1.8 skaitymas()	90
8.18.1.9 skaitymasIsFailo()	91
8.18.1.10 studentoVardoPavardesIvestis()	91
8.19 isvestis.h	92
8.20 vector/isvestis.hpp Failo Nuoroda	92
8.20.1 Funkcijos Dokumentacija	92
8.20.1.1 skaitymasIsFailo()	92
8.21 isvestis.hpp	93
8.22 vector/main.cpp Failo Nuoroda	93
8.22.1 Funkcijos Dokumentacija	94
8.22.1.1 failoApdorojimas()	94
8.22.1.2 main()	94
8.23 main.cpp	94
8.24 vector/main.h Failo Nuoroda	97
8.24.1 Funkcijos Dokumentacija	97
8.24.1.1 failoApdorojimas()	97
8.25 main.h	98
8.26 vector/main.hpp Failo Nuoroda	98
8.26.1 Funkcijos Dokumentacija	98
8.26.1.1 failoTestavimas()	98
8.26.1.2 skirstymoPasirinkimas() [1/2]	99
8.26.1.3 skirstymoPasirinkimas() [2/2]	100
8.27 main.hpp	100
8.28 vector/random.cpp Failo Nuoroda	101

8.28.1 Funkcijos Dokumentacija	102
8.28.1.1 iverstiStudentusRandom()	102
8.28.1.2 rng()	102
8.29 random.cpp	102
8.30 vector/random.h Failo Nuoroda	103
8.30.1 Funkcijos Dokumentacija	103
8.30.1.1 iverstiStudentusRandom()	103
8.30.1.2 rng()	104
8.31 random.h	104
8.32 vector/studentas.cpp Failo Nuoroda	104
8.32.1 Funkcijos Dokumentacija	104
8.32.1.1 operator<<()	104
8.32.1.2 operator>>()	105
8.33 studentas.cpp	105
8.34 vector/studentas.h Failo Nuoroda	108
8.34.1 Kintamojo Dokumentacija	108
8.34.1.1 maxNdKiekis	108
8.35 studentas.h	109
8.36 vector/test.cpp Failo Nuoroda	110
8.36.1 Funkcijos Dokumentacija	110
8.36.1.1 testas()	110
8.36.1.2 testCopyAssignmentOperator()	110
8.36.1.3 testCopyKonstruktorius()	111
8.36.1.4 testDefaultKonstruktorius()	111
8.36.1.5 testDestructor()	111
8.36.1.6 testInputOperator()	111
8.36.1.7 testMoveAssignmentOperator()	111
8.36.1.8 testMoveKonstruktorius()	111
8.36.1.9 testRezultatas()	111
8.36.1.10 testOutputOperator()	112
8.36.1.11 testStreamKonstruktorius()	112
8.37 test.cpp	112
8.38 vector/test.h Failo Nuoroda	113
8.38.1 Funkcijos Dokumentacija	114
8.38.1.1 testas()	114
8.38.1.2 testCopyAssignmentOperator()	114
8.38.1.3 testCopyKonstruktorius()	114
8.38.1.4 testDefaultKonstruktorius()	115
8.38.1.5 testDestructor()	115
8.38.1.6 testInputOperator()	115
8.38.1.7 testMoveAssignmentOperator()	115
8.38.1.8 testMoveKonstruktorius()	115

8.38.1.9 testoRezultatas()	115
8.38.1.10 testOutputOperator()	116
8.38.1.11 testStreamKonstruktorius()	116
8.39 test.h	116
8.40 vector/Timer.h Failo Nuoroda	116
8.41 Timer.h	117
8.42 vector/vector.h Failo Nuoroda	117
8.42.1 Funkcijos Dokumentacija	118
8.42.1.1 erase()	118
8.42.1.2 erase_if()	118
8.42.1.3 operator<=>()	119
8.42.1.4 operator==()	119
8.42.1.5 swap()	120
8.43 vector.h	120
8.44 vector/vector_compare.cpp Failo Nuoroda	127
8.44.1 Funkcijos Dokumentacija	128
8.44.1.1 vector_compare()	128
8.45 vector_compare.cpp	128
8.46 vector/vector_compare.h Failo Nuoroda	128
8.46.1 Funkcijos Dokumentacija	129
8.46.1.1 vector_compare()	129
8.47 vector_compare.h	129
8.48 vector/zmogus.cpp Failo Nuoroda	129
8.49 zmogus.cpp	129
8.50 vector/zmogus.h Failo Nuoroda	130
8.51 zmogus.h	130
Rodyklė	131

skyrius 1

Programos naudojimas:

1.1 Reikalavimai:

C++ kompiliatorius su C++20 palaikymu. Unix OS (Linux arba MacOS) arba WSL (Windows Subsystem for Linux)
git

1.2 Programos diegimas:

```
git clone https://github.com/AtlantasLTU/obj-prog-2.git cd ./obj-prog-2
```

1.2.1 main (paprastas programos veikimas):

```
make main ./main
```

1.2.2 test (programos testas):

```
make test
```

Jei jau sukompiliuotas tests paleidimo failas, tai tiesiog: `./tests`

1.2.3 clean (ištrina main bei tests sukompiliuotas programas):

```
make clean
```

1.3 Programos naudojimas:

1.3.1 main:

Funkcijos: Pasirinkti išvestis į failą arba į terminalą. Pasirinktinai galutinio rezultato skaičiavimas, remiantis vidurkiu arba mediana. Direktoriijoje esančių .txt failų pasirinkimas. Rūšiavimas pasirinktu būdu.

1 parinktis - rankinis duomenų įvedimas, studento vardo, pavardės, namų darbų rezultatų, egzamino rezultato, jų apdorojimas ir išvedimas. 2 parinktis - pusiau rankinis duomenų įvedimas, studento vardo, pavardės, rezultatų generavimas, duomenų apdorojimas ir išvedimas. 3 parinktis - automatinis studentų vardų, pavardžių, rezultatų generavimas, jų apdorojimas ir išvedimas. 4 parinktis - skaitymas iš pasirinktinio failo, rūšiavimas pasirinktinu būdu, duomenų apdorojimas ir išvedimas. 5 parinktis - testavimas su failais, pasirenkamas failas, konteinerio tipas, strategija, testų skaičius, failai apdorojami (nuskaitymas, skaičiavimas, rūšiavimas, skirstymas) ir išvedami testo rezultatai į terminalą. 6 parinktis - studentų failų generavimas, studentų, namų darbų kiekio pasirinkimas ir išvedimas į studentai*.txt failą. 7 parinktis - studento klasės testavimas. 8 parinktis - programos nutraukimas.

1.3.2 test:

`make test` paleidžia testus, kurie ištestuoja rule of five, pasitelkiant gtest bibliotekos įrankiais.

1.4 v2.0:

1.4.1 Pridėta doxygen sukurta dokumentacija:

Aprašyti metodai bei klasės. Sukurtos nuorodos tarp metodų.

1.4.2 Pridėtas unit testing:

Naudojamas gtest Automatinis studento klasės rule of five testavimas

1.5 v1.5:

1.5.1 Pridėta bazinė klasė Zmogus:

Klasė studentas dabar išvestinė. Studento klasę paveldi iš žmogaus klasės vardą bei pavardę, metodus susijusiais su šiais kintamaisiais Klasė abstrakti dėl virtualaus destruktoriaus.

- Studento klasės rule of five atnaujintas, kad veiktų su bazinė klase.
- Rule of five taip pat realizuotas Zmogaus klasėje.
- Išlaikytas v1.2 testas bei funkcionalumas.

Bandant sukurti Zmogaus tipo objektą, išmeta klaidą, kad klasė abstrakti:

1.6 v1.2:

1.6.1 Realizuota "Rule of Five":

Destruktorius Copy konstruktorius Copy assignment operatorius Move konstruktorius Move assignment operatorius

- Realizuotas įvesties ir išvesties operatorių perdengimas.
- Sukurtas studento klasės metodų testas.

1.6.1.1 Įvesties operatoriaus naudojimas (veikia su visais įvedimo srautais):

Kai naudojamas `operator>>` su failo srautu, pažymių kiekis nustatomas automatiškai: visi skaičiai po vardo ir pavardės laikomi namų darbų pažymiais, o pats paskutinis – egzamino balu.

```
Studentas A;
std::istringstream iss("Jonas Jonaitis 1 2 10");
iss >> A; // įvestis iš įvedimo srauto.
std::cin >> A; // įvestis iš konsolės.
```

1.6.1.2 Išvesties operatoriaus naudojimas (veikia su visais išvesties srautais):

`Studentas` A; `std::cout << A;` // išvestis į ekraną `std::ostringstream out; out << A;` // išvesties srauto kūrimas `std::ofstream fout("studentai.txt"); fout << A;` // išveda į failą

1.6.2 Studento klasės metodų testas.

1.7 v1.1 testavimas:

- Kompiuterio, su kuriuo testuota parametrai:

1.7.0.0.1 Skirstymo strategijos:

- 0 - pradiniam relize naudota strategija. Dviejų konteinerių "vargšiukų" ir "kietiukų" sukūrimas, duomenys perkeliama iš "studentai" konteinerio su `std::move`.
- 1 strategija: Bendro studentai konteinerio (vector, list ir deque tipų) skaidymas (rūšiavimas) į du naujus to paties tipo konteinerius: "vargšiukų" ir "kietiukų". Dviejų konteinerių "vargšiukų" ir "kietiukų" sukūrimas, duomenys kopijuojami iš "studentai" konteinerio.
- 2 strategija: Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant tik vieną naują konteinerį: "vargšiukai". Studentai konteineris išrūšiuotas, randamas iteratorius rodantis į pirmąjį galvočių, viskas iki iteratoriaus perkeliama į vargšiukų konteinerį ir ištrinama iš studentai konteinerio. Iteratoriui rasti naudojamas `lower_bound` metodas
- 3 strategija: Bendro studentų konteinerio (vector, list ir deque) skaidymas (rūšiavimas) panaudojant greičiausiai veikianti 1 arba 2 strategiją įtraukiant į ją "efektyvius" darbo su konteineriais metodus. Šioje strategijoje naudojamas `partition` metodas.

1.7.0.0.2 Atlikta programos veikimo greičio (spartos) analizė:

- lyginamas v1.0 struct tipas su v1.1 realizuota klasė.
- lyginamas vienas konteinerius - vektorius, su pačia greičiausia dalijimo strategija - 2.
- 100000 ir 1000000 studentų failai.
- lyginamos kompiliatoriaus optimizavimo vėliavėlės (-Ofast, -O1, -O2, -O3).

Testuota tik naudojant terminalą, visos kitos pašalinės programos testavimo metu buvo išjungtos bei įrenginys "performance" režime. Visi testavimo atvejai testuoti 10 kartų, su medianų skaičiavimu.

1.7.1 Spartos palyginimas:**1.7.1.0.1 100000 studentų:**

Tipas	Optimizavimo vėliavėlė	Bendras veikimo laikas (s)	Failo dydis (KB)
struct	-Ofast	1.65366 s	349.3 KB
class	-Ofast	4.66815 s	332.7 KB
struct	-O1	2.43243 s	300.6 KB
class	-O1	7.11753 s	293.2 KB
struct	-O2	2.24629 s	314.2 KB
class	-O2	6.3197 s	308.2 KB
struct	-O3	2.1662 s	349.5 KB
class	-O3	5.94076 s	332.7 KB

1.7.1.0.2 1000000 studentų:

Tipas	Optimizavimo vėliavėlė	Bendras veikimo laikas (s)	Failo dydis (KB)
struct	-Ofast	18.4434 s	349.3 KB
class	-Ofast	66.1051 s	332.7 KB
struct	-O1	20.5588 s	300.6 KB
class	-O1	79.2437 s	293.2 KB
struct	-O2	18.8914 s	314.2 KB
class	-O2	72.3371 s	308.2 KB
struct	-O3	18.0112 s	349.5 KB
class	-O3	55.4038 s	332.7 KB

1.7.1.1 Pastebėjimai

Ta pati programa perdaryta su class tipu lėtesnė ~3x negu su struct, priklausomai nuo optimizacijos vėliavėlės. Taip yra todėl, kad class tipo programoje, galutinis balas skaičiuojamas, kai jo prireikia ir nėra saugojamas atminty, tai sutaupo atminties, tačiau padidina skaičiavimų laiką. Pagrindė -Ofast vėliavėlė greičiausia, tačiau esant milijonui studentų -O3 aplenkia -Ofast vėliavėlę, galbūt -O3 geriau suoptimizuoja failų skaitymą. Mažiausią failą sukuria -O1 vėliavėlė, o didžiausią -O3. Taip yra todėl, kad -O1 skirtas balansui tarp greičio ir dydžio, o -O3 skirtas našesniai programos veikimui.

Nors struct tipas greitesnis, tačiau class tipas saugesnis ir plečiant programos apimtį, enkapsuliacija, paveldamumas ir kitos class tipo ypatybės padaro jį naudingesniu.

1.8 Relizų aprašas

v1.5

Sukurta bazinė klasė Zmogus.
Klasė Studentas paveldi vardą bei pavardę iš klasės Zmogus.
Kodas pritaikytas darbui su bazine klase.

v1.2

Realizuota rule of five.
Realizuotas įvesties išvesties operatorių perkrovimas.
Sukurta studento klasės metodų testas.

v1.1

Studento struktūra paversta į klasę
Implementuoti get'eriai, set'eriai, konstruktoriai bei destruktoriai.
Kintamieji privati, pasiekiami tik per studento klasės metodus.
Refaktorintas ir patobulintas random vardu, pavardžių generatorius.
Nebesaugojamas kintamasis galutinio rezultato, jis skaičiuojamas tik, kai jo prireikia.

v1.0

Padidintas template naudojimas, siekiant ištestuoti vector, deque ir list konteinerius.
Pridėtos trys studentų skirstymo į „vargšiukus“ (vidurkis < 5.0) ir „kietiakus“ strategijos:
1 strategija: Bendro konteinerio skaidymas į du naujus, kopijuojant į vargsiukai ir galvočiai konteinerius.
2 strategija: „Vargšiukų“ perkėlimas į naują konteinerį, juos ištrinant iš studentų konteinerio.
3 strategija: Optimizuotas skirstymas naudojant efektyvius algoritmus (std::partition).

Paruoštas pilnas README.md su tyrimo rezultatais, lentelėmis ir naudojimo instrukcija.
Pridėtas Makefile lengvam programos kompiliavimui.

v0.4

Patobulinta išvestis.
Pridėtas skirstymas į "galvočius" ir "vargšiukus".
Pridėtas failų generavimas.
Pridėtas failų pasirinkimas.

v0.3

Kodas išskirstytas į daugiau dalių (.h ir .cpp failus).
Įterptas try-catch blokas failų egzistavimo tikrinimui, kitų klaidų gaudymui.
Panaudotas template rūšiavimo funkcijoje, kodo skaitomumui pagerinti.
Refaktorius metu apšvarintas main.cpp failas.

v0.2

Pridėtas duomenų nuskaitymas iš išorinių failų.
Pridėti testavimo atvejai.

v0.1

Pridėtas automatinis pažymių generavimas.
Įdiegtas „sąžiningas“ vidurkio skaičiavimas (atsižvelgiant į trūkstantus namų darbus).
Patobulintas įvesties valdymas (apsauga nuo neteisingos įvesties).

V.pradinė

Sukurta studento struktūra.
Realizuotas vidurkio ir medianos skaičiavimas.
Pradinė įvesties apsauga.
Hardcoded reikšmės testavimui.

skyrius 2

Direktorių hierarchija

2.1 Direktorijos

vector	15
apdorojimas.cpp	63
apdorojimas.h	65
apdorojimas.hpp	67
isvestis.cpp	72
isvestis.h	78
isvestis.hpp	82
investis.cpp	84
investis.h	88
investis.hpp	92
main.cpp	93
main.h	97
main.hpp	98
random.cpp	101
random.h	103
studentas.cpp	104
studentas.h	108
test.cpp	110
test.h	113
Timer.h	116
vector.h	117
vector_compare.cpp	127
vector_compare.h	128
zmogus.cpp	129
zmogus.h	130

skyrius 3

Hierarchijos Indeksas

3.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Timer	27
Vector< T, Allocator >	28
Zmogus	55
Studentas	17

skyrius 4

Klasės Indeksas

4.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	
Klasė, sukurianti studento objektą su pažymiais	17
Timer	27
Vector< T, Allocator >	
Dinaminio masyvo konteineris, atitinkantis std::vector sąsają (C++20)	28
Zmogus	
Abstrakti bazinė klasė, turinti vardą ir pavardę	55

skyrius 5

Failo Indeksas

5.1 Failai

Visų failų sąrašas su trumpais aprašymais:

gtest.cpp	61
vector/apdorojimas.cpp	63
vector/apdorojimas.h	65
vector/apdorojimas.tpp	67
vector/isvestis.cpp	72
vector/isvestis.h	78
vector/isvestis.tpp	82
vector/ivestis.cpp	84
vector/ivestis.h	88
vector/ivestis.tpp	92
vector/main.cpp	93
vector/main.h	97
vector/main.tpp	98
vector/random.cpp	101
vector/random.h	103
vector/studentas.cpp	104
vector/studentas.h	108
vector/test.cpp	110
vector/test.h	113
vector/Timer.h	116
vector/vector.h	117
vector/vector_compare.cpp	127
vector/vector_compare.h	128
vector/zmogus.cpp	129
vector/zmogus.h	130

skyrius 6

Direktorių dokumentacija

6.1 vector Directorijos aprašas

Failai

- failas [apdorojimas.cpp](#)
- failas [apdorojimas.h](#)
- failas [apdorojimas.tpp](#)
- failas [isvestis.cpp](#)
- failas [isvestis.h](#)
- failas [isvestis.tpp](#)
- failas [investis.cpp](#)
- failas [investis.h](#)
- failas [investis.tpp](#)
- failas [main.cpp](#)
- failas [main.h](#)
- failas [main.tpp](#)
- failas [random.cpp](#)
- failas [random.h](#)
- failas [studentas.cpp](#)
- failas [studentas.h](#)
- failas [test.cpp](#)
- failas [test.h](#)
- failas [Timer.h](#)
- failas [vector.h](#)
- failas [vector_compare.cpp](#)
- failas [vector_compare.h](#)
- failas [zmogus.cpp](#)
- failas [zmogus.h](#)

skyrius 7

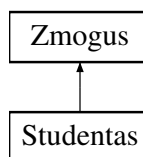
Klasės Dokumentacija

7.1 Studentas Klasė

Klasė, sukurianti studento objektą su pažymiais.

```
#include <studentas.h>
```

Paveldimumo diagrama Studentas:



Vieši Metodai

- `Studentas ()`
Default konstruktorius.
- `Studentas (std::string v, std::string p, std::vector< int > n, int r)`
Konstruktorius su visais parametrais.
- `Studentas (std::istream &is, int ndKiekis=0)`
Konstruktorius, nuskaitantis studentą iš srauto.
- `Studentas (const Studentas &kitas)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&) noexcept`
Perkėlimo konstruktorius.
- `Studentas & operator= (Studentas &&) noexcept`
Perkėlimo priskyrimo operatorius.
- `const std::vector< int > & getNd () const`
- `int getRez () const`
- `void setNd (std::vector< int > n)`
Nustato namų darbų pažymius.
- `void setRez (int r)`

- *Nustato egzamino rezultatą.*
- `std::istream & readStudentStream (std::istream &is, int ndKiekis)`
Nuskaityto studentą iš failo srauto.
- `std::istream & readStudentConsole (std::istream &is)`
Nuskaityto studentą iš terminalo.
- `void read (std::istream &is, int ndKiekis=0)`
Nuskaityto studentą iš srauto (automatiškai parenka readStudentConsole arba readStudentStream).
- `bool readSemiRandom ()`
Dalinai atsitiktinis įvedimas: vardas/pavardė ranka, pažymiai atsitiktiniai.
- `void readRandom (int ndKiekis)`
Visiškai atsitiktinis studento ir jo rezultatų sugeneravimas.
- `void addNd (int paz)`
Prideda vieną namų darbų pažymį.
- `void resizeNd (int n, int skaicius=0)`
Resize'ina namų darbų vektorių ir užpildo naujas vietas nurodyta reikšme.
- `double galutinis (bool medianos, int ndKiekis=0) const`
Apskaičiuoja galutinį pažymį.
- `~Studentas ()`
Destruktorius.

Vieši Metodai inherited from **Zmogus**

- `Zmogus ()`
Numatytasis konstruktorius – tuščias vardas ir pavardė.
- `Zmogus (const std::string v, const std::string p)`
Konstruktorius su vardu ir pavarde.
- `Zmogus (const Zmogus &kitas)`
Kopijavimo konstruktorius.
- `Zmogus & operator= (const Zmogus &)`
Kopijavimo priskyrimo operatorius.
- `Zmogus (Zmogus &&) noexcept`
Perkėlimo konstruktorius.
- `Zmogus & operator= (Zmogus &&) noexcept`
Perkėlimo priskyrimo operatorius.
- `const std::string & getVardas () const`
- `const std::string & getPavarde () const`
- `void setVardas (std::string v)`
Nustato vardą.
- `void setPavarde (std::string p)`
Nustato pavardę.
- `virtual ~Zmogus ()=0`
Virtualus destruktoriaus – padaro klasę abstrakčia.

Privatatūs Metodai

- bool `studentoVardoPavardesIvestis` (const std::string &eilute)
Nuskaityto vardą ir pavardę iš eilutės.
- void `namuDarbuRezultatulvestis` ()
Nuskaityto namų darbų rezultatus iš terminalo.
- void `egzaminoRezultatulvestis` ()
Nuskaityto egzamino rezultatą iš terminalo.
- void `genVardaPavarde` ()
Atsitiktinai sugeneruoja vardą ir pavardę.
- void `ndRandom` (int ndKiekis)
Atsitiktinai sugeneruoja namų darbų rezultatus.
- void `egzRandom` ()
Atsitiktinai sugeneruoja egzamino rezultatą.

Privatūs Atributai

- std::vector< int > `nd`
Namų darbų pažymių vektorius.
- int `rez`
Egzamino pažymys.

Draugai

- std::istream & `operator>>` (std::istream &in, `Studentas` &A)
Ivesties operatorius (>>).
- std::ostream & `operator<<` (std::ostream &out, const `Studentas` &A)
Išvesties operatorius (<<).

Additional Inherited Members

Apsaugoti Atributai inherited from `Zmogus`

- std::string `vardas`
Zmogaus vardas.
- std::string `pavarde`
Zmogaus pavardė.

7.1.1 Smulkus aprašymas

Klasė, sukurianti studento objektą su pažymiais.

Paveldi iš klasės `Zmogus`. Saugo namų darbų pažymius ir egzamino rezultatą. Leidžia skaičiuoti galutinį pažymį (pagal vidurkį arba medianą).

Apibrėžimas failo `studentas.h` eilutėje 19.

7.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.1.2.1 Studentas() [1/5]

```
Studentas::Studentas () [inline]
```

Default konstruktorius.

Apibrėžimas failo [studentas.h](#) eilutėje 51.

7.1.2.2 Studentas() [2/5]

```
Studentas::Studentas (
    std::string v,
    std::string p,
    std::vector< int > n,
    int r) [inline]
```

Konstruktorius su visais parametrais.

Parametrai

<i>v</i>	Vardas.
<i>p</i>	Pavardė.
<i>n</i>	Namų darbų pažymių vektorius.
<i>r</i>	Egzamino rezultatas.

Apibrėžimas failo [studentas.h](#) eilutėje 60.

7.1.2.3 Studentas() [3/5]

```
Studentas::Studentas (
    std::istream & is,
    int ndKiekis = 0) [inline]
```

Konstruktorius, nuskaitantis studentą iš srauto.

Parametrai

<i>is</i>	Išvesties srautas.
<i>ndKiekis</i>	Namų darbų skaičius (jei 0, tai nustatoma iš duomenų).

Apibrėžimas failo [studentas.h](#) eilutėje 68.

7.1.2.4 Studentas() [4/5]

```
Studentas::Studentas (
    const Studentas & kitas)
```

Kopijavimo konstruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 222.

7.1.2.5 Studentas() [5/5]

```
Studentas::Studentas (
    Studentas && kitas) [noexcept]
```

Perkėlimo konstruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 239.

7.1.2.6 ~Studentas()

```
Studentas::~~Studentas ()
```

Destruktorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 216.

7.1.3 Metodų Dokumentacija**7.1.3.1 addNd()**

```
void Studentas::addNd (
    int paz)
```

Prideda vieną namų darbų pažymį.

Parametrai

<i>paz</i>	Pažymys (1-10).
------------	-----------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 12.

7.1.3.2 egzaminoRezultatolvestis()

```
void Studentas::egzaminoRezultatoIvestis () [private]
```

Nuskaityto egzamino rezultatą iš terminalo.

Apibrėžimas failo [studentas.cpp](#) eilutėje 144.

7.1.3.3 egzRandom()

```
void Studentas::egzRandom () [private]
```

Atsitiktinai sugeneruoja egzamino rezultatą.

Apibrėžimas failo [studentas.cpp](#) eilutėje 210.

7.1.3.4 galutinis()

```
double Studentas::galutinis (
    bool medianos,
    int ndKiekis = 0) const
```

Apskaičiuoja galutinį pažymį.

Parametrai

<i>medianos</i>	Jei true – naudoja medianą, kitu atveju – vidurkį.
-----------------	--

<i>ndKiekis</i>	Jei >0 – naudoja šį ND skaičių, kitu atveju – maxNdKiekis .
-----------------	--

Gražina

Galutinis pažymys ($0.4 * \text{ND} + 0.6 * \text{egz.}$).

Apibrėžimas failo [studentas.cpp](#) eilutėje 22.

7.1.3.5 genVardaPavarde()

```
void Studentas::genVardaPavarde () [private]
```

Atsitiktinai sugeneruoja vardą ir pavardę.

Apibrėžimas failo [studentas.cpp](#) eilutėje 181.

7.1.3.6 getNd()

```
const std::vector< int > & Studentas::getNd () const [inline]
```

Gražina

Namų darbų pažymių vektoriaus konstantinė nuoroda.

Apibrėžimas failo [studentas.h](#) eilutėje 84.

7.1.3.7 getRez()

```
int Studentas::getRez () const [inline]
```

Gražina

Egzamino rezultatas.

Apibrėžimas failo [studentas.h](#) eilutėje 87.

7.1.3.8 namuDarbuRezultatulvestis()

```
void Studentas::namuDarbuRezultatuIvestis () [private]
```

Nuskaityto namų darbų rezultatus iš terminalo.

Apibrėžimas failo [studentas.cpp](#) eilutėje 128.

7.1.3.9 ndRandom()

```
void Studentas::ndRandom (
    int ndKiekis) [private]
```

Atsitiktinai sugeneruoja namų darbų rezultatus.

Parametrai

<i>ndKiekis</i>	Kiek rezultatų generuoti.
-----------------	---------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 203.

7.1.3.10 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & kitas)
```

Kopijavimo priskyrimo operatorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 228.

7.1.3.11 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && kitas) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [studentas.cpp](#) eilutėje 245.

7.1.3.12 read()

```
void Studentas::read (
    std::istream & is,
    int ndKiekis = 0)
```

Nuskaito studentą iš srauto (automatiškai parenka readStudentConsole arba readStudentStream).

Parametrai

<i>is</i>	Ivesties srautas.
<i>ndKiekis</i>	Namų darbų skaičius (naudojamas tik failo skaitymui).

Apibrėžimas failo [studentas.cpp](#) eilutėje 268.

7.1.3.13 readRandom()

```
void Studentas::readRandom (
    int ndKiekis)
```

Visiškai atsitiktinis studento ir jo rezultatų sugeneravimas.

Parametrai

<i>ndKiekis</i>	Kiek namų darbų pažymių sugeneruoti.
-----------------	--------------------------------------

Apibrėžimas failo [studentas.cpp](#) eilutėje 174.

7.1.3.14 readSemiRandom()

```
bool Studentas::readSemiRandom ()
```

Dalinai atsitiktinis įvedimas: vardas/pavardė ranka, pažymiai atsitiktiniai.

Gražina

true jei vartotojas įvedė ne tuščią eilutę, false jei įveda ENTER - užbaigia įvedimą.

Apibrėžimas failo [studentas.cpp](#) eilutėje 149.

7.1.3.15 readStudentConsole()

```
std::istream & Studentas::readStudentConsole (
    std::istream & is)
```

Nuskaito studentą iš terminalo.

Parametrai

<i>is</i>	Įvesties srautas (paprastai std::cin).
-----------	--

Gražina

Srauto nuoroda.

Apibrėžimas failo [studentas.cpp](#) eilutėje 85.

7.1.3.16 readStudentStream()

```
std::istream & Studentas::readStudentStream (
    std::istream & is,
    int ndKiekis)
```

Nuskaito studentą iš failo srauto.

Parametrai

<i>is</i>	Įvesties srautas.
-----------	-------------------

<i>ndKiekis</i>	Jei >0, skaitoma būtent tiek nd rezultatų, kitaip skaitoma iki failo galo ir egzamino rezultatas nustatomas automatiškai.
-----------------	---

Gražina

Srauto nuoroda.

Apibrėžimas failo [studentas.cpp](#) eilutėje 53.

7.1.3.17 `resizeNd()`

```
void Studentas::resizeNd (
    int n,
    int skaicius = 0)
```

Resize'ina namų darbų vektorių ir užpildo naujas vietas nurodyta reikšme.

Parametrai

<i>n</i>	Naujas dydis.
<i>skaicius</i>	Reikšmė naujiems elementams (numatyta 0).

Apibrėžimas failo [studentas.cpp](#) eilutėje 17.

7.1.3.18 `setNd()`

```
void Studentas::setNd (
    std::vector< int > n) [inline]
```

Nustato namų darbų pažymius.

Parametrai

<i>n</i>	Naujas pažymių vektorius.
----------	---------------------------

Apibrėžimas failo [studentas.h](#) eilutėje 94.

7.1.3.19 `setRez()`

```
void Studentas::setRez (
    int r) [inline]
```

Nustato egzamino rezultatą.

Parametrai

<i>r</i>	Naujas egzamino pažymys.
----------	--------------------------

Apibrėžimas failo [studentas.h](#) eilutėje 100.

7.1.3.20 studentoVardoPavardesIvestis()

```
bool Studentas::studentoVardoPavardesIvestis (
    const std::string & eilute) [private]
```

Nuskaityto vardą ir pavardę iš eilutės.

Parametrai

<i>eilute</i>	Eilutė, kurioje tikimasi dviejų žodžių.
---------------	---

Gražina

true jei nuskaitymas sėkmingas, false jei formatas neteisingas.

Apibrėžimas failo [studentas.cpp](#) eilutėje 110.

7.1.4 Draugiškų Ir Susijusių Funkcijų Dokumentacija

7.1.4.1 operator<<

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & A) [friend]
```

Išvesties operatorius (<<).

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.

7.1.4.2 operator>>

```
std::istream & operator>> (
    std::istream & in,
    Studentas & A) [friend]
```

Išvesties operatorius (>>).

Apibrėžimas failo [studentas.cpp](#) eilutėje 255.

7.1.5 Atributų Dokumentacija

7.1.5.1 nd

```
std::vector<int> Studentas::nd [private]
```

Namų darbų pažymių vektorius.

Apibrėžimas failo [studentas.h](#) eilutėje 20.

7.1.5.2 rez

```
int Studentas::rez [private]
```

Egzamino pažymys.

Apibrėžimas failo [studentas.h](#) eilutėje 21.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- vector/[studentas.h](#)
- vector/[studentas.cpp](#)

7.2 Timer Klasė

```
#include <Timer.h>
```

Vieši Metodai

- [Timer](#) ()
- void [reset](#) ()
- double [elapsed](#) () const

Privatūs Tipai

- using [hrClock](#) = std::chrono::high_resolution_clock
- using [durationDouble](#) = std::chrono::duration<double>

Privatūs Atributai

- std::chrono::time_point< [hrClock](#) > [start](#)

7.2.1 Smulkus aprašymas

Apibrėžimas failo [Timer.h](#) eilutėje 5.

7.2.2 Tipo Aprašymo Dokumentacija

7.2.2.1 durationDouble

```
using Timer::durationDouble = std::chrono::duration<double> [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 9.

7.2.2.2 hrClock

```
using Timer::hrClock = std::chrono::high_resolution_clock [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 8.

7.2.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.2.3.1 Timer()

```
Timer::Timer () [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 12.

7.2.4 Metodų Dokumentacija

7.2.4.1 elapsed()

```
double Timer::elapsed () const [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 16.

7.2.4.2 reset()

```
void Timer::reset () [inline]
```

Apibrėžimas failo [Timer.h](#) eilutėje 13.

7.2.5 Atributų Dokumentacija

7.2.5.1 start

```
std::chrono::time_point<hrClock> Timer::start [private]
```

Apibrėžimas failo [Timer.h](#) eilutėje 10.

Dokumentacija šiai klasei sugeneruota iš šio failo:

- [vector/Timer.h](#)

7.3 Vector< T, Allocator > Klasė Šablonas

Dinaminio masyvo konteineris, atitinkantis `std::vector` sąsają (C++20).

```
#include <vector.h>
```


Vieši Tipai

- using `value_type` = T
- using `allocator_type` = Allocator
- using `size_type` = std::size_t
- using `difference_type` = std::ptrdiff_t
- using `reference` = T&
- using `const_reference` = const T&
- using `pointer` = typename std::allocator_traits<Allocator>::pointer
- using `const_pointer` = typename std::allocator_traits<Allocator>::const_pointer
- using `iterator` = `pointer`
- using `const_iterator` = `const_pointer`
- using `reverse_iterator` = std::reverse_iterator<`iterator`>
- using `const_reverse_iterator` = std::reverse_iterator<`const_iterator`>

Vieši Metodai

- `Vector` ()
Sukuria tuščią vektorių.
- `Vector` (`size_type` count, const Allocator &alloc=Allocator{})
Sukuria vektorių su `count` numatytais elementais.
- `Vector` (`size_type` count, `const_reference` value, const Allocator &alloc=Allocator{})
Sukuria vektorių su `count` kopijomis reikšmės `value`.
- template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
`Vector` (InputIt first, InputIt last, const Allocator &alloc=Allocator{})
Sukuria vektorių iš iteratorių intervalo [first, last).
- `Vector` (std::initializer_list< T > ilist, const Allocator &alloc=Allocator{})
Sukuria vektorių iš std::initializer_list.
- `Vector` (const `Vector` &other)
Kopijavimo konstruktorius.
- `Vector` (const `Vector` &other, const Allocator &alloc)
Kopijavimo konstruktorius su nurodytu paskirstytoju.
- `Vector` & `operator=` (const `Vector` &other)
Kopijavimo priskyrimo operatorius.
- `Vector` (`Vector` &&other) noexcept
Perkėlimo konstruktorius.
- `Vector` (`Vector` &&other, const Allocator &alloc)
Perkėlimo konstruktorius su nurodytu paskirstytoju.
- `Vector` & `operator=` (std::initializer_list< T > ilist)
Priskiria std::initializer_list reikšmės.
- `Vector` & `operator=` (`Vector` &&other) noexcept
Perkėlimo priskyrimo operatorius.
- `~Vector` ()
Destruktorius.
- void `assign` (`size_type` count, `const_reference` value)
Pakeičia turinį `count` kopijomis reikšmės `value`.
- template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
void `assign` (InputIt first, InputIt last)
Pakeičia turinį elementais iš iteratorių intervalo [first, last).
- void `assign` (std::initializer_list< T > ilist)
Pakeičia turinį std::initializer_list elementais.

- `allocator_type get_allocator () const`
Grąžina naudojamo atminties paskirstytojo kopiją.
- `reference at (size_type pos)`
Grąžina nuorodą į elementą pozicijoje `pos` su ribų tikrinimu.
- `const_reference at (size_type pos) const`
Grąžina konstantinę nuorodą į elementą pozicijoje `pos` su ribų tikrinimu.
- `reference operator[] (size_type pos)`
Grąžina nuorodą į elementą pozicijoje `pos` be ribų tikrinimo.
- `const_reference operator[] (size_type pos) const`
Grąžina konstantinę nuorodą į elementą pozicijoje `pos` be ribų tikrinimo.
- `reference front ()`
Grąžina nuorodą į pirmą elementą.
- `const_reference front () const`
Grąžina konstantinę nuorodą į pirmą elementą.
- `reference back ()`
Grąžina nuorodą į paskutinį elementą.
- `const_reference back () const`
Grąžina konstantinę nuorodą į paskutinį elementą.
- `pointer data ()`
Grąžina rodyklę į vidinį duomenų masyvą.
- `const_pointer data () const`
Grąžina konstantinę rodyklę į vidinį duomenų masyvą.
- `iterator begin ()`
Grąžina iteratorių į pirmą elementą.
- `const_iterator begin () const`
Grąžina konstantinį iteratorių į pirmą elementą.
- `const_iterator cbegin () const noexcept`
Grąžina konstantinį iteratorių į pirmą elementą (const vektoriams).
- `iterator end () noexcept`
Grąžina iteratorių į poziciją po paskutinio elemento.
- `const_iterator end () const noexcept`
Grąžina konstantinį iteratorių į poziciją po paskutinio elemento.
- `const_iterator cend () const noexcept`
Grąžina konstantinį iteratorių į poziciją po paskutinio elemento (const vektoriams).
- `reverse_iterator rbegin ()`
Grąžina atbulinį iteratorių į pirmą elementą atbuline tvarka (t.y.
- `const_reverse_iterator rbegin () const`
Grąžina konstantinį atbulinį iteratorių į pirmą elementą atbuline tvarka.
- `const_reverse_iterator crbegin () const noexcept`
Grąžina konstantinį atbulinį iteratorių į pirmą elementą atbuline tvarka (const vektoriams).
- `reverse_iterator rend ()`
Grąžina atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka.
- `const_reverse_iterator rend () const`
Grąžina konstantinį atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka.
- `const_reverse_iterator crend () const noexcept`
Grąžina konstantinį atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka (const vektoriams).
- `bool empty () const`
Patikrina, ar vektorius tuščias.
- `size_type size () const`
Grąžina elementų skaičių.
- `size_type max_size () const`

- Grąžina maksimalų galimą elementų skaičių pagal paskirstytoją.*
- void `reserve (size_type new_cap)`
Rezervuoja atmintį bent new_cap elementams.
- `size_type capacity () const`
Grąžina dabartinę talpą (kiek elementų galima sutalpinti be perskirstymo).
- void `shrink_to_fit ()`
Sumažina talpą iki esamo dydžio, atlaisvindama nenaudojamą atmintį.
- void `clear ()`
Sunaikina visus elementus; dydis tampa 0, talpa nesikeičia.
- `iterator insert (const_iterator pos, const_reference value)`
Įterpia value kopiją prieš pos.
- `iterator insert (const_iterator pos, T &&value)`
Įterpia value perkeltinai prieš pos.
- `iterator insert (const_iterator pos, size_type count, const_reference value)`
Įterpia count kopijų reikšmės value prieš pos.
- `template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>`
`iterator insert (const_iterator pos, InputIt first, InputIt last)`
Įterpia elementus iš iteratorių intervalo [first, last) prieš pos.
- `iterator insert (const_iterator pos, std::initializer_list< T > ilist)`
Įterpia std::initializer_list elementus prieš pos.
- `template<class... Args>`
`iterator emplace (const_iterator pos, Args &&... args)`
Sukuria elementą vietoje prieš pos, perduodant argumentus jo konstruktoriui.
- `iterator erase (const_iterator pos)`
Pašalina elementą, į kurį rodo pos.
- `iterator erase (const_iterator first, const_iterator last)`
Pašalina elementus intervale [first, last).
- void `push_back (const_reference value)`
Prideda value kopiją į vektoriaus pabaigą.
- void `push_back (value_type &&value)`
Prideda value perkeltinai į vektoriaus pabaigą.
- `template<class... Args>`
`reference emplace_back (Args &&... args)`
Sukuria elementą vektoriaus pabaigoje, perduodant argumentus jo konstruktoriui.
- void `pop_back ()`
Pašalina paskutinį vektoriaus elementą.
- void `resize (size_type count)`
Pakeičia vektoriaus dydį į count.
- void `resize (size_type count, const_reference value)`
Pakeičia vektoriaus dydį į count, užpildant naujus elementus reikšme value.
- void `swap (Vector &other) noexcept`
Sukeičia turinį su other vektoriumi.

Privatūs Metodai

- void `reallocate (size_type new_cap)`
Perskirsto vidinį masyvą į new_cap dydžio buferį.
- void `shift_right (size_type index, size_type count)`
Pastumia elementus į dešinę nuo index, atlaisvindama count pozicijų.
- void `shift_left (size_type index, size_type count)`
Pastumia elementus į kairę nuo index, uždengdama count pozicijų.

Privatūs Atributai

- `pointer data_` = nullptr
Rodyklė į elementų masyvą.
- `size_type size_` = 0
Esamas elementų skaičius.
- `size_type capacity_` = 0
Rezervuota atmintis (elementais).
- `allocator_type alloc_`
Atminties paskirstytojas.

7.3.1 Smulkus aprašymas

```
template<class T, class Allocator = std::allocator<T>>
class Vector< T, Allocator >
```

Dinaminio masyvo konteineris, atitinkantis `std::vector` sąsają (C++20).

Template Parameters

<i>T</i>	Elementų tipas.
<i>Allocator</i>	Atminties paskirstymo tipo (numatytasis – <code>std::allocator<T></code>).

Apibrėžimas failo [vector.h](#) eilutėje 25.

7.3.2 Tipo Aprašymo Dokumentacija

7.3.2.1 allocator_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::allocator_type = Allocator
```

Apibrėžimas failo [vector.h](#) eilutėje 29.

7.3.2.2 const_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_iterator = const_pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 37.

7.3.2.3 const_pointer

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_pointer = typename std::allocator_traits<Allocator>::const_pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 35.

7.3.2.4 const_reference

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reference = const T&
```

Apibrėžimas failo [vector.h](#) eilutėje 33.

7.3.2.5 const_reverse_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Apibrėžimas failo [vector.h](#) eilutėje 39.

7.3.2.6 difference_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::difference_type = std::ptrdiff_t
```

Apibrėžimas failo [vector.h](#) eilutėje 31.

7.3.2.7 iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::iterator = pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 36.

7.3.2.8 pointer

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::pointer = typename std::allocator_traits<Allocator>::pointer
```

Apibrėžimas failo [vector.h](#) eilutėje 34.

7.3.2.9 reference

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::reference = T&
```

Apibrėžimas failo [vector.h](#) eilutėje 32.

7.3.2.10 reverse_iterator

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::reverse_iterator = std::reverse_iterator<iterator>
```

Apibrėžimas failo [vector.h](#) eilutėje 38.

7.3.2.11 size_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::size_type = std::size_t
```

Apibrėžimas failo [vector.h](#) eilutėje 30.

7.3.2.12 value_type

```
template<class T, class Allocator = std::allocator<T>>
using Vector< T, Allocator >::value_type = T
```

Apibrėžimas failo [vector.h](#) eilutėje 28.

7.3.3 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.3.3.1 Vector() [1/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector () [inline]
```

Sukuria tuščią vektorių.

Apibrėžimas failo [vector.h](#) eilutėje 46.

7.3.3.2 Vector() [2/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const Allocator & alloc = Allocator{}) [inline], [explicit]
```

Sukuria vektorių su `count` numatytais elementais.

Parametrai

<i>count</i>	Elementų skaičius.
<i>alloc</i>	Naudojamas atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 54.

7.3.3.3 Vector() [3/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    size_type count,
    const_reference value,
    const Allocator & alloc = Allocator{}) [inline]
```

Sukuria vektorių su `count` kopijomis reikšmės `value`.

Parametrai

<i>count</i>	Elementų skaičius.
--------------	--------------------

<i>value</i>	Reikšmė, kuria užpildomi elementai.
<i>alloc</i>	Naudojamas atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 66.

7.3.3.4 Vector() [4/9]

```
template<class T, class Allocator = std::allocator<T>>
template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
Vector< T, Allocator >::Vector (
    InputIt first,
    InputIt last,
    const Allocator & alloc = Allocator{}) [inline]
```

Sukuria vektorių iš iteratorių intervalo [first, last).

Šis konstruktorius neigalinamas, jei InputIt yra integralus tipas (kad nebūtų painiojamas su count konstruktoriumi).

Template Parameters

<i>InputIt</i>	Ivesties iteratoriaus tipas.
----------------	------------------------------

Parametrai

<i>first</i>	Intervalo pradžia.
<i>last</i>	Intervalo pabaiga.
<i>alloc</i>	Naudojamas atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 86.

7.3.3.5 Vector() [5/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    std::initializer_list< T > ilist,
    const Allocator & alloc = Allocator{}) [inline]
```

Sukuria vektorių iš std::initializer_list.

Parametrai

<i>ilist</i>	Inicijavimo sąrašas.
<i>alloc</i>	Naudojamas atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 102.

7.3.3.6 Vector() [6/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo konstruktorius.

Sukuria vektorių, identišką `other`, naudodamas `other` paskirstytojo kopiją.

Parametrai

<i>other</i>	Kopijuojamas vektorius.
--------------	-------------------------

Apibrėžimas failo [vector.h](#) eilutėje 112.

7.3.3.7 Vector() [7/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    const Vector< T, Allocator > & other,
    const Allocator & alloc) [inline]
```

Kopijavimo konstruktorius su nurodytu paskirstytoju.

Sukuria vektorių, identišką `other`, tačiau naudoja duotą paskirstytoją `alloc`.

Parametrai

<i>other</i>	Kopijuojamas vektorius.
<i>alloc</i>	Šiam vektoriui naudojamas paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 129.

7.3.3.8 Vector() [8/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkėlimo konstruktorius.

Perima `other` vidinius išteklis, palikdamas `other` tuščią.

Parametrai

<i>other</i>	Perkeliamas vektorius.
--------------	------------------------

Apibrėžimas failo [vector.h](#) eilutėje 171.

7.3.3.9 Vector() [9/9]

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::Vector (
    Vector< T, Allocator > && other,
    const Allocator & alloc) [inline]
```

Perkėlimo konstruktorius su nurodytu paskirstytoju.

Jei pateiktas paskirstytojas `alloc` sutampa su `other` paskirstytoju, perima išteklius; priešingu atveju atlieka perkėlimo konstrukciją elementas po elemento.

Parametrai

<i>other</i>	Perkeliamas vektorius.
<i>alloc</i>	Naudojamas atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 187.

7.3.3.10 ~Vector()

```
template<class T, class Allocator = std::allocator<T>>
Vector< T, Allocator >::~~Vector () [inline]
```

Destruktorius.

Sunaikina visus elementus ir atlaisvina atmintį.

Apibrėžimas failo [vector.h](#) eilutėje 249.

7.3.4 Metodų Dokumentacija

7.3.4.1 assign() [1/3]

```
template<class T, class Allocator = std::allocator<T>>
template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::↔
type>
void Vector< T, Allocator >::assign (
    InputIt first,
    InputIt last) [inline]
```

Pakeičia turinį elementais iš iteratorių intervalo [first, last).

Template Parameters

<i>Input↔ It</i>	Įvesties iteratoriaus tipas.
----------------------	------------------------------

Parametrai

<i>first</i>	Intervalo pradžia.
--------------	--------------------

<i>last</i>	Intervalo pabaiga.
-------------	--------------------

Apibrėžimas failo [vector.h](#) eilutėje 299.

7.3.4.2 assign() [2/3]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    size_type count,
    const_reference value) [inline]
```

Pakeičia turinį `count` kopijomis reikšmės `value`.

Jei `count` mažesnis už esamą dydį, pertekliniai elementai sunaikinami. Jei reikia, perskirsto atmintį.

Parametrai

<i>count</i>	Naujas elementų skaičius.
<i>value</i>	Reikšmė, kuria užpildomi elementai.

Apibrėžimas failo [vector.h](#) eilutėje 267.

7.3.4.3 assign() [3/3]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::assign (
    std::initializer_list< T > ilist) [inline]
```

Pakeičia turinį `std::initializer_list` elementais.

Parametrai

<i>ilist</i>	Inicijavimo sąrašas.
--------------	----------------------

Apibrėžimas failo [vector.h](#) eilutėje 329.

7.3.4.4 at() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::at (
    size_type pos) [inline]
```

Grąžina nuorodą į elementą pozicijoje `pos` su ribų tikrinimu.

Parametrai

<i>pos</i>	Elemento indeksas.
------------	--------------------

Gražina

reference

Išimtys

<code>std::out_of_range</code>	Jei <code>pos >= size()</code> .
--------------------------------	-------------------------------------

Apibrėžimas failo [vector.h](#) eilutėje 351.

7.3.4.5 at() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::at (
    size_type pos) const [inline]
```

Gražina konstantinę nuorodą į elementą pozicijoje `pos` su ribų tikrinimu.

Parametrai

<code>pos</code>	Elemento indeksas.
------------------	--------------------

Gražina

const_reference

Išimtys

<code>std::out_of_range</code>	Jei <code>pos >= size()</code> .
--------------------------------	-------------------------------------

Apibrėžimas failo [vector.h](#) eilutėje 363.

7.3.4.6 back() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::back () [inline]
```

Gražina nuorodą į paskutinį elementą.

Gražina

reference

Apibrėžimas failo [vector.h](#) eilutėje 411.

7.3.4.7 back() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::back () const [inline]
```

Gražina konstantinę nuorodą į paskutinį elementą.

Gražina

`const_reference`

Apibrėžimas failo [vector.h](#) eilutėje 420.

7.3.4.8 begin() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::begin () [inline]
```

Gražina iteratorių į pirmą elementą.

Gražina

`iterator`

Apibrėžimas failo [vector.h](#) eilutėje 448.

7.3.4.9 begin() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::begin () const [inline]
```

Gražina konstantinį iteratorių į pirmą elementą.

Gražina

`const_iterator`

Apibrėžimas failo [vector.h](#) eilutėje 456.

7.3.4.10 capacity()

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::capacity () const [inline]
```

Gražina dabartinę talpą (kiek elementų galima sutalpinti be perskirstymo).

Gražina

`size_type`

Apibrėžimas failo [vector.h](#) eilutėje 584.

7.3.4.11 cbegin()

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cbegin () const [inline], [noexcept]
```

Grąžina konstantinį iteratorių į pirmą elementą (const vektoriams).

Gražina

`const_iterator`

Apibrėžimas failo `vector.h` eilutėje 464.

7.3.4.12 cend()

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::cend () const [inline], [noexcept]
```

Grąžina konstantinį iteratorių į poziciją po paskutinio elemento (const vektoriams).

Gražina

`const_iterator`

Apibrėžimas failo `vector.h` eilutėje 488.

7.3.4.13 clear()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::clear () [inline]
```

Sunaikina visus elementus; dydis tampa 0, talpa nesikeičia.

Apibrėžimas failo `vector.h` eilutėje 608.

7.3.4.14 crbegin()

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crbegin () const [inline], [noexcept]
```

Grąžina konstantinį atbulinį iteratorių į pirmą elementą atbuline tvarka (const vektoriams).

Gražina

`const_reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 512.

7.3.4.15 crend()

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::crend () const [inline], [noexcept]
```

Gražina konstantinį atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka (const vektoriams).

Gražina

`const_reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 536.

7.3.4.16 data() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
pointer Vector< T, Allocator >::data () [inline]
```

Gražina rodyklę į vidinį duomenų masyvą.

Gražina

`pointer`

Apibrėžimas failo `vector.h` eilutėje 429.

7.3.4.17 data() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_pointer Vector< T, Allocator >::data () const [inline]
```

Gražina konstantinę rodyklę į vidinį duomenų masyvą.

Gražina

`const_pointer`

Apibrėžimas failo `vector.h` eilutėje 438.

7.3.4.18 emplace()

```
template<class T, class Allocator = std::allocator<T>>
template<class... Args>
iterator Vector< T, Allocator >::emplace (
    const_iterator pos,
    Args &&... args) [inline]
```

Sukuria elementą vietoje prieš `pos`, perduodant argumentus jo konstruktoriui.

Template Parameters

<i>Args</i>	Argumentų tipai.
-------------	------------------

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį kurti.
------------	---

<i>args</i>	Argumentai elemento konstruktoriui.
-------------	-------------------------------------

Gražina

Iteratorius į naują elementą.

Apibrėžimas failo [vector.h](#) eilutėje 716.

7.3.4.19 `emplace_back()`

```
template<class T, class Allocator = std::allocator<T>>
template<class... Args>
reference Vector< T, Allocator >::emplace_back (
    Args &&... args) [inline]
```

Sukuria elementą vektoriaus pabaigoje, perduodant argumentus į konstruktoriui.

Template Parameters

<i>Args</i>	Argumentų tipai.
-------------	------------------

Parametrai

<i>args</i>	Argumentai elemento konstruktoriui.
-------------	-------------------------------------

Gražina

Nuoroda į naujai sukurtą elementą.

Apibrėžimas failo [vector.h](#) eilutėje 789.

7.3.4.20 `empty()`

```
template<class T, class Allocator = std::allocator<T>>
bool Vector< T, Allocator >::empty () const [inline]
```

Patikrina, ar vektorius tuščias.

Gražina

true jei [size\(\)](#) == 0, kitaip false.

Apibrėžimas failo [vector.h](#) eilutėje 546.

7.3.4.21 end() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
const_iterator Vector< T, Allocator >::end () const [inline], [noexcept]
```

Grąžina konstantinį iteratorių į poziciją po paskutinio elemento.

Gražina

`const_iterator`

Apibrėžimas failo [vector.h](#) eilutėje 480.

7.3.4.22 end() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::end () [inline], [noexcept]
```

Grąžina iteratorių į poziciją po paskutinio elemento.

Gražina

`iterator`

Apibrėžimas failo [vector.h](#) eilutėje 472.

7.3.4.23 erase() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator first,
    const_iterator last) [inline]
```

Pašalina elementus intervale [first, last).

Parametrai

<i>first</i>	Konstantinis iteratorius į pirmą šalinamą elementą.
<i>last</i>	Konstantinis iteratorius po paskutinio šalinamo elemento.

Gražina

Iteratorius į elementą, buvusį po pašalintų.

Apibrėžimas failo [vector.h](#) eilutėje 746.

7.3.4.24 erase() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::erase (
    const_iterator pos) [inline]
```

Pašalina elementą, į kurį rodo `pos`.

Parametrai

<code>pos</code>	Konstantinis iteratorius į šalinamą elementą.
------------------	---

Gražina

Iteratorius į elementą, buvusį po pašalinto.

Apibrėžimas failo [vector.h](#) eilutėje 731.

7.3.4.25 front() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::front () [inline]
```

Gražina nuorodą į pirmą elementą.

Gražina

reference

Apibrėžimas failo [vector.h](#) eilutėje 393.

7.3.4.26 front() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::front () const [inline]
```

Gražina konstantinę nuorodą į pirmą elementą.

Gražina

`const_reference`

Apibrėžimas failo [vector.h](#) eilutėje 402.

7.3.4.27 `get_allocator()`

```
template<class T, class Allocator = std::allocator<T>>
allocator_type Vector< T, Allocator >::get_allocator () const [inline]
```

Gražina naudojamo atminties paskirstytojo kopiją.

Gražina

`allocator_type`

Apibrėžimas failo `vector.h` eilutėje 338.

7.3.4.28 `insert()` [1/5]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    const_reference value) [inline]
```

Įterpia `value` kopiją prieš `pos`.

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį įterpti.
<i>value</i>	Reikšmė.

Gražina

Iteratorius į įterptą elementą.

Apibrėžimas failo `vector.h` eilutėje 622.

7.3.4.29 `insert()` [2/5]

```
template<class T, class Allocator = std::allocator<T>>
template<class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::↵
type>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last) [inline]
```

Įterpia elementus iš iteratorių intervalo `[first, last)` prieš `pos`.

Template Parameters

<i>Input↵ It</i>	Įvesties iteratoriaus tipas.
----------------------	------------------------------

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį įterpti.
------------	---

<i>first</i>	Intervalo pradžia.
<i>last</i>	Intervalo pabaiga.

Gražina

Iteratorius į pirmą įterptą elementą (arba `pos`, jei `first == last`).

Apibrėžimas failo [vector.h](#) eilutėje 676.

7.3.4.30 insert() [3/5]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    size_type count,
    const_reference value) [inline]
```

Įterpia `count` kopijų reikšmės `value` prieš `pos`.

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį įterpti.
<i>count</i>	Kopijų skaičius.
<i>value</i>	Reikšmė.

Gražina

Iteratorius į pirmą įterptą elementą (arba `pos`, jei `count == 0`).

Apibrėžimas failo [vector.h](#) eilutėje 655.

7.3.4.31 insert() [4/5]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    std::initializer_list< T > ilist) [inline]
```

Įterpia `std::initializer_list` elementus prieš `pos`.

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį įterpti.
<i>ilist</i>	Inicijavimo sąrašas.

Gražina

Iteratorius į pirmą įterptą elementą.

Apibrėžimas failo [vector.h](#) eilutėje 703.

7.3.4.32 insert() [5/5]

```
template<class T, class Allocator = std::allocator<T>>
iterator Vector< T, Allocator >::insert (
    const_iterator pos,
    T && value) [inline]
```

Įterpia *value* perkeltinai prieš *pos*.

Parametrai

<i>pos</i>	Konstantinis iteratorius, prieš kurį įterpti.
<i>value</i>	Perkeliamą reikšmę.

Gražina

Iteratorius į įterptą elementą.

Apibrėžimas failo [vector.h](#) eilutėje 638.

7.3.4.33 max_size()

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::max_size () const [inline]
```

Gražina maksimalų galimų elementų skaičių pagal paskirstytoją.

Gražina

[size_type](#)

Apibrėžimas failo [vector.h](#) eilutėje 562.

7.3.4.34 operator=() [1/3]

```
template<class T, class Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    const Vector< T, Allocator > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Pakeičia esamą turinį *other* kopija.

Parametrai

<i>other</i>	Kopijuojamas vektorius.
--------------	-------------------------

Gražina

*this

Apibrėžimas failo [vector.h](#) eilutėje 146.

7.3.4.35 operator=() [2/3]

```
template<class T, class Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    std::initializer_list< T > ilist) [inline]
```

Priskiria std::initializer_list reikšmes.

Pakeičia esamą turinį inicijavimo sąrašo elementais.

Parametrai

<i>ilist</i>	Inicijavimo sąrašas.
--------------	----------------------

Gražina

*this

Apibrėžimas failo [vector.h](#) eilutėje 214.

7.3.4.36 operator=() [3/3]

```
template<class T, class Allocator = std::allocator<T>>
Vector & Vector< T, Allocator >::operator= (
    Vector< T, Allocator > && other) [inline], [noexcept]
```

Perkélimo priskyrimo operatorius.

Atlaisvina esamus išteklius ir perima other vidinius išteklius.

Parametrai

<i>other</i>	Perkeliamas vektorius.
--------------	------------------------

Gražina

*this

Apibrėžimas failo [vector.h](#) eilutėje 227.

7.3.4.37 operator[]() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reference Vector< T, Allocator >::operator[] (
    size_type pos) [inline]
```

Gražina nuorodą į elementą pozicijoje pos be ribų tikrinimo.

Parametrai

<i>pos</i>	Elemento indeksas.
------------	--------------------

Gražina

reference

Apibrėžimas failo [vector.h](#) eilutėje 374.

7.3.4.38 operator[]() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reference Vector< T, Allocator >::operator[] (
    size_type pos) const [inline]
```

Gražina konstantinę nuorodą į elementą pozicijoje `pos` be ribų tikrinimo.

Parametrai

<i>pos</i>	Elemento indeksas.
------------	--------------------

Gražina

`const_reference`

Apibrėžimas failo [vector.h](#) eilutėje 384.

7.3.4.39 pop_back()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::pop_back () [inline]
```

Pašalina paskutinį vektoriaus elementą.

Pastaba

Vektorius turi būti netuščias.

Apibrėžimas failo [vector.h](#) eilutėje 801.

7.3.4.40 push_back() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    const_reference value) [inline]
```

Prideda `value` kopiją į vektoriaus pabaigą.

Jei nėra pakankamai talpos, perskirsto atmintį.

Parametrai

<i>value</i>	Pridedama reikšmė.
--------------	--------------------

Apibrėžimas failo [vector.h](#) eilutėje 764.

7.3.4.41 push_back() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::push_back (
    value_type && value) [inline]
```

Prideda `value` perkeltinai į vektoriaus pabaigą.

Parametrai

<code>value</code>	Perkeliama reikšmė.
--------------------	---------------------

Apibrėžimas failo `vector.h` eilutėje 775.

7.3.4.42 rbegin() [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rbegin () [inline]
```

Gražina atbulinį iteratorių į pirmą elementą atbuline tvarka (t.y. paskutinį).

Gražina

`reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 496.

7.3.4.43 rbegin() [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rbegin () const [inline]
```

Gražina konstantinį atbulinį iteratorių į pirmą elementą atbuline tvarka.

Gražina

`const_reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 504.

7.3.4.44 reallocate()

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::reallocate (
    size_type new_cap) [inline], [private]
```

Perskirsto vidinį masyvą į `new_cap` dydžio buferį.

Perkelia visus esamus elementus naudodamas `move_if_noexcept`, sunaikina senus ir atlaisvina buvusį buferį.

Parametrai

<code>new_cap</code>	Nauja talpa.
----------------------	--------------

Apibrėžimas failo `vector.h` eilutėje 886.

7.3.4.45 `rend()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
reverse_iterator Vector< T, Allocator >::rend () [inline]
```

Gražina atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka.

Gražina

`reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 520.

7.3.4.46 `rend()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
const_reverse_iterator Vector< T, Allocator >::rend () const [inline]
```

Gražina konstantinį atbulinį iteratorių į poziciją prieš pirmą elementą atbuline tvarka.

Gražina

`const_reverse_iterator`

Apibrėžimas failo `vector.h` eilutėje 528.

7.3.4.47 `reserve()`

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::reserve (
    size_type new_cap) [inline]
```

Rezervuoja atmintį bent `new_cap` elementams.

Jei `new_cap > capacity()`, perskirsto vidinį masyvą.

Parametrai

<code>new_cap</code>	Nauja talpa.
----------------------	--------------

Išimtys

<code>std::length_error</code>	Jei <code>new_cap > max_size()</code> .
--------------------------------	--

Apibrėžimas failo `vector.h` eilutėje 574.

7.3.4.48 `resize()` [1/2]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count) [inline]
```

Pakeičia vektoriaus dydį į `count`.

Jei `count > size()`, pridedami numatyti elementai. Jei `count < size()`, elementai pašalinami nuo galo.

Parametrai

<i>count</i>	Naujas dydis.
--------------	---------------

Apibrėžimas failo `vector.h` eilutėje 815.

7.3.4.49 `resize()` [2/2]

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::resize (
    size_type count,
    const_reference value) [inline]
```

Pakeičia vektoriaus dydį į `count`, užpildant naujus elementus reikšme `value`.

Parametrai

<i>count</i>	Naujas dydis.
<i>value</i>	Reikšmė, kuria užpildomi nauji elementai.

Apibrėžimas failo `vector.h` eilutėje 841.

7.3.4.50 `shift_left()`

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::shift_left (
    size_type index,
    size_type count) [inline], [private]
```

Pastumia elementus į kairę nuo `index`, uždengdama `count` pozicijų.

Elementai perkeliama naudojant `move_if_noexcept`.

Parametrai

<i>index</i>	Pozicija, nuo kurios pradedama stumti.
<i>count</i>	Pašalintų pozicijų skaičius.

Apibrėžimas failo `vector.h` eilutėje 935.

7.3.4.51 `shift_right()`

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::shift_right (
    size_type index,
    size_type count) [inline], [private]
```

Pastumia elementus į dešinę nuo `index`, atlaisvindama `count` pozicijų.

Elementai perkeliama naudojant `move_if_noexcept`.

Parametrai

<i>index</i>	Pozicija, nuo kurios elementai stumiami.
<i>count</i>	Laisvų pozicijų skaičius.

Apibrėžimas failo [vector.h](#) eilutėje 919.

7.3.4.52 `shrink_to_fit()`

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::shrink_to_fit () [inline]
```

Sumažina talpą iki esamo dydžio, atlaisvindama nenaudojamą atmintį.

Apibrėžimas failo [vector.h](#) eilutėje 591.

7.3.4.53 `size()`

```
template<class T, class Allocator = std::allocator<T>>
size_type Vector< T, Allocator >::size () const [inline]
```

Grąžina elementų skaičių.

Gražina

`size_type`

Apibrėžimas failo [vector.h](#) eilutėje 554.

7.3.4.54 `swap()`

```
template<class T, class Allocator = std::allocator<T>>
void Vector< T, Allocator >::swap (
    Vector< T, Allocator > & other) [inline], [noexcept]
```

Sukeičia turinį su `other` vektoriumi.

Parametrai

<i>other</i>	Kitas vektorius.
--------------	------------------

Apibrėžimas failo [vector.h](#) eilutėje 866.

7.3.5 Atributų Dokumentacija

7.3.5.1 alloc_

```
template<class T, class Allocator = std::allocator<T>>  
allocator_type Vector< T, Allocator >::alloc_ [private]
```

Atminties paskirstytojas.

Apibrėžimas failo [vector.h](#) eilutėje 877.

7.3.5.2 capacity_

```
template<class T, class Allocator = std::allocator<T>>  
size_type Vector< T, Allocator >::capacity_ = 0 [private]
```

Rezervuota atmintis (elementais).

Apibrėžimas failo [vector.h](#) eilutėje 876.

7.3.5.3 data_

```
template<class T, class Allocator = std::allocator<T>>  
pointer Vector< T, Allocator >::data_ = nullptr [private]
```

Rodyklė į elementų masyvą.

Apibrėžimas failo [vector.h](#) eilutėje 874.

7.3.5.4 size_

```
template<class T, class Allocator = std::allocator<T>>  
size_type Vector< T, Allocator >::size_ = 0 [private]
```

Esamas elementų skaičius.

Apibrėžimas failo [vector.h](#) eilutėje 875.

Dokumentacija šiai klasei sugeneruota iš šio failo:

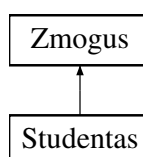
- [vector/vector.h](#)

7.4 Zmogus Klasė

Abstrakti bazinė klasė, turinti vardą ir pavardę.

```
#include <zmogus.h>
```

Paveldimumo diagrama Zmogus:



Vieši Metodai

- [Zmogus](#) ()
Numatytasis konstruktorius – tuščias vardas ir pavardė.
- [Zmogus](#) (const std::string v, const std::string p)
Konstruktorius su vardu ir pavarde.
- [Zmogus](#) (const [Zmogus](#) &kitas)
Kopijavimo konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) ([Zmogus](#) &&) noexcept
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&) noexcept
Perkėlimo priskyrimo operatorius.
- const std::string & [getVardas](#) () const
- const std::string & [getPavarde](#) () const
- void [setVardas](#) (std::string v)
Nustato vardą.
- void [setPavarde](#) (std::string p)
Nustato pavardę.
- virtual ~[Zmogus](#) ()=0
Virtualus destruktorius – padaro klasę abstrakčia.

Apsaugoti Atributai

- std::string [vardas](#)
Zmogaus vardas.
- std::string [pavarde](#)
Zmogaus pavardė.

7.4.1 Smulkus aprašymas

Abstrakti bazinė klasė, turinti vardą ir pavardę.

Klasė yra abstrakti dėl virtualaus destruktoriaus. Skirta paveldėjimui (pvz., [Studentas](#)).

Apibrėžimas failo [zmogus.h](#) eilutėje 13.

7.4.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

7.4.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus () [inline]
```

Numatytasis konstruktorius – tuščias vardas ir pavardė.

Apibrėžimas failo [zmogus.h](#) eilutėje 20.

7.4.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string v,
    const std::string p) [inline]
```

Konstruktorius su vardu ir pavarde.

Parametrai

v	Vardas.
---	---------

<i>p</i>	Pavardė.
----------	----------

Apibrėžimas failo [zmogus.h](#) eilutėje 27.

7.4.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & kitas)
```

Kopijavimo konstruktorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 4.

7.4.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && kitas) [noexcept]
```

Perkėlimo konstruktorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 21.

7.4.2.5 ~Zmogus()

```
Zmogus::~Zmogus () [pure virtual]
```

Virtualus destruktorius – padaro klasę abstrakčia.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 37.

7.4.3 Metodų Dokumentacija

7.4.3.1 getPavarde()

```
const std::string & Zmogus::getPavarde () const [inline]
```

Gražina

Pavarde (konstantinė nuoroda).

Apibrėžimas failo [zmogus.h](#) eilutėje 46.

7.4.3.2 getVardas()

```
const std::string & Zmogus::getVardas () const [inline]
```

Gražina

Vardą (konstantinė nuoroda).

Apibrėžimas failo [zmogus.h](#) eilutėje 43.

7.4.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & kitas)
```

Kopijavimo priskyrimo operatorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 10.

7.4.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && kitas) [noexcept]
```

Perkėlimo priskyrimo operatorius.

Apibrėžimas failo [zmogus.cpp](#) eilutėje 27.

7.4.3.5 setPavarde()

```
void Zmogus::setPavarde (
    std::string p) [inline]
```

Nustato pavardę.

Parametrai

<i>p</i>	Nauja pavardė.
----------	----------------

Apibrėžimas failo [zmogus.h](#) eilutėje 58.

7.4.3.6 setVardas()

```
void Zmogus::setVardas (
    std::string v) [inline]
```

Nustato vardą.

Parametrai

<i>v</i>	Naujas vardas.
----------	----------------

Apibrėžimas failo [zmogus.h](#) eilutėje 52.

7.4.4 Atributų Dokumentacija

7.4.4.1 pavarde

```
std::string Zmogus::pavarde [protected]
```

Zmogaus pavardė.

Apibrėžimas failo [zmogus.h](#) eilutėje 16.

7.4.4.2 vardas

```
std::string Zmogus::vardas [protected]
```

Zmogaus vardas.

Apibrėžimas failo [zmogus.h](#) eilutėje 15.

Dokumentacija šiai klasei sugeneruota iš šių failų:

- [vector/zmogus.h](#)
- [vector/zmogus.cpp](#)

skyrius 8

Failo Dokumentacija

8.1 gtest.cpp Failo Nuoroda

```
#include <gtest/gtest.h>
#include "../vector/studentas.h"
#include <sstream>
#include <memory>
```

Funkcijos

- [TEST](#) (RuleOfFive, CopyConstructor)
- [TEST](#) (RuleOfFive, CopyAssignment)
- [TEST](#) (RuleOfFive, MoveConstructor)
- [TEST](#) (RuleOfFive, MoveAssignment)
- [TEST](#) (RuleOfFive, Destructor)
- [TEST](#) (Functional, MedianCalculation)

8.1.1 Funkcijos Dokumentacija

8.1.1.1 TEST() [1/6]

```
TEST (
    Functional ,
    MedianCalculation )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 76.

8.1.1.2 TEST() [2/6]

```
TEST (
    RuleOfFive ,
    CopyAssignment )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 19.

8.1.1.3 TEST() [3/6]

```
TEST (
    RuleOfFive ,
    CopyConstructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 7.

8.1.1.4 TEST() [4/6]

```
TEST (
    RuleOfFive ,
    Destructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 65.

8.1.1.5 TEST() [5/6]

```
TEST (
    RuleOfFive ,
    MoveAssignment )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 48.

8.1.1.6 TEST() [6/6]

```
TEST (
    RuleOfFive ,
    MoveConstructor )
```

Apibrėžimas failo [gtest.cpp](#) eilutėje 32.

8.2 gtest.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include <gtest/gtest.h>
00002 #include "../vector/studentas.h"
00003 #include <sstream>
00004 #include <memory>
00005
00006 // 1. Kopijavimo konstruktorius
00007 TEST(RuleOfFive, CopyConstructor)
00008 {
00009     Studentas A("A", "AAA", {1, 2}, 10);
00010     Studentas B(A);
00011
00012     EXPECT_EQ(B.getVardas(), "A");
00013     EXPECT_EQ(B.getPavarde(), "AAA");
00014     EXPECT_EQ(B.getNd().size(), 2);
00015     EXPECT_EQ(B.getRez(), 10);
00016 }
00017
00018 // 2. Kopijavimo priskyrimas
00019 TEST(RuleOfFive, CopyAssignment)
00020 {
00021     Studentas A("A", "AAA", {1, 2}, 10);
00022     Studentas B;
```

```

00023     B = A;
00024
00025     EXPECT_EQ(B.getVardas(), "A");
00026     EXPECT_EQ(B.getPavarde(), "AAA");
00027     EXPECT_EQ(B.getNd().size(), 2);
00028     EXPECT_EQ(B.getRez(), 10);
00029 }
00030
00031 // 3. Perkëlimo konstruktorius
00032 TEST(RuleOfFive, MoveConstructor)
00033 {
00034     Studentas A("A", "AAA", {1, 2}, 10);
00035     Studentas B(std::move(A));
00036
00037     EXPECT_EQ(B.getVardas(), "A");
00038     EXPECT_EQ(B.getPavarde(), "AAA");
00039     EXPECT_EQ(B.getNd().size(), 2);
00040     EXPECT_EQ(B.getRez(), 10);
00041     EXPECT_TRUE(A.getVardas().empty());
00042     EXPECT_TRUE(A.getPavarde().empty());
00043     EXPECT_TRUE(A.getNd().empty());
00044     EXPECT_EQ(A.getRez(), 0);
00045 }
00046
00047 // 4. Perkëlimo priskyrimas
00048 TEST(RuleOfFive, MoveAssignment)
00049 {
00050     Studentas A("A", "AAA", {1, 2}, 10);
00051     Studentas B;
00052     B = std::move(A);
00053
00054     EXPECT_EQ(B.getVardas(), "A");
00055     EXPECT_EQ(B.getPavarde(), "AAA");
00056     EXPECT_EQ(B.getNd().size(), 2);
00057     EXPECT_EQ(B.getRez(), 10);
00058     EXPECT_TRUE(A.getVardas().empty());
00059     EXPECT_TRUE(A.getPavarde().empty());
00060     EXPECT_TRUE(A.getNd().empty());
00061     EXPECT_EQ(A.getRez(), 0);
00062 }
00063
00064 // 5. Destruktorius
00065 TEST(RuleOfFive, Destructor)
00066 {
00067     std::weak_ptr<Studentas> weak;
00068     {
00069         std::shared_ptr<Studentas> A = std::make_shared<Studentas>("A", "AAA", std::vector<int>{1,2},
10);
00070         weak = A;
00071     }
00072     EXPECT_TRUE(weak.expired());
00073 }
00074
00075 // 6. Papildomas: galutinis() su mediana
00076 TEST(Functional, MedianCalculation)
00077 {
00078     Studentas A("A", "AAA", {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, 10);
00079     double result = A.galutinis(true, 10);
00080     EXPECT_DOUBLE_EQ(result, 8.2);
00081 }

```

8.3 README.md Failo Nuoroda

8.4 vector/apdorojimas.cpp Failo Nuoroda

```
#include "apdorojimas.h"
```

Funkcijos

- void **failoGeneravimas** (int studentuKiekis, int ndKiekis)
Generuoja failą su studentų duomenimis.
- void **setw** (const std::string &tekstas, int plotis, std::string &out)
Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

8.4.1 Funkcijos Dokumentacija

8.4.1.1 failoGeneravimas()

```
void failoGeneravimas (
    int studentuKiekis,
    int ndKiekis)
```

Generuoja failą su studentų duomenimis.

Parametrai

<i>studentuKiekis</i>	Kiek studentų generuoti.
<i>ndKiekis</i>	Kiek namų darbų rezultatų generuoti kiekvienam studentui.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 3.

8.4.1.2 setw()

```
void setw (
    const std::string & tekstas,
    int plotis,
    std::string & out)
```

Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

Parametrai

<i>tekstas</i>	Tekstas
<i>plotis</i>	Minimalus eilutės plotis.
<i>out</i>	Nuoroda į išvesties eilutę.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 29.

8.5 apdorojimas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "apdorojimas.h"
00002
00003 void failoGeneravimas(int studentuKiekis, int ndKiekis)
00004 {
00005     std::string out;
00006     out.reserve(studentuKiekis*100/* 46+(ndKiekis*10)+studentuKiekis*(43+ndKiekis*10) */);
00007     setw("Vardas", 20, out);
00008     setw("Pavardė", 21, out);
00009     for(int i = 1; i <= ndKiekis; i++){
00010         setw(std::format("ND{} ", i), 10, out);
00011     }
00012     out += "Egz.\n";
00013     for(int i = 0; i < studentuKiekis; i++){
00014         Studentas A;
00015         A.readRandom(ndKiekis);
00016         setw(A.getVardas(), 20, out);
00017         setw(A.getPavarde(), 20, out);
00018         for(int j = 0; j < ndKiekis; j++){
```

```

00019         setw(std::to_string(A.getNd().at(j)), 10, out);
00020     }
00021     out += std::to_string(A.getRez()) + "\n";
00022 }
00023 std::string failoPavadinimas = std::string("studentai") + std::to_string(studentuKiekis) + ".txt";
00024 std::ofstream fout(failoPavadinimas);
00025 fout << out;
00026 fout.close();
00027 }
00028
00029 void setw(const std::string &tekstas, int plotis, std::string& out) {
00030     out += tekstas;
00031     if (tekstas.size() < plotis)
00032         out.append(plotis - tekstas.size(), ' ');
00033 };

```

8.6 vector/apdorojimas.h Failo Nuoroda

```

#include <algorithm>
#include <format>
#include "studentas.h"
#include "random.h"
#include <fstream>
#include <list>

```

Funkcijos

- void [failoGeneravimas](#) (int studentuKiekis, int ndKiekis)
Generuoja failą su studentų duomenimis.
- void [setw](#) (const std::string &tekstas, int plotis, std::string &out)
Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

8.6.1 Funkcijos Dokumentacija

8.6.1.1 failoGeneravimas()

```

void failoGeneravimas (
    int studentuKiekis,
    int ndKiekis)

```

Generuoja failą su studentų duomenimis.

Parametrai

<i>studentuKiekis</i>	Kiek studentų generuoti.
<i>ndKiekis</i>	Kiek namų darbų rezultatų generuoti kiekvienam studentui.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 3.

8.6.1.2 setw()

```
void setw (
    const std::string & tekstas,
    int plotis,
    std::string & out)
```

Prideda tekstą prie išvesties eilutės, sulygiuodama jį nurodytu pločiu.

Parametrai

<i>tekstas</i>	Tekstas
----------------	---------

<i>plotis</i>	Minimalus eilutės plotis.
<i>out</i>	Nuoroda į išvesties eilutę.

Apibrėžimas failo [apdorojimas.cpp](#) eilutėje 29.

8.7 apdorojimas.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef APDOROJIMAS_H
00002 #define APDOROJIMAS_H
00003
00004 #include <algorithm>
00005 #include <format>
00006 #include "studentas.h"
00007 #include "random.h"
00008 #include <fstream>
00009 #include <list>
00010
00016 void failoGeneravimas(int studentuKiekis, int ndKiekis);
00017
00024 void setw(const std::string &tekstas, int plotis, std::string& out);
00025
00026 #endif
```

8.8 vector/apdorojimas.hpp Failo Nuoroda

```
#include "apdorojimas.h"
```

Funkcijos

- `template<class T, class Konteineris>`
void [rusiavimasPagal](#) (Konteineris &studentai, T lambdaFunkcija, bool didejanciai=true)
Rūšiuoja konteinerio studentus pagal nurodytą lambda funkciją.
- `template<class Konteineris>`
void [rusiavimasSkirstymas](#) (Konteineris &studentai, int rPasirinkimas, bool medianos, int ndKiekis=0)
Rūšiuoja studentus pagal vartotojo pasirinkimą (vardą, pavardę, galutinį pažymį).
- `template<class Konteineris>`
void [skirstymas](#) (Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
Skirsto studentus į "galvočius" (≥ 5) ir "vargsiukus" (< 5) naudojant judinimą (move).
- `template<class Konteineris>`
void [skirstymasStrat1](#) (Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
1 strategija: skirstymas be perkėlimo (kopijavimas).
- `template<class Konteineris>`
void [skirstymasStrat2](#) (Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
2 strategija: naudoja `std::lower_bound`, kai konteineris jau surūšiuotas.
- `template<class Konteineris>`
void [skirstymasStrat3](#) (Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
3 strategija: naudoja `std::partition`.

8.8.1 Funkcijos Dokumentacija

8.8.1.1 rusiavimasPagal()

```
template<class T, class Konteineris>
void rusiavimasPagal (
    Konteineris & studentai,
    T lambdaFunkcija,
    bool didejanciai = true)
```

Rūšiuoja konteinerio studentus pagal nurodytą lambda funkciją.

Template Parameters

<i>T</i>	lambda funkcijos tipas.
<i>Konteineris</i>	(list, vector, deque).

Parametrai

<i>studentai</i>	Studentų konteineris.
<i>lambdaFunkcija</i>	Funkcija, grąžinanti rūšiavimo būdą.
<i>didejanciai</i>	Jei true – rūšiuoti didėjančiai, kitu atveju – mažėjančiai.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 12.

8.8.1.2 rusiavimasSkirstymas()

```
template<class Konteineris>
void rusiavimasSkirstymas (
    Konteineris & studentai,
    int rPasirinkimas,
    bool medianos,
    int ndKiekis = 0)
```

Rūšiuoja studentus pagal vartotojo pasirinkimą (vardą, pavardę, galutinį pažymį).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris.
<i>rPasirinkimas</i>	Rūšiavimo būdas (1-6).
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų kiekis (jei žinomas).

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 43.

8.8.1.3 skirstymas()

```
template<class Konteineris>
void skirstymas (
    Konteineris & studentai,
    Konteineris & galvociai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

Skirsto studentus į "galvočius" (≥ 5) ir "vargšiukus" (< 5) naudojant judinimą (move).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris (ištuštinamas).
<i>galvociai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 68.

8.8.1.4 skirstymasStrat1()

```
template<class Konteineris>
void skirstymasStrat1 (
    Konteineris & studentai,
    Konteineris & galvociai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

1 strategija: skirstymas be perkėlimo (kopijavimas).

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Studentų konteineris (ištuštinamas).
<i>galvociai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .

<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 95.

8.8.1.5 skirstymasStrat2()

```
template<class Konteineris>
void skirstymasStrat2 (
    Konteineris & studentai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

2 strategija: naudoja `std::lower_bound`, kai konteineris jau surūšiuotas.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Surūšiuotas studentų konteineris (išsaugomi "galvočiai", kurių galutinis ≥ 5).
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo [apdorojimas.tpp](#) eilutėje 121.

8.8.1.6 skirstymasStrat3()

```
template<class Konteineris>
void skirstymasStrat3 (
    Konteineris & studentai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

3 strategija: naudoja `std::partition`.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>studentai</i>	Surūšiuotas studentų konteineris (išsaugomi "galvočiai", kurių galutinis ≥ 5).
------------------	--

<i>vargsiukai</i>	Konteineris studentams, kurių galutinis <5.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo `apdorojimas.cpp` eilutėje 140.

8.9 apdorojimas.cpp

Eiti į šio failo dokumentaciją.

```

00001 #include "apdorojimas.h"
00002
00011 template<class T, class Konteineris>
00012 void rusiavimasPagal(Konteineris &studentai, T lambdaFunkcija, bool didejanciai = true)
00013 {
00014     if constexpr(std::is_same_v<Konteineris, std::list<Studentas>>){
00015         if(didejanciai){
00016             studentai.sort([lambdaFunkcija](const Studentas &A, const Studentas &B){
00017                 return lambdaFunkcija(A) < lambdaFunkcija(B);
00018             });
00019         } else {
00020             studentai.sort([lambdaFunkcija](const Studentas &A, const Studentas &B){
00021                 return lambdaFunkcija(A) > lambdaFunkcija(B);
00022             });
00023         }
00024     } else {
00025         std::sort(studentai.begin(), studentai.end(),
00026             [didejanciai, lambdaFunkcija](const Studentas &A, const Studentas &B)
00027             {
00028                 return didejanciai ? lambdaFunkcija(A) < lambdaFunkcija(B) : lambdaFunkcija(A) >
00029                     lambdaFunkcija(B);
00030             });
00031     }
00032 }
00033
00042 template<class Konteineris>
00043 void rusiavimasSkirstymas(Konteineris &studentai, int rPasirinkimas, bool medianos, int ndKiekis = 0){
00044     switch(rPasirinkimas){
00045         case 1: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00046             studentas.getVardas();}); break;
00047         case 2: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00048             studentas.getVardas();}, false); break;
00049         case 3: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00050             studentas.getPavarde();}); break;
00051         case 4: rusiavimasPagal(studentai, [](const Studentas &studentas){return
00052             studentas.getPavarde();}, false); break;
00053         case 5: rusiavimasPagal(studentai, [medianos, ndKiekis](const Studentas &studentas){return
00054             studentas.galutinis(medianos, ndKiekis);}); break;
00055         case 6: rusiavimasPagal(studentai, [medianos, ndKiekis](const Studentas &studentas){return
00056             studentas.galutinis(medianos, ndKiekis);}, false); break;
00057         default:
00058         {
00059             break;
00060         }
00061     }
00062 }
00063
00067 template<class Konteineris>
00068 void skirstymas(Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool
00069     medianos, int ndKiekis = 0)
00070 {
00071     for(Studentas &A : studentai)
00072     {
00073         if(A.galutinis(medianos, ndKiekis)<5){
00074             vargsiukai.push_back(std::move(A));
00075         } else {
00076             galvociai.push_back(std::move(A));
00077         }
00078     }
00079     studentai.clear();
00080     if constexpr(!std::is_same_v<Konteineris, std::list<Studentas>>){
00081         vargsiukai.shrink_to_fit();
00082         galvociai.shrink_to_fit();
00083     }
00084 }

```

```

00084
00094 template<class Konteineris>
00095 void skirstymasStrat1(Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool
medianos, int ndKiekis = 0)
00096 {
00097     for(Studentas &A : studentai)
00098     {
00099         if(A.galutinis(medianos, ndKiekis)<5){
00100             vargsiukai.push_back(A);
00101         } else {
00102             galvociai.push_back(A);
00103         }
00104     }
00105     studentai.clear();
00106     if constexpr(!std::is_same_v<Konteineris, std::list<Studentas>)){
00107         vargsiukai.shrink_to_fit();
00108         galvociai.shrink_to_fit();
00109     }
00110 }
00111
00120 template<class Konteineris>
00121 void skirstymasStrat2(Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis =
0){
00122     auto it = std::lower_bound(studentai.begin(), studentai.end(), 5, [medianos, ndKiekis](const
Studentas &A, int skaic)
00123     {
00124         return A.galutinis(medianos, ndKiekis) < skaic;
00125     }
00126     );
00127     vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(studentai.begin()),
std::make_move_iterator(it));
00128     studentai.erase(studentai.begin(), it);
00129 }
00130
00139 template<class Konteineris>
00140 void skirstymasStrat3(Konteineris &studentai, Konteineris &vargsiukai, bool medianos, int ndKiekis =
0){
00141     auto it = std::partition(studentai.begin(), studentai.end(),
00142     [medianos, ndKiekis](const Studentas& A){
00143         return A.galutinis(medianos, ndKiekis) >= 5;
00144     }
00145     );
00146     vargsiukai.insert(vargsiukai.end(), std::make_move_iterator(it),
std::make_move_iterator(studentai.end()));
00147     studentai.erase(it, studentai.end());
00148 }

```

8.10 vector/isvestis.cpp Failo Nuoroda

```

#include "isvestis.h"
#include "ivestis.h"

```

Funkcijos

- int **menu** ()
Parodo pagrindinį meniu ir grąžina vartotojo pasirinkimą.
- int **rusiavimoPasirinkimas** ()
Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.
- int **gautiTipaPasirinkima** ()
Pasirinkimas, kokį konteinerį naudoti testavimui.
- int **testavimoPasirinkimas** ()
Kiek kartų kartoti testavimą.
- int **strategijosPasirinkimas** ()
Pasirinkimas, kurią skirstymo strategiją naudoti.
- int **studentuPasirinkimas** ()
Įvedamas norimas studentų skaičius generavimui.
- int **ndPasirinkimas** ()

- Įvedamas namų darbų rezultatų skaičius generavimui.*
- bool [medianosUzklausa](#) ()
Ar skaičiuoti galutinį pažymį naudojant medianą?
- bool [failoUzklausa](#) ()
Ar išvesti rezultatus į failą?
- int [lietuviskosRaides](#) (const std::string &eilute)
Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).
- void [failoPasirinkimas](#) (int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const std::string &vieta)
Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

8.10.1 Funkcijos Dokumentacija

8.10.1.1 failoPasirinkimas()

```
void failoPasirinkimas (
    int & rezervas,
    bool & egzistuoja,
    std::string & failoPavadinimas,
    const std::string & vieta = ".")
```

Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

Parametrai

<i>rezervas</i>	Išvedamas rezervavimo dydis (jei failo pavadinime yra skaičius).
<i>egzistuoja</i>	Ar rastas bent vienas tinkamas failas.
<i>failoPavadinimas</i>	Pasirinkto failo pavadinimas.
<i>vieta</i>	direktorija, kurioje numatyta ieškoti failų (numatyta ".").

Apibrėžimas failo [isvestis.cpp](#) eilutėje 65.

8.10.1.2 failoUzklausa()

```
bool failoUzklausa ()
```

Ar išvesti rezultatus į failą?

Gražina

true – į failą, false – į terminalą.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 44.

8.10.1.3 gautiTipoPasirinkima()

```
int gautiTipoPasirinkima ()
```

Pasirinkimas, kokį konteinerį naudoti testavimui.

Gražina

1 – STL vector, 2 – deque, 3 – list, 4 - nuosavas vector.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 14.

8.10.1.4 lietuiskosRaides()

```
int lietuiskosRaides (  
    const std::string & eilute)
```

Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).

Parametrai

<i>eilutė</i>	- tekstas.
---------------	------------

Gražina

Papildomų baitų skaičius (eilutės ilgis – simbolių skaičius).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 50.

8.10.1.5 medianosUzklausa()

```
bool medianosUzklausa ()
```

Ar skaičiuoti galutinį pažymį naudojant medianą?

Gražina

true – medianą, false – vidurkį.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 39.

8.10.1.6 menu()

```
int menu ()
```

Parodo pagrindinį meniu ir gražina vartotojo pasirinkimą.

Gražina

Pasirinkimo numeris (1-9).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 4.

8.10.1.7 ndPasirinkimas()

```
int ndPasirinkimas ()
```

Įvedamas namų darbų rezultatų skaičius generavimui.

Gražina

nd rezultatų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 34.

8.10.1.8 rusiavimoPasirinkimas()

```
int rusiavimoPasirinkimas ()
```

Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.

Gražina

Pasirinkimas (1-6).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 9.

8.10.1.9 strategijosPasirinkimas()

```
int strategijosPasirinkimas ()
```

Pasirinkimas, kurią skirstymo strategiją naudoti.

Gražina

0-3.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 24.

8.10.1.10 studentuPasirinkimas()

```
int studentuPasirinkimas ()
```

Įvedamas norimas studentų skaičius generavimui.

Gražina

Studentų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 29.

8.10.1.11 testavimoPasirinkimas()

```
int testavimoPasirinkimas ()
```

Kiek kartų kartoti testavimą.

Gražina

Kartų skaičius.

Apibrėžimas failo `isvestis.cpp` eilutėje 19.

8.11 isvestis.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "isvestis.h"
00002 #include "ivestis.h"
00003
00004 int menu()
00005 {
00006     return gautiSkaiciu("Pasirinkite programos eiga:\n1 - ranka,\n2 - generuoti tik pažymius,\n3 -
    generuoti studentų vardus, pavardės ir pažymius,\n4 - skaityti studentus iš failo,\n5 - testavimas su
    failais,\n6 - generuoti failą,\n7 - testuoti klasę Studentas,\n8 - nuosavo vektoriaus ir STL
    vektoriaus palyginimas,\n9 - baigti darbą: ", 1, 9);
00007 }
00008
00009 int rusiavimoPasirinkimas()
00010 {
00011     return gautiSkaiciu("Pasirinkite pagal ką rūšiuoti: \n1 - vardą (A->Ž),\n2 - vardą (Ž->A),\n3 -
    pavardę (A->Ž),\n4 - pavardę (Ž->A),\n5 - galutinį pažymį didėjančiai (1->10),\n6 - galutinį pažymį
    mažėjančiai (10->1)\n", 1, 6);
00012 }
00013
00014 int gautiTipoPasirinkima()
00015 {
00016     return gautiSkaiciu("Koki konteinerio tipą norite testuoti?\n1 - STL vektorių,\n2 - deque,\n3 -
    list,\n4 - nuosavas vektorius: ", 1, 4);
00017 }
00018
00019 int testavimoPasirinkimas()
00020 {
00021     return gautiSkaiciu("Kiek kartų testuoti failą? (1-100):", 1, 100);
00022 }
00023
00024 int strategijosPasirinkimas()
00025 {
00026     return gautiSkaiciu("Pagal kokią strategiją testuoti?\n0 - pradiniam relize naudota
    strategija,\n1 - pirma strategija,\n2 - antra strategija,\n3 - trečia strategija:", 0, 3);
00027 }
00028
00029 int studentuPasirinkimas()
00030 {
00031     return gautiSkaiciu("Įveskite norimą generuoti studentų kiekį (1-10000000):", 1, 10000000);
00032 }
00033
00034 int ndPasirinkimas()
00035 {
00036     return gautiSkaiciu("Įveskite norimą generuoti namų darbų rezultatų kiekį (0-10000000): ", 0,
    10000000);
00037 }
00038
00039 bool medianosUzklausa()
00040 {
00041     return gautiPatvirtinima("Skaičiuoti tik medianas? Jei ne, tai galutinis rezultatas bus
    skaičiuojamas su vidurkiu");
00042 }
00043
00044 bool failoUzklausa()
00045 {
00046     return gautiPatvirtinima("Ar išvesti į terminalą? Jei ne, tai bus išvedama į rezultatai.txt
    failą");
00047 }
00048
```



```

00049 // apskaiciuoti kiek string su lietuviskais raidėmis sudaro baitu, kadangi viena lietuviska raide - 2
      baitai, o ne 1 baitas. Kitaip sakant vardas Ažuolas turi 7 raides, o jį sudaro 8 baitai, o setw mato
      baitus, tai jei setw(20), tai jis pridės 12 tusciu tarpu, o ne 13.
00050 int lietuviskosRaides(const std::string& eilute)
00051 {
00052     int simboliuKiekis = 0;
00053     for (char c : eilute)
00054     {
00055         // jei baitas neprasideda su 10xxxxxx, tai naujas simbolis
00056         if ((c & 0xC0) != 0x80)
00057         { // paprastas ASCII simbolis prasideda su 0, keliu baitu pvz lietuviskos raides prasideda su
11 arba 111 arba 1111, priklausomai nuo kodavimo | 0xC0 = 11000000, 0x80 = 10000000. & (AND) bit'u
      operacija atranda ar c prasideda su 0 ar 1. antras, trecias ar ketvirtas baitas UTF-8 kodavime visad
      prasides su 10xxxxxx
00058             simboliuKiekis++;
00059         }
00060     }
00061     // grazinam trukstama isvesties ploti.
00062     return eilute.length() - simboliuKiekis;
00063 }
00064
00065 void failoPasirinkimas(int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const
      std::string& vieta)
00066 {
00067     std::vector<std::filesystem::directory_entry> failai;
00068     std::vector<int> rezervai;
00069
00070     for(auto &failas : std::filesystem::directory_iterator(vieta))
00071     {
00072         if(!failas.is_regular_file())
00073             continue;
00074
00075         if(failas.path().extension() != ".txt")
00076             continue;
00077
00078         std::string vardas = failas.path().filename().string();
00079
00080         if(vardas == "rezultatai.txt" || vardas == "galvociiai.txt" || vardas == "vargsiukai.txt")
00081             continue;
00082
00083         if(vardas.find("studentai", 0) == 0)
00084         {
00085             std::string skaicius = vardas.substr(9, vardas.size() - 9 - 4);
00086
00087             if(skaicius.empty() || !std::all_of(skaicius.begin(), skaicius.end(), [](unsigned char
      simbolis)
00088                 {
00089                     return std::isdigit(simbolis);
00090                 })))
00091             {
00092                 rezervai.push_back(0);
00093             }
00094             else
00095             {
00096                 rezervai.push_back(std::stoi(skaicius));
00097             }
00098         } else {
00099             rezervai.push_back(0);
00100         }
00101
00102         failai.push_back(failas);
00103     }
00104
00105     if(failai.empty())
00106     {
00107         std::cout << "Nerasta tekstinių failų vietoje." << vieta << "\n";
00108         egzistuoja = false;
00109     } else {
00110
00111         std::cout << "Pasirinkite failą:\n";
00112         for(int i = 0; i < failai.size(); i++)
00113         {
00114             std::cout << (i + 1) << ": " << failai.at(i).path().filename().string() << "\n";
00115         }
00116
00117         int pasirinkimas = gautiSkaiciu("Įveskite failo numerį: ", 1, failai.size(), false);
00118         failoPavadinimas = failai.at(pasirinkimas-1).path().filename().string();
00119         rezervas = rezervai.at(pasirinkimas-1);
00120     }
00121 }

```

8.12 vector/isvestis.h Failo Nuoroda

```
#include "studentas.h"
#include "apdorojimas.h"
#include <iomanip>
#include <iostream>
#include <fstream>
#include <filesystem>
#include <format>
```

Funkcijos

- int [menu](#) ()
Parodo pagrindinį meniu ir grąžina vartotojo pasirinkimą.
- int [rusiavimoPasirinkimas](#) ()
Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.
- int [gautiTipaPasirinkima](#) ()
Pasirinkimas, kokį konteinerį naudoti testavimui.
- int [testavimoPasirinkimas](#) ()
Kiek kartų kartoti testavimą.
- int [strategijosPasirinkimas](#) ()
Pasirinkimas, kurią skirstymo strategiją naudoti.
- int [studentuPasirinkimas](#) ()
Įvedamas norimas studentų skaičius generavimui.
- int [ndPasirinkimas](#) ()
Įvedamas namų darbų rezultatų skaičius generavimui.
- bool [medianosUzklausa](#) ()
Ar skaičiuoti galutinį pažymį naudojant medianą?
- bool [failoUzklausa](#) ()
Ar išvesti rezultatus į failą?
- int [lietuviskosRaides](#) (const std::string &eilute)
Suskaičiuoja, kiek papildomų baidų užima lietuviškos raidės (UTF-8).
- void [failoPasirinkimas](#) (int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const std::string &vieta=".")
Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

8.12.1 Funkcijos Dokumentacija

8.12.1.1 failoPasirinkimas()

```
void failoPasirinkimas (
    int & rezervas,
    bool & egzistuoja,
    std::string & failoPavadinimas,
    const std::string & vieta = ".")
```

Leidžia vartotojui pasirinkti failą iš esamų .txt failų programos direktorijoje.

Parametrai

<i>rezervas</i>	Išvedamas rezervavimo dydis (jei failo pavadinime yra skaičius).
-----------------	--

<i>egzistuoja</i>	Ar rastas bent vienas tinkamas failas.
<i>failoPavadinimas</i>	Pasirinkto failo pavadinimas.
<i>vieta</i>	direktorija, kurioje numatyta ieškoti failų (numatyta ".").

Apibrėžimas failo [isvestis.cpp](#) eilutėje 65.

8.12.1.2 failoUzklausa()

```
bool failoUzklausa ()
```

Ar išvesti rezultatus į failą?

Gražina

true – į failą, false – į terminalą.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 44.

8.12.1.3 gautiTipPasirinkima()

```
int gautiTipPasirinkima ()
```

Pasirinkimas, kokį konteinerį naudoti testavimui.

Gražina

1 – STL vector, 2 – deque, 3 – list, 4 - nuosavas vector.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 14.

8.12.1.4 lietuiskosRaides()

```
int lietuiskosRaides (
    const std::string & eilute)
```

Suskaičiuoja, kiek papildomų baitų užima lietuviškos raidės (UTF-8).

Parametrai

<i>eilutė</i>	- tekstas.
---------------	------------

Gražina

Papildomų baitų skaičius (eilutės ilgis – simbolių skaičius).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 50.

8.12.1.5 medianosUzklausa()

```
bool medianosUzklausa ()
```

Ar skaičiuoti galutinį pažymį naudojant medianą?

Gražina

true – medianą, false – vidurkį.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 39.

8.12.1.6 menu()

```
int menu ()
```

Parodo pagrindinį meniu ir gražina vartotojo pasirinkimą.

Gražina

Pasirinkimo numeris (1-9).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 4.

8.12.1.7 ndPasirinkimas()

```
int ndPasirinkimas ()
```

Įvedamas namų darbų rezultatų skaičius generavimui.

Gražina

nd rezultatų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 34.

8.12.1.8 rusiavimoPasirinkimas()

```
int rusiavimoPasirinkimas ()
```

Vartotojo pasirinkimas, pagal ką rūšiuoti studentus.

Gražina

Pasirinkimas (1-6).

Apibrėžimas failo [isvestis.cpp](#) eilutėje 9.

8.12.1.9 strategijosPasirinkimas()

```
int strategijosPasirinkimas ()
```

Pasirinkimas, kurią skirstymo strategiją naudoti.

Gražina

0-3.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 24.

8.12.1.10 studentuPasirinkimas()

```
int studentuPasirinkimas ()
```

Įvedamas norimas studentų skaičius generavimui.

Gražina

Studentų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 29.

8.12.1.11 testavimoPasirinkimas()

```
int testavimoPasirinkimas ()
```

Kiek kartų kartoti testavimą.

Gražina

Kartų skaičius.

Apibrėžimas failo [isvestis.cpp](#) eilutėje 19.

8.13 isvestis.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef ISVESTIS_H
00002 #define ISVESTIS_H
00003
00004 #include "studentas.h"
00005 #include "apdorojimas.h"
00006 #include <iomanip>
00007 #include <iostream>
00008 #include <fstream>
00009 #include <filesystem>
00010 #include <format>
00011
00016 int menu();
00017
00022 int rusiavimoPasirinkimas();
00023
00028 int gautiTipoPasirinkima();
00029
00034 int testavimoPasirinkimas();
00035
00040 int strategijosPasirinkimas();
00041
00046 int studentuPasirinkimas();
00047
00052 int ndPasirinkimas();
00053
00058 bool medianosUzklausa();
00059
00064 bool failoUzklausa();
00065
00071 int lietuviskosRaides(const std::string& eilute);
00072
00080 void failoPasirinkimas(int &rezervas, bool &egzistuoja, std::string &failoPavadinimas, const
    std::string& vieta = ".");
00081
00082 #endif
```

8.14 vector/isvestis.hpp Failo Nuoroda

```
#include "isvestis.h"
```

Funkcijos

- `template<class Konteineris>`
`void isvestis (Konteineris &A, bool medianos, bool failas, const std::string &failoPavadinimas="rezultatai.txt",`
`int ndKiekis=0)`
Išveda studentų sąrašą į failą arba ekraną.

8.14.1 Funkcijos Dokumentacija

8.14.1.1 isvestis()

```
template<class Konteineris>
void isvestis (
    Konteineris & A,
    bool medianos,
    bool failas,
```

```
const std::string & failoPavadinimas = "rezultatai.txt",  
int ndKiekis = 0)
```

Išveda studentų sąrašą į failą arba ekraną.

Template Parameters

<i>Konteineris</i>	(list, vector, deque).
--------------------	------------------------

Parametrai

<i>A</i>	Studentų konteineris.
----------	-----------------------

<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>failas</i>	Jei true – išvesti į failą "rezultatai.txt", kitu atveju – į std::cout.
<i>failoPavadinimas</i>	Išvesties failo pavadinimas (numatyta "rezultatai.txt").
<i>ndKiekis</i>	Namų darbų skaičius.

Apibrėžimas failo `isvestis.hpp` eilutėje 13.

8.15 isvestis.hpp

Eiti į šio failo dokumentaciją.

```

00001 #include "isvestis.h"
00002
00012 template<class Konteineris>
00013 void isvestis(Konteineris &A, bool medianos, bool failas, const std::string &failoPavadinimas =
    "rezultatai.txt", int ndKiekis = 0)
00014 {
00015     std::string out;
00016
00017     setw("Vardas", 20, out);
00018     setw("Pavardė", 21, out);
00019     out += "Galutinis (Vid.) / Galutinis (Med.)\n";
00020     out += std::string(75, '-') + "\n";
00021     for(const Studentas &X : A)
00022     {
00023         int vardoPlotis = 20 + lietuviskosRaides(X.getVardas());
00024         int pavardesPlotis = 20 + lietuviskosRaides(X.getPavarde());
00025
00026         setw(X.getVardas(), vardoPlotis, out);
00027         setw(X.getPavarde(), pavardesPlotis, out);
00028         if (medianos)
00029         {
00030             setw("x.xx", 19, out);
00031             out += std::format("{:.2f}", X.galutinis(medianos, ndKiekis));
00032             out += '\n';
00033         }
00034         else
00035         {
00036             setw(std::format("{:.2f}", X.galutinis(medianos, ndKiekis)), 19, out);
00037             out += "y.yy\n";
00038         }
00039     }
00040     if(failas){
00041         std::ofstream fout(failoPavadinimas);
00042         fout << out;
00043         fout.close();
00044     } else {
00045         std::cout << out;
00046     }
00047 }
```

8.16 vector<Ivestis>.cpp Failo Nuoroda

```
#include "ivestis.h"
```

Funkcijos

- bool `gautiPatvirtinima` (const std::string &pranesimas)
Paprašo vartotojo patvirtinimo (y/n).
- std::vector< `Studentas` > `ivestiStudentus` ()
Nuskaito studentus iš standartinės įvesties iki EOF.

- int [gautiSkaiciu](#) (const std::string &pranešimas, int min, int max, bool galiButiTuscia)
Nuskaityto sveikąjį skaičių iš konsolės su patikrinimais.
- bool [arTikSkaicius](#) (const std::string &eilute)
Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.
- void [cinEOFgaudymas](#) ()
Apdoroja EOF atvejį cin sraute.

8.16.1 Funkcijos Dokumentacija

8.16.1.1 arTikSkaicius()

```
bool arTikSkaicius (
    const std::string & s)
```

Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.

Parametrai

s	Tikrinama eilutė.
---	-------------------

Gražina

true jei taip.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 76.

8.16.1.2 cinEOFgaudymas()

```
void cinEOFgaudymas ()
```

Apdoroja EOF atvejį cin sraute.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 85.

8.16.1.3 gautiPatvirtinima()

```
bool gautiPatvirtinima (
    const std::string & pranesimas)
```

Paprašo vartotojo patvirtinimo (y/n).

Parametrai

pranesimas	Tekstas, rodomas prieš įvedimą.
------------	---------------------------------

Gražina

true jei 'y', false jei 'n'.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 3.

8.16.1.4 gautiSkaiciu()

```
int gautiSkaiciu (
    const std::string & pranešimas,
    int min,
    int max,
    bool galiButiTuscia = false)
```

Nuskaityto sveikąjį skaičių iš konsolės su patikrinimais.

Parametrai

<i>pranešimas</i>	- tekstas, rodomas prieš įvedimą.
<i>min</i>	Mažiausia leistina reikšmė.
<i>max</i>	Didžiausia leistina reikšmė.
<i>galiButiTuscia</i>	Ar galima įvesti tuščią eilutę (grąžina -1).

Gražina

Įvestas skaičius.

Apibrėžimas failo [investis.cpp](#) eilutėje 43.

8.16.1.5 ivestiStudentus()

```
std::vector< Studentas > ivestiStudentus ()
```

Nuskaityto studentus iš standartinės įvesties iki EOF.

Gražina

Vektorius su nuskaitytais studentais.

Apibrėžimas failo [investis.cpp](#) eilutėje 29.

8.17 investis.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "investis.h"
00002
00003 bool gautiPatvirtinima(const std::string &pranesimas)
00004 {
00005     std::string ivestis;
00006     while (true) {
00007         std::cout << pranesimas << " (y/n): ";
00008
00009         if (!std::getline(std::cin, ivestis)) {
00010             cinEOFgaudymas();
00011             continue;
00012         }
00013
00014         ivestis.erase(0, ivestis.find_first_not_of(" \t")); // randa pirma simboli kuris nera tarpas
00015         arba tabuliacija, tada trina nuo 0-inio indekso iki rasto simbolio.
```

```

00015         ivistis.erase(ivistis.find_last_not_of(" \t") + 1); // randa pirma simboli kuris nera tarpas
           arba tabuliacija nuo galo ir istrina viska po to
00016
00017         // ivisties ilgio patikrinimas ir konvertavimas
00018         if (ivistis.length() == 1) {
00019             // iversti vercia i mazaja | to lower tikisi unsigned char arba EOF pagal standarta,
           static_cast keicia char interpretavima i unsigned char.
00020             if (std::tolower(static_cast<unsigned char>(ivistis[0])) == 'y') return true;
00021             if (std::tolower(static_cast<unsigned char>(ivistis[0])) == 'n') return false;
00022         }
00023
00024         // jei ivistis neteisinga, t.y. nieko nebuvo returninta, tai prompt'ina vartotoja vel iversti y
           ar n!
00025         std::cout << "Neteisinga iverstis! Prašome iversti tik 'y' arba 'n'.\\n";
00026     }
00027 }
00028
00029 std::vector<Studentas> iverstiStudentus()
00030 {
00031     std::vector<Studentas> studentai;
00032     while(true)
00033     {
00034         Studentas A;
00035         std::cin >> A;
00036         if(!std::cin) break;
00037         studentai.push_back(std::move(A));
00038     }
00039     std::cin.clear();
00040     return studentai;
00041 }
00042
00043 int gautiSkaiciu(const std::string &pranešimas, int min, int max, bool galiButiTuscia /* = false */)
00044 {
00045     std::string ivistis;
00046     while (true) {
00047         std::cout << pranešimas;
00048
00049         if (!std::getline(std::cin, ivistis)) {
00050             if (std::cin.eof())
00051                 cinEOFgaudymas();
00052             continue;
00053         }
00054
00055         // ivisties nutraukimas su ENTER
00056         if (galiButiTuscia && ivistis.empty()) return -1;
00057
00058         try {
00059             //
00060             if (!arTikSkaicius(ivistis)) throw std::invalid_argument("Ne skaičius");
00061
00062             int skaicius = std::stoi(ivistis);
00063
00064             //tikriname ar iverstas skaicius atitinka nuo maziausio leistino iki didžiausio leistino
00065             if (skaicius >= min && skaicius <= max) {
00066                 return skaicius;
00067             } else {
00068                 std::cout << "Klaida, skaičius turi būti tarp " << min << " ir " << max << "\\n";
00069             }
00070         } catch (...) {
00071             std::cout << "Klaida, įveskite sveikąjį skaičių!\\n";
00072         }
00073     }
00074 }
00075
00076 bool arTikSkaicius(const std::string& eilute)
00077 { // jei eilute tuscia grazinama false, std::all_of pereina nuo eilutes.begin() pradžios iki galo
           eilutes.end() per kiekviena simboli, kiekvienam simboliui jei jis skaicius ar tarpas grazina true, jei
           tai tiesiog raide – grazinama false ir toliau eilute nebetikrinama
00078     return !eilute.empty() &&
           std::all_of(eilute.begin(), eilute.end(), [](unsigned char simbolis)
00079     {
00080         return std::isdigit(simbolis) || std::isspace(simbolis);
00081     });
00082 }
00083
00084
00085 void cinEOFgaudymas()
00086 {
00087     if (std::cin.eof())
00088     { // apsauga nuo CTRL+D (linux), CTRL+Z (windows)
00089         throw std::runtime_error("Ivesties pabaiga (EOF). Darbas su programa baigtas");
00090     }
00091     std::cin.clear(); // atstatome std::cin fail flag'a
00092 }

```

8.18 vector/ivestis.h Failo Nuoroda

```
#include "studentas.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <algorithm>
#include "ivestis.tpp"
```

Funkcijos

- bool [gautiPatvirtinima](#) (const std::string &pranesimas)
Paprašo vartotojo patvirtinimo (y/n).
- std::vector< [Studentas](#) > [ivestiStudentus](#) ()
Nuskaityti studentus iš standartinės įvesties iki EOF.
- bool [skaitymas](#) ([Studentas](#) &A)
Nuskaityti vieną studentą iš srauto (naudojama skaitymui iš failo ar konsolės).
- bool [studentoVardoPavardesIvestis](#) ([Studentas](#) &A, const std::string &eilute)
Nuskaityti vardą ir pavardę iš eilutės.
- void [namuDarbuRezultatulvestis](#) ([Studentas](#) &A)
Nuskaityti namų darbų rezultatus (terminale).
- void [egzaminoRezultatulvestis](#) ([Studentas](#) &A)
Nuskaityti egzamino rezultatą (terminale).
- int [gautiSkaiciu](#) (const std::string &pranešimas, int min, int max, bool galiButiTuscia=false)
Nuskaityti sveikąjį skaičių iš konsolės su patikrinimais.
- bool [arTikSkaicius](#) (const std::string &s)
Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.
- template<class Konteineris>
Konteineris [skaitymasIsFailo](#) (const std::string &failoPavadinimas, int &ndKiekis, int rezervas)
Nuskaityti studentus iš failo.
- void [cinEOFgaudymas](#) ()
Apdoroja EOF atvejį cin sraute.

8.18.1 Funkcijos Dokumentacija

8.18.1.1 arTikSkaicius()

```
bool arTikSkaicius (
    const std::string & s)
```

Patikrina, ar eilutę sudaro tik skaitmenys ir tarpai.

Parametrai

s	Tikrinama eilutė.
---	-------------------

Gražina

true jei taip.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 76.

8.18.1.2 cinEOFgaudymas()

```
void cinEOFgaudymas ()
```

Apdoroja EOF atvejį cin sraute.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 85.

8.18.1.3 egzaminoRezultatIvestis()

```
void egzaminoRezultatoIvestis (
    Studentas & A)
```

Nuskaityto egzamino rezultatą (terminale).

Parametrai

A	Studento objektas.
---	--------------------

8.18.1.4 gautiPatvirtinima()

```
bool gautiPatvirtinima (
    const std::string & pranesimas)
```

Paprašo vartotojo patvirtinimo (y/n).

Parametrai

<i>pranesimas</i>	Tekstas, rodomas prieš įvedimą.
-------------------	---------------------------------

Gražina

true jei 'y', false jei 'n'.

Apibrėžimas failo [ivestis.cpp](#) eilutėje 3.

8.18.1.5 gautiSkaiciu()

```
int gautiSkaiciu (
    const std::string & pranešimas,
    int min,
    int max,
    bool galiButiTuscia = false)
```

Nuskaityto sveikąjį skaičių iš konsolės su patikrinimais.

Parametrai

<i>pranešimas</i>	- tekstas, rodomas prieš įvedimą.
-------------------	-----------------------------------

<i>min</i>	Mažiausia leistina reikšmė.
<i>max</i>	Didžiausia leistina reikšmė.
<i>galiButiTuscia</i>	Ar galima įvesti tuščią eilutę (grąžina -1).

Gražina

Įvestas skaičius.

Apibrėžimas failo [investis.cpp](#) eilutėje 43.

8.18.1.6 investStudentus()

```
std::vector< Studentas > investStudentus ()
```

Nuskaityto studentus iš standartinės įvesties iki EOF.

Gražina

Vektorius su nuskaitytais studentais.

Apibrėžimas failo [investis.cpp](#) eilutėje 29.

8.18.1.7 namuDaruRezultatulvestis()

```
void namuDaruRezultatulvestis (
    Studentas & A)
```

Nuskaityto namų darbų rezultatus (terminale).

Parametrai

<i>A</i>	Studento objektas.
----------	--------------------

8.18.1.8 skaitymas()

```
bool skaitymas (
    Studentas & A)
```

Nuskaityto vieną studentą iš srauto (naudojama skaitymui iš failo ar konsolės).

Parametrai

<i>A</i>	Studento objektas, kur bus įrašyti duomenys.
----------	--

Gražina

true jei nuskaitymas sėkmingas.

8.18.1.9 skaitymasIsFailo()

```
template<class Konteineris>
Konteineris skaitymasIsFailo (
    const std::string & failoPavadinimas,
    int & ndKiekis,
    int rezervas)
```

Nuskaito studentus iš failo.

Template Parameters

<i>Konteineris</i>	(STL vector, deque, list, nuosavas vector).
--------------------	---

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
<i>ndKiekis</i>	Išvestinis namų darbų skaičius (nustatomas iš antraštės).
<i>rezervas</i>	Preliminarus konteinerio rezervavimo dydis (jei palaikomas).

Gražina

Konteineris su nuskaitytais studentais.

Apibrėžimas failo [ivestis.tpp](#) eilutėje 10.

8.18.1.10 studentoVardoPavardesIvestis()

```
bool studentoVardoPavardesIvestis (
    Studentas & A,
    const std::string & eilute)
```

Nuskaito vardą ir pavardę iš eilutės.

Parametrai

<i>A</i>	Studento objektas.
<i>eilute</i>	Eilutė su vardu ir pavarde.

Gražina

true jei nuskaityta sėkmingai.

8.19 iverstis.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef IVESTIS_H
00002 #define IVESTIS_H
00003
00004 #include "studentas.h"
00005 #include <iostream>
00006 #include <fstream>
00007 #include <sstream>
00008 #include <algorithm>
00009 #include "iverstis.hpp"
00010
00016 bool gautiPatvirtinima(const std::string &pranesimas);
00017
00022 std::vector<Studentas> iverstiStudentus();
00023
00029 bool skaitymas(Studentas &A);
00030
00037 bool studentoVardoPavardesIvestis(Studentas &A, const std::string &eilute);
00038
00043 void namuDarbuRezultatuIvestis(Studentas &A);
00044
00049 void egzaminoRezultatuIvestis(Studentas &A);
00050
00059 int gautiSkaiciu(const std::string &pranešimas, int min, int max, bool galiButiTuscia = false);
00060
00066 bool arTikSkaicius(const std::string &s);
00067
00076 template<class Konteineris>
00077 Konteineris skaitymasIsFailo(const std::string &failoPavadinimas, int &ndKiekis, int rezervas);
00078
00082 void cinEOFgaudymas();
00083
00084 #endif

```

8.20 vector/iverstis.hpp Failo Nuoroda

Funkcijos

- template<class Konteineris>
Konteineris **skaitymasIsFailo** (const std::string &failoPavadinimas, int &ndKiekis, int rezervas)
Nuskaityti studentus iš failo.

8.20.1 Funkcijos Dokumentacija

8.20.1.1 skaitymasIsFailo()

```

template<class Konteineris>
Konteineris skaitymasIsFailo (
    const std::string & failoPavadinimas,
    int & ndKiekis,
    int rezervas)

```

Nuskaityti studentus iš failo.

Template Parameters

<i>Konteineris</i>	(STL vector, deque, list, nuosavas vector).
--------------------	---

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
-------------------------	--------------------

<i>ndKiekis</i>	Išvestinis namų darbų skaičius (nustatomas iš antraštės).
<i>rezervas</i>	Preliminarus konteinerio rezervavimo dydis (jei palaikomas).

Gražina

Konteineris su nuskaitytais studentais.

Apibrėžimas failo `ivestis.hpp` eilutėje 10.

8.21 ivestis.hpp

Eiti į šio failo dokumentaciją.

```

00001
00009 template<class Konteineris>
00010 Konteineris skaitymasIsFailo(const std::string &failoPavadinimas, int &ndKiekis, int rezervas){
00011     Konteineris studentai;
00012     if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);})
00013         studentai.reserve(rezervas);
00014     std::string eil, t;
00015
00016     std::ifstream open_f(failoPavadinimas);
00017     if (!open_f.is_open()) throw std::runtime_error("Klaida: failas \"" + failoPavadinimas + "\"
nerastas.");
00018
00019     if(!std::getline(open_f, eil)) throw std::runtime_error("Klaida: failas tuščias arba
netinkantis");
00020     std::stringstream antraste(eil);
00021     antraste >> t >> t;
00022     while(antraste >> t){
00023         if(t == "Egz." || t == "Egzaminas") break;
00024         ndKiekis++;
00025     }
00026
00027     std::string vardas, pavarde;
00028     int paz;
00029
00030     while (true) {
00031         Studentas studentas;
00032         if(!(studentas.readStudentStream(open_f, ndKiekis))) break;
00033         studentai.push_back(std::move(studentas));
00034     }
00035
00036     open_f.close();
00037     return studentai;
00038 }

```

8.22 vector/main.cpp Failo Nuoroda

```
#include "main.h"
```

Funkcijos

- int `main` ()
- void `failoApdorojimas` (const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos, int rPasirinkimas, bool failas)

Apdoroja failą: nuskaito, surūšiuoja, išveda.

8.22.1 Funkcijos Dokumentacija

8.22.1.1 failoApdorojimas()

```
void failoApdorojimas (
    const std::string & failoPavadinimas,
    int rezervas,
    int & ndKiekis,
    bool medianos,
    int rPasirinkimas,
    bool failas)
```

Apdoroja failą: nuskaityti, surūšiuoja, išveda.

Parametrai

<i>failoPavadinimas</i>	Failo, iš kurio skaityti, pavadinimas.
<i>rezervas</i>	Rezervavimo dydis konteneriui (jei žinomas).
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>rPasirinkimas</i>	Rūšiavimo būdas.
<i>failas</i>	Ar išvesti į failą (true) ar į ekraną (false).

Apibrėžimas failo [main.cpp](#) eilutėje 155.

8.22.1.2 main()

```
int main ()
```

Apibrėžimas failo [main.cpp](#) eilutėje 3.

8.23 main.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "main.h"
00002
00003 int main()
00004 {
00005     try
00006     {
00007         #ifdef _WIN32 // Jei kompiliuojama Windows operacinei sistemai nustatyti konsolės įvestį ir
išvestį UTF-8 užkodavimui.
00008         SetConsoleOutputCP(CP_UTF8); // pakeičiame išvesties code page į UTF-8
00009         SetConsoleCP(CP_UTF8); // pakeičiame įvesties code page į UTF-8
00010         #endif
00011         bool veikimas = true;
00012         while(veikimas)
00013         {
00014             //pasirinkima galima tobulint su enumeratorium del type safety ir jei butu norima valdyti
atminti.
00015             int pasirinkimas = menu();
00016             bool failas = false;
00017             bool medianos = false;
00018             if(pasirinkimas<5)
00019             {
00020                 failas = !failoUzklausa();
00021                 medianos = medianosUzklausa();
```

```

00022         } else if(pasirinkimas == 5){
00023             medianos = medianosUzklausa();
00024         }
00025
00026         switch(pasirinkimas){
00027             case 1: // rankinis ivedimas
00028             {
00029                 std::vector<Studentas> studentai = iverstiStudentus();
00030                 skaiciavimas(studentai, medianos); /*
00031                 isvestis(studentai, medianos, failas);
00032                 break;
00033             }
00034             case 2: // tik pazymiu generavimas.
00035             {
00036                 std::vector<Studentas> studentai = iverstiStudentusRandom(pasirinkimas);
00037                 skaiciavimas(studentai, medianos); /*
00038                 isvestis(studentai, medianos, failas);
00039                 break;
00040             }
00041             case 3: // studentu ir pazymiu generavimas;
00042             {
00043                 std::vector<Studentas> studentai = iverstiStudentusRandom(pasirinkimas);
00044                 skaiciavimas(studentai, medianos); /*
00045                 isvestis(studentai, medianos, failas);
00046                 break;
00047             }
00048             case 4: // skaitymas is failo
00049             {
00050                 int ndKiekis = 0;
00051                 int rezervas = 0;
00052                 std::string fPasirinkimas;
00053                 bool egzistuojaFailai = true;
00054                 failoPasirinkimas(rezervas, egzistuojaFailai, fPasirinkimas);
00055                 if(egzistuojaFailai == false) break;
00056                 int rPasirinkimas = rusiavimoPasirinkimas();
00057                 failoApdorojimas(fPasirinkimas, rezervas, ndKiekis, medianos, rPasirinkimas,
00058                 failas);
00059                 break;
00060             }
00061             case 5: // testavimas su failais // kiekviename test case'e uzkomentuota koda arba jo
00062                 dalis galima atkomentuoti bei keisti parametrus, kad pakeisti kas yra testuojama, kadangi tiksliai
00063                 neapibrezta pagal ka testuoti.
00064             {
00065                 int ndKiekis = 0;
00066                 int rezervas = 0;
00067                 std::string fPasirinkimas;
00068                 bool egzistuojaFailai = true;
00069                 failoPasirinkimas(rezervas, egzistuojaFailai, fPasirinkimas);
00070                 if(egzistuojaFailai == false) break;
00071                 int tipoPasirinkimas = gautiTipoPasirinkima();
00072                 int tPasirinkimas = testavimoPasirinkimas();
00073                 int sPasirinkimas = strategijosPasirinkimas();
00074                 if(tPasirinkimas <= 0) throw std::invalid_argument("Testavimo skaičius turi būti
00075                 daugiau už 0!");
00076                 switch(tipoPasirinkimas){
00077                     case 1:
00078                     {
00079                         failoTestavimas<std::vector<Studentas>>(fPasirinkimas, rezervas,
00080                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00081                         break;
00082                     }
00083                     case 2:
00084                     {
00085                         failoTestavimas<std::deque<Studentas>>(fPasirinkimas, rezervas,
00086                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00087                         break;
00088                     }
00089                     case 3:
00090                     {
00091                         failoTestavimas<std::list<Studentas>>(fPasirinkimas, rezervas,
00092                         tPasirinkimas, sPasirinkimas, ndKiekis, medianos);
00093                         break;
00094                     }
00095                     case 4:
00096                     {
00097                         failoTestavimas<Vector<Studentas>>(fPasirinkimas, rezervas, tPasirinkimas,
00098                         sPasirinkimas, ndKiekis, medianos);
00099                         break;
00100                     }
00101                     default:
00102                     {
00103                         }
00104                     }
00105                 }
00106                 break;
00107             }
00108             case 6: // failo generavimas

```

```

00101         {
00102             int studentuKiekis = studentuPasirinkimas();
00103             int ndKiekis = ndPasirinkimas();
00104             int tPasirinkimas = testavimoPasirinkimas();
00105             if(tPasirinkimas <= 0) throw std::invalid_argument("Testavimo skaičius turi būti
daugiau už 0!");
00106             double trukme = 0;
00107             for(int i = 0; i < tPasirinkimas+1; i++)
00108             {
00109                 if(i!=0)
00110                 {
00111                     Timer t;
00112                     failoGeneravimas(studentuKiekis, ndKiekis);
00113                     trukme += t.elapsed();
00114                 }
00115                 else
00116                 {
00117                     failoGeneravimas(studentuKiekis, ndKiekis);
00118                 }
00119             }
00120             std::cout << "Failų(-o) generavimas vidutiniškai užtruko: " << trukme/tPasirinkimas
<< " s\n";
00121             break;
00122         }
00123         case 7: // studento klasės testas
00124         {
00125             testas();
00126             break;
00127         }
00128         case 8: // nuosavo vektoriaus ir STL vektoriaus testas
00129         {
00130             vector_compare();
00131             break;
00132         }
00133         case 9: // darbo baigtis
00134         {
00135             std::cout << "Darbas su programa baigtas.";
00136             veikimas = false;
00137             return 0;
00138         }
00139         default:
00140         {
00141             std::cout << "How did we get here?" << std::endl; //
https://minecraft.wiki/w/Tutorial:Advancement\_guide/How\_Did\_We\_Get\_Here%3F
00142             return 0;
00143         }
00144     }
00145 }
00146 }
00147 catch(const std::exception &klaida)
00148 {
00149     std::cerr << klaida.what() << "\n";
00150     return 1;
00151 }
00152 return 0;
00153 }
00154
00155 void failoApdorojimas(const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos,
int rPasirinkimas, bool failas)
00156 {
00157     Timer t;
00158     std::vector<Studentas> studentai = skaitymasIsFailo<std::vector<Studentas>>(failoPavadinimas,
ndKiekis, rezervas);
00159     double skaitymoTrukme = t.elapsed(); // Skirtumas (s)
00160     /* t.reset();
00161     skaiciavimas(studentai, medianos, ndKiekis);
00162     double skaiciavimoTrukme = t.elapsed(); */
00163     t.reset();
00164     rusiavimasSkirstymas(studentai, rPasirinkimas, medianos);
00165     double rusiavimoTrukme = t.elapsed();
00166     t.reset();
00167     isvestis(studentai, medianos, failas);
00168     double isvedimoTrukme = t.elapsed(); // Skirtumas (s)
00169     std::cout << "Failo nuskaitymas į studentai vektorių užtruko: " << skaitymoTrukme << " s\n";
00170     /* std::cout << "Rezultatų skaičiavimas užtruko: " << skaiciavimoTrukme << " s\n"; */
00171     std::cout << "Duomenų rūšiavimas pagal pasirinktą parametą užtruko: " << rusiavimoTrukme << " s\n";
00172     std::cout << "Studentų išvedimas užtruko: " << isvedimoTrukme << " s\n";
00173     std::cout << "Bendra trukmė: " << skaitymoTrukme + /* skaiciavimoTrukme + */ rusiavimoTrukme +
isvedimoTrukme << " s\n";
00174 }

```

8.24 vector/main.h Failo Nuoroda

```
#include "investis.h"
#include "isvestis.h"
#include "apdorojimas.h"
#include "random.h"
#include "Timer.h"
#include "test.h"
#include "main.tpp"
#include <deque>
#include <list>
#include "vector_compare.h"
#include "vector.h"
```

Funkcijos

- void [failoApdorojimas](#) (const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos, int rPasirinkimas, bool failas)

Apdoroja failą: nuskaityti, surūšiuoja, išveda.

8.24.1 Funkcijos Dokumentacija

8.24.1.1 failoApdorojimas()

```
void failoApdorojimas (
    const std::string & failoPavadinimas,
    int rezervas,
    int & ndKiekis,
    bool medianos,
    int rPasirinkimas,
    bool failas)
```

Apdoroja failą: nuskaityti, surūšiuoja, išveda.

Parametrai

<i>failoPavadinimas</i>	Failo, iš kurio skaityti, pavadinimas.
<i>rezervas</i>	Rezervavimo dydis konteneriui (jei žinomas).
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>rPasirinkimas</i>	Rūšiavimo būdas.
<i>failas</i>	Ar išvesti į failą (true) ar į ekraną (false).

Apibrėžimas failo [main.cpp](#) eilutėje 155.

8.25 main.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef MAIN_H
00002 #define MAIN_H
00003
00004 //Header file includes
00005 #include "investis.h"
00006 #include "isvestis.h"
00007 #include "apdorojimas.h"
00008 #include "random.h"
00009 #include "Timer.h"
00010 #include "test.h"
00011 #include "main.tpp"
00012 #include <deque>
00013 #include <list>
00014 #include "vector_compare.h"
00015 #include "vector.h"
00016
00017 #ifdef _WIN32 // naudojame preprocesorių, kad kompiliatorius, naudojant Windows, pridėtų windows.h
               // antraščių failą, kad vėliau galėtume pakeisti terminalo išvesties ir įvesties užkodavimą į UTF-8
00018 #include <windows.h> // windows antraščių failas
00019 #endif
00020
00030 void failoApdorojimas(const std::string &failoPavadinimas, int rezervas, int &ndKiekis, bool medianos,
                       int rPasirinkimas, bool failas);
00031
00032 #endif
```

8.26 vector/main.tpp Failo Nuoroda

```
#include "main.h"
#include "apdorojimas.tpp"
#include "isvestis.tpp"
```

Funkcijos

- `template<class Konteineris>`
`void skirstymoPasirinkimas` (int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai)
Pasirenka skirstymo strategiją.
- `template<class Konteineris>`
`void failoTestavimas` (const std::string &failoPavadinimas, int rezervas, int tPasirinkimas, int sPasirinkimas, int &ndKiekis, bool medianos)
Atlieka failo testavimą su duotu konteinerio tipu.
- `template<class Konteineris>`
`void skirstymoPasirinkimas` (int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai, Konteineris &vargsiukai, bool medianos, int ndKiekis=0)
Įgyvendina skirstymo strategijos pasirinkimą.

8.26.1 Funkcijos Dokumentacija

8.26.1.1 failoTestavimas()

```
template<class Konteineris>
void failoTestavimas (
```

```
const std::string & failoPavadinimas,
int rezervas,
int tPasirinkimas,
int sPasirinkimas,
int & ndKiekis,
bool medianos)
```

Atlieka failo testavimą su duotu konteinerio tipu.

Template Parameters

<i>Konteineris</i>	(STL vector, deque, list, nuosavas vector).
--------------------	---

Parametrai

<i>failoPavadinimas</i>	Failo pavadinimas.
<i>rezervas</i>	Rezervavimo dydis.
<i>tPasirinkimas</i>	Kiek kartų testuoti.
<i>sPasirinkimas</i>	Skirstymo strategija.
<i>ndKiekis</i>	Namų darbų skaičius.
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.

Apibrėžimas failo [main.hpp](#) eilutėje 29.

8.26.1.2 skirstymoPasirinkimas() [1/2]

```
template<class Konteineris>
void skirstymoPasirinkimas (
    int sPasirinkimas,
    Konteineris & studentai,
    Konteineris & galvociiai,
    Konteineris & vargsiukai)
```

Pasirenka skirstymo strategiją.

Template Parameters

<i>Konteineris</i>	(list, STL vector, deque, nuosavas vector).
--------------------	---

Parametrai

<i>sPasirinkimas</i>	Strategijos numeris (0-3).
<i>studentai</i>	Studentų konteineris.
<i>galvociiai</i>	Konteineris studentams, kurių galutinis ≥ 5 .
<i>vargsiukai</i>	Konteineris studentams, kurių galutinis < 5 .
<i>medianos</i>	Ar naudoti medianą galutiniam pažymiui.
<i>ndKiekis</i>	Namų darbų skaičius.

8.26.1.3 skirstymoPasirinkimas() [2/2]

```
template<class Konteineris>
void skirstymoPasirinkimas (
    int sPasirinkimas,
    Konteineris & studentai,
    Konteineris & galvociai,
    Konteineris & vargsiukai,
    bool medianos,
    int ndKiekis = 0)
```

Igyvendina skirstymo strategijos pasirinkimą.

Template Parameters

<i>Konteineris</i>	(STL vector, deque, list, nuosavas vector).
--------------------	---

Apibrėžimas failo [main.tpp](#) eilutėje 92.

8.27 main.tpp

Eiti į šio failo dokumentaciją.

```
00001 #include "main.h"
00002 #include "apdorojimas.tpp"
00003 #include "isvestis.tpp"
00004
00015 template <class Konteineris>
00016 void skirstymoPasirinkimas(int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai,
    Konteineris &vargsiukai);
00017
00028 template <class Konteineris>
00029 void failoTestavimas(const std::string &failoPavadinimas, int rezervas, int tPasirinkimas, int
    sPasirinkimas, int &ndKiekis, bool medianos)
00030 {
00031     double skaitymoTrukme = 0;
00032     double skaiciavimoTrukme = 0;
00033     double rusiavimoTrukme = 0;
00034     double skirstymoTrukme = 0;
00035     double isvedimoTrukme1 = 0;
00036     double isvedimoTrukme2 = 0;
00037     ndKiekis = 0;
00038     Konteineris studentai = skaitymasIsFailo<Konteineris>(failoPavadinimas, ndKiekis, rezervas); //
nuskaitymas
00039 /*     skaiciavimas<Konteineris>(studentai, medianos, ndKiekis); */// rezultatu apsiskaiciavimas
    pries rikiavima
00040     rusiavimasSkirstymas(studentai, 5, medianos, ndKiekis); // rusiavimas didejanciai
00041     Konteineris vargsiukai;
00042     Konteineris galvociai;
00043     if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);}){
00044         galvociai.reserve(studentai.size());
00045         vargsiukai.reserve(studentai.size());
00046     }
00047     skirstymoPasirinkimas(sPasirinkimas, studentai, galvociai, vargsiukai, medianos, ndKiekis);
00048     /*     isvestis(galvociai, medianos, true, "galvociai.txt");
00049         isvestis(vargsiukai, medianos, true, "vargsiukai.txt"); */
00050     for(int i = 0; i < tPasirinkimas; i++)
00051     {
00052         ndKiekis = 0;
00053         Timer t;
00054         Konteineris studentai = skaitymasIsFailo<Konteineris>(failoPavadinimas, ndKiekis, rezervas);
// nuskaitymas
00055         skaitymoTrukme += t.elapsed(); // Skirtumas (s)
00056         /*     t.reset();
00057         skaiciavimas<Konteineris>(studentai, medianos, ndKiekis); // rezultatu apsiskaiciavimas pries
    rikiavima
00058         skaiciavimoTrukme += t.elapsed(); */
00059         t.reset();
00060         rusiavimasSkirstymas(studentai, 5, medianos, ndKiekis); // rusiavimas didejanciai
00061         rusiavimoTrukme += t.elapsed();
```



```

00062         t.reset();
00063         Konteineris vargsiukai;
00064         Konteineris galvociai;
00065         if constexpr(requires(Konteineris konteineris){konteineris.reserve(0);}){
00066             galvociai.reserve(studentai.size());
00067             vargsiukai.reserve(studentai.size());
00068         }
00069         skirstymoPasirinkimas(sPasirinkimas, studentai, galvociai, vargsiukai, medianos, ndKiekis);
00070         skirstymoTrukme += t.elapsed();
00071         t.reset();
00072         ivestis(galvociai, medianos, true, "galvociai.txt");
00073         isvedimoTrukme1 += t.elapsed(); // Skirtumas (s)
00074         t.reset();
00075         isvestis(vargsiukai, medianos, true, "vargsiukai.txt");
00076         isvedimoTrukme2 += t.elapsed(); // Skirtumas (s)
00077     }
00078     std::cout << "Failo nuskaitymas į studentai konteinerį vidutiniškai užtruko: " <<
skaitymoTrukme/tPasirinkimas << " s\n";
00079     /* std::cout << "Rezultatų skaičiavimas vidutiniškai užtruko: " << skaiciavimoTrukme/tPasirinkimas <<
" s\n"; */
00080     std::cout << "Duomenų rūšiavimas didėjančiai vidutiniškai užtruko: " <<
rusiavimoTrukme/tPasirinkimas << " s\n";
00081     std::cout << "Studentų skirstymas pagal pažymius vidutiniškai užtruko: " <<
skirstymoTrukme/tPasirinkimas << " s\n";
00082     std::cout << "Studentų išvedimas į galvociai.txt vidutiniškai užtruko: " <<
isvedimoTrukme1/tPasirinkimas << " s\n";
00083     std::cout << "Studentų išvedimas į vargsiukai.txt vidutiniškai užtruko: " <<
isvedimoTrukme2/tPasirinkimas << " s\n";
00084     std::cout << "Bendra trukmė: " << skaitymoTrukme + skaiciavimoTrukme + rusiavimoTrukme +
skirstymoTrukme + isvedimoTrukme1 + isvedimoTrukme2 << " s\n";
00085 }
00086
00091 template <class Konteineris>
00092 void skirstymoPasirinkimas(int sPasirinkimas, Konteineris &studentai, Konteineris &galvociai,
Konteineris &vargsiukai, bool medianos, int ndKiekis = 0)
00093 {
00094     switch (sPasirinkimas)
00095     {
00096     case 0:
00097     {
00098         skirstymas(studentai, galvociai, vargsiukai, medianos, ndKiekis);
00099         break;
00100     }
00101     case 1:
00102     {
00103         skirstymasStrat1(studentai, galvociai, vargsiukai, medianos, ndKiekis);
00104         break;
00105     }
00106     case 2:
00107     {
00108         skirstymasStrat2(studentai, vargsiukai, medianos, ndKiekis);
00109         break;
00110     }
00111     case 3:
00112     {
00113         skirstymasStrat3(studentai, vargsiukai, medianos, ndKiekis);
00114         break;
00115     }
00116     default:
00117     {
00118         break;
00119     }
00120     }
00121 }

```

8.28 vector/random.cpp Failo Nuoroda

```

#include "random.h"
#include "ivestis.h"

```

Funkcijos

- `std::vector< Studentas > ivesitiStudentusRandom` (int pasirinkimas)
Generuoja studentų sąrašą su atsitiktiniais duomenimis.
- `std::mt19937 & rng` ()
Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

8.28.1 Funkcijos Dokumentacija

8.28.1.1 iverstiStudentusRandom()

```
std::vector< Studentas > iverstiStudentusRandom (
    int pasirinkimas)
```

Generuoja studentų sąrašą su atsitiktiniais duomenimis.

Parametrai

<i>pasirinkimas</i>	2 – generuoti tik pažymius (vardai/pavardės įvedami ranka), 3 – generuoti viską.
---------------------	--

Gražina

Vektorius su sugeneruotais studentais.

Apibrėžimas failo [random.cpp](#) eilutėje 4.

8.28.1.2 rng()

```
std::mt19937 & rng ()
```

Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

Gražina

Nuoroda į generatorių.

Apibrėžimas failo [random.cpp](#) eilutėje 40.

8.29 random.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "random.h"
00002 #include "investis.h"
00003
00004 std::vector<Studentas> iverstiStudentusRandom(int pasirinkimas)
00005 {
00006     std::vector<Studentas> studentai;
00007     switch(pasirinkimas)
00008     {
00009         case 2:
00010             {
00011                 Studentas A;
00012                 while(A.readSemiRandom())
00013                 {
00014                     studentai.push_back(std::move(A));
00015                     A = Studentas{};
00016                 }
00017                 break;
00018             }
00019         case 3:
00020             {
00021                 int studKiekis = gautiSkaiciu("Įveskite norimą generuoti studentų kiekį: ", 0, 100);
00022                 studentai.reserve(studKiekis);
00023                 for (int i = 0; i < studKiekis; i++)
```

```

00024         {
00025             Studentas A;
00026             int ndKiekis = gautiSkaiciu("Įveskite norimą generuoti namų darbų rezultatų kiekį: ",
0, 100);
00027             A.readRandom(ndKiekis);
00028             studentai.push_back(std::move(A));
00029         }
00030         break;
00031     }
00032     default:
00033     {
00034         std::cout << "IvestiStudentusRandom default atvejis" << std::endl;
00035     }
00036 }
00037 return studentai;
00038 }
00039
00040 std::mt19937& rng() {
00041     static std::mt19937 rng(std::random_device{}());
00042     return rng;
00043 }

```

8.30 vector/random.h Failo Nuoroda

```

#include "studentas.h"
#include <random>

```

Funkcijos

- `std::vector< Studentas > ivesitiStudentusRandom` (int pasirinkimas)
Generuoja studentų sąrašą su atsitiktiniais duomenimis.
- `std::mt19937 & rng ()`
Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

8.30.1 Funkcijos Dokumentacija

8.30.1.1 ivesitiStudentusRandom()

```

std::vector< Studentas > ivesitiStudentusRandom (
    int pasirinkimas)

```

Generuoja studentų sąrašą su atsitiktiniais duomenimis.

Parametrai

<i>pasirinkimas</i>	2 – generuoti tik pažymius (vardai/pavardės įvedami ranka), 3 – generuoti viską.
---------------------	--

Gražina

Vektorius su sugeneruotais studentais.

Apibrėžimas failo `random.cpp` eilutėje 4.

8.30.1.2 rng()

```
std::mt19937 & rng ()
```

Gražina statinį Mersenne Twister atsitiktinių skaičių generatorių.

Gražina

Nuoroda į generatorių.

Apibrėžimas failo [random.cpp](#) eilutėje 40.

8.31 random.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef RANDOM_H
00002 #define RANDOM_H
00003 #include "studentas.h"
00004 #include <random>
00005
00012 std::vector<Studentas> iverstiStudentusRandom(int pasirinkimas);
00013
00018 std::mt19937& rng();
00019
00020 #endif
```

8.32 vector/studentas.cpp Failo Nuoroda

```
#include "studentas.h"
#include "investis.h"
#include "random.h"
#include <algorithm>
#include <numeric>
#include <iostream>
#include <fstream>
#include <random>
#include <chrono>
#include <utility>
```

Funkcijos

- `std::istream & operator>>` (`std::istream &is`, `Studentas &A`)
- `std::ostream & operator<<` (`std::ostream &out`, `const Studentas &A`)

8.32.1 Funkcijos Dokumentacija

8.32.1.1 operator<<()

```
std::ostream & operator<< (
    std::ostream & out,
    const Studentas & A)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 261.

8.32.1.2 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & A)
```

Apibrėžimas failo [studentas.cpp](#) eilutėje 255.

8.33 studentas.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "studentas.h"
00002 #include "investis.h"
00003 #include "random.h"
00004 #include <algorithm>
00005 #include <numeric>
00006 #include <iostream>
00007 #include <fstream>
00008 #include <random>
00009 #include <chrono>
00010 #include <utility>
00011
00012 void Studentas::addNd(int paz)
00013 {
00014     nd.push_back(paz);
00015 }
00016
00017 void Studentas::resizeNd(int n, int skaicius)
00018 {
00019     nd.resize(n, skaicius);
00020 };
00021
00022 double Studentas::galutinis(bool medianos, int ndKiekis) const
00023 {
00024     const int max = (ndKiekis != 0) ? ndKiekis : maxNdKiekis;
00025     if(nd.empty()) return 0.6*rez;
00026     if(medianos)
00027     {
00028         std::vector<int> ND = nd;
00029         if(ND.size() < max) ND.resize(max, 0);
00030         std::sort(ND.begin(), ND.end());
00031         const int vidurys = max / 2;
00032         std::nth_element(ND.begin(), ND.begin() + vidurys, ND.end());
00033         double med;
00034         if(max % 2 == 0)
00035         {
00036             med = (*std::max_element(ND.begin(), ND.begin() + vidurys) + ND[vidurys]) / 2.0;
00037         }
00038         else
00039         {
00040             med = ND[vidurys];
00041         }
00042         return 0.4 * med + 0.6 * rez;
00043     }
00044     else
00045     {
00046         int sum = std::accumulate(nd.begin(), nd.end(), 0);
00047
00048         double vid = (double)sum / max;
00049         return 0.4 * vid + 0.6 * rez;
00050     }
00051 }
00052
00053 std::istream& Studentas::readStudentStream(std::istream& is, int ndKiekis)
00054 {
00055     if(!(is >> vardas >> pavarde)) return is;
00056     int paz;
00057     if(ndKiekis == 0)
00058     {
00059         std::vector<int> skaiciai;
00060         while(is >> paz)
00061         {
00062             skaiciai.push_back(paz);
00063         }
00064         if(skaiciai.empty()){
00065             throw std::runtime_error("Klaida: įvesty nerasta nei namų darbų, nei egzamino pažymių");
00066         }
00067     }
00068 }
```

```

00067         rez = skaiciai.back();
00068         skaiciai.pop_back();
00069         nd = skaiciai;
00070     }
00071     else
00072     {
00073         nd.clear();
00074         nd.reserve(ndKiekis);
00075         for(int i = 0; i < ndKiekis; i++)
00076         {
00077             if(!(is » paz)) throw std::runtime_error("Klaida: netinkami pažymiai faile.");
00078             nd.push_back(paz);
00079         }
00080         if(!(is » rez)) throw std::runtime_error("Klaida: netinkamas egzamino rezultatas faile.");
00081     }
00082     return is;
00083 }
00084
00085 std::istream& Studentas::readStudentConsole(std::istream& is){
00086     std::string eilute;
00087     std::cout << "Įveskite studento vardą bei pavardę (ENTER - nutraukti įvedimą): ";
00088
00089     // perskaito eilutę ir jei perskaitymas nesekmingas, tai žiuri ar std::cin.eof, jei taip, tai
    programa užbaigiama, jei ne, tai išvalo investies stream'o veliaveles ir vel prasoma ivesti
00090     if (!std::getline(std::cin, eilute))
00091     {
00092         cinEOFgaudymas();
00093         return is;
00094     } // jei enter - iseina
00095     if(eilute.empty())
00096     {
00097         is.setstate(std::ios::failbit);
00098         return is;
00099     }
00100     while(!studentoVardoPavardesIvestis(eilute))
00101     {
00102         std::cout << "Įveskite studento vardą bei pavardę (ENTER - baigti): ";
00103         if(!std::getline(std::cin, eilute) || eilute.empty()) return is;
00104     }
00105     namuDarbuRezultatuIvestis();
00106     egzaminoRezultatoIvestis();
00107     return is;
00108 }
00109
00110 bool Studentas::studentoVardoPavardesIvestis(const std::string& eilute)
00111 {
00112     std::istringstream iss(eilute);
00113     if (!(iss » vardas » pavarde))
00114     {
00115         std::cout << "Įveskite vardą ir pavardę (du žodžiai)!\n";
00116         return false;
00117     }
00118     // tikriname ar po vardo ir pavardės yra dar žodžių
00119     std::string ekstra;
00120     if (iss » ekstra)
00121     {
00122         std::cout << "Įvesta per daug žodžių -- reikia tik vardo ir pavardės!\n";
00123         return false;
00124     }
00125     return true;
00126 }
00127
00128 void Studentas::namuDarbuRezultatuIvestis()
00129 {
00130     std::cout << "Įveskite " << maxNdKiekis << " namų darbų rezultatų." << std::endl;
00131     while (nd.size() < maxNdKiekis)
00132     {
00133         int balas = gautiSkaiciu("Įveskite namų darbų rezultatą nuo 1 iki 10 (ENTER - baigti): ", 1,
00134 10, true);
00135         if (balas == -1) break;
00136         nd.push_back(balas);
00137     }
00138     if(nd.size()==maxNdKiekis)
00139     {
00140         std::cout << "Įvestas didžiausias namų darbų rezultatų kiekis" << std::endl;
00141     }
00142 }
00143
00144 void Studentas::egzaminoRezultatoIvestis()
00145 {
00146     rez = gautiSkaiciu("Įveskite egzamino rezultatą (1-10): ", 1, 10);
00147 }
00148
00149 bool Studentas::readSemiRandom()
00150 {
00151     std::string eilute;

```

```

00152     std::cout << "Įveskite studento vardą bei pavardę (ENTER - nutraukti įvedimą): ";
00153
00154     // perskaity eilutę ir jei perskaitymas nesekmingas, tai žiuri ar std::cin.eof, jei taip, tai
    programa uzbaiigiama, jei ne, tai išvalo įvesties stream'o vėliavėles ir vėl prasoma įvesti
00155     if (!std::getline(std::cin, eilutė))
00156     {
00157         cinEOFgaudymas();
00158         return false;
00159     } // jei enter - išeina
00160     if (eilutė.empty())
00161     {
00162         return false;
00163     }
00164     while (!studentoVardoPavardėsIvestis(eilutė))
00165     {
00166         std::cout << "Įveskite studento vardą bei pavardę (ENTER - baigti): ";
00167         if (!std::getline(std::cin, eilutė) || eilutė.empty()) return false;
00168     }
00169     ndRandom(gautiSkaiciu("Įveskite norimą generuoti namų darbų rezultatų kiekį: ", 0, 100));
00170     egzRandom();
00171     return true;
00172 }
00173
00174 void Studentas::readRandom(int ndKiekis)
00175 {
00176     genVardaPavarde();
00177     ndRandom(ndKiekis);
00178     egzRandom();
00179 }
00180
00181 void Studentas::genVardaPavarde()
00182 {
00183     std::uniform_int_distribution<int> dist(0, 9);
00184
00185     static const std::vector<std::string> vardai = {
00186         "Irma", "Alma", "Irena", "Eglė", "Jolanta",
00187         "Petras", "Jonas", "Ignas", "Darius", "Simas"
00188     };
00189     static const std::vector<std::string> pavardės_m = {
00190         "Pavardaitė1", "Pavardaitė2", "Pavardaitė3", "Pavardaitė4", "Pavardaitė5",
00191         "Pavardaitė6", "Pavardaitė7", "Pavardaitė8", "Pavardaitė9", "Pavardaitė10"
00192     };
00193     static const std::vector<std::string> pavardės_v = {
00194         "Pavardenis1", "Pavardenis2", "Pavardenis3", "Pavardenis4", "Pavardenis5",
00195         "Pavardenis6", "Pavardenis7", "Pavardenis8", "Pavardenis9", "Pavardenis10"
00196     };
00197
00198     vardas = vardai[dist(rng())];
00199     // paskutinis simbolis 's' = vyras
00200     pavarde = (*vardas.rbegin() == 's') ? pavardės_v[dist(rng())] : pavardės_m[dist(rng())];
00201 }
00202
00203 void Studentas::ndRandom(int ndKiekis)
00204 {
00205     std::uniform_int_distribution<int> dist(1, 10);
00206     for (int i = 0; i < ndKiekis; i++)
00207         nd.push_back(dist(rng()));
00208 }
00209
00210 void Studentas::egzRandom()
00211 {
00212     std::uniform_int_distribution<int> dist(1, 10);
00213     rez = dist(rng());
00214 }
00215
00216 Studentas::~Studentas()
00217 {
00218     nd.clear();
00219     rez = 0;
00220 }
00221
00222 Studentas::Studentas(const Studentas& kitas)
00223     : Zmogus(kitas),
00224     nd(kitas.nd),
00225     rez(kitas.rez)
00226 {}
00227
00228 Studentas& Studentas::operator=(const Studentas& kitas)
00229 {
00230     if (this != &kitas)
00231     {
00232         Zmogus::operator=(kitas);
00233         nd = kitas.nd;
00234         rez = kitas.rez;
00235     }
00236     return *this;
00237 }

```

```

00238
00239 Studentas::Studentas(Studentas&& kitas) noexcept
00240     : Zmogus(std::move(kitas)),
00241     nd(std::move(kitas.nd)),
00242     rez(std::exchange(kitas.rez, 0))
00243 {}
00244
00245 Studentas& Studentas::operator=(Studentas&& kitas) noexcept
00246 {
00247     if(this != &kitas){
00248         Zmogus::operator=(std::move(kitas));
00249         nd = std::move(kitas.nd);
00250         rez = std::exchange(kitas.rez, 0);
00251     }
00252     return *this;
00253 }
00254
00255 std::istream& operator>(std::istream& is, Studentas &A)
00256 {
00257     A.read(is);
00258     return is;
00259 }
00260
00261 std::ostream& operator<(std::ostream& out, const Studentas &A){
00262     out << A.getVardas() << " " << A.getPavarde() << " ";
00263     for(int X : A.getNd()) out << X << " ";
00264     out << A.getRez();
00265     return out;
00266 }
00267
00268 void Studentas::read(std::istream& is, int ndKiekis){
00269     if(&is == &std::cin)
00270     {
00271         readStudentConsole(is);
00272     }
00273     else
00274     {
00275         readStudentStream(is, ndKiekis);
00276     }
00277 }

```

8.34 vector/studentas.h Failo Nuoroda

```

#include "zmogus.h"
#include <string>
#include <vector>
#include <istream>
#include <ostream>

```

Klasės

- class [Studentas](#)

Klasė, sukurianti studento objektą su pažymiais.

Kintamieji

- constexpr int [maxNdKiekis](#) = 10

Maksimalus namų darbų kiekis (naudojamas jei nenurodyta kitaip).

8.34.1 Kintamojo Dokumentacija

8.34.1.1 maxNdKiekis

```
int maxNdKiekis = 10 [constexpr]
```

Maksimalus namų darbų kiekis (naudojamas jei nenurodyta kitaip).

Apibrėžimas failo [studentas.h](#) eilutėje 10.

8.35 studentas.h

Eiti į šio failo dokumentaciją.

```

00001 #ifndef STUDENTAS_H
00002 #define STUDENTAS_H
00003
00004 #include "zmogus.h"
00005 #include <string>
00006 #include <vector>
00007 #include <istream>
00008 #include <ostream>
00009
00010 constexpr int maxNdKiekis = 10;
00011
00019 class Studentas : public Zmogus {
00020     std::vector<int> nd;
00021     int rez;
00022
00028     bool studentoVardoPavardesIvestis(const std::string& eilute);
00029
00031     void namuDarbuRezultatuIvestis();
00032
00034     void egzaminoRezultatuIvestis();
00035
00037     void genVardaPavarde();
00038
00043     void ndRandom(int ndKiekis);
00044
00046     void egzRandom();
00047
00048 public:
00049     // constructors
00051     Studentas() : Zmogus(), nd(), rez(0){};
00052
00060     Studentas(std::string v, std::string p, std::vector<int> n, int r)
00061         : Zmogus(std::move(v), std::move(p), nd(std::move(n)), rez(r) {});
00062
00068     Studentas(std::istream& is, int ndKiekis = 0){ read(is, ndKiekis); };
00069
00071     Studentas(const Studentas &kitas);
00072
00074     Studentas& operator=(const Studentas&);
00075
00077     Studentas(Studentas&&) noexcept;
00078
00080     Studentas& operator=(Studentas&&) noexcept;
00081
00082     // getters
00084     const std::vector<int> &getNd() const { return nd; };
00085
00087     int getRez() const { return rez; };
00088
00089     // setters
00094     void setNd(std::vector<int> n) { nd = std::move(n); }
00095
00100     void setRez(int r) { rez = r; }
00101
00108     std::istream& readStudentStream(std::istream& is, int ndKiekis);
00109
00115     std::istream& readStudentConsole(std::istream& is);
00116
00122     void read(std::istream& is, int ndKiekis = 0);
00123
00128     bool readSemiRandom();
00129
00134     void readRandom(int ndKiekis);
00135
00140     void addNd(int paz);
00141
00147     void resizeNd(int n, int skaicius = 0);
00148
00155     double galutinis(bool medianos, int ndKiekis = 0) const;
00156
00158     ~Studentas();
00159
00161     friend std::istream& operator>(std::istream& in, Studentas &A);
00162
00164     friend std::ostream& operator<(std::ostream& out, const Studentas &A);
00165 };
00166
00167 #endif // STUDENTAS_H

```

8.36 vector/test.cpp Failo Nuoroda

```
#include "test.h"  
#include <memory>
```

Funkcijos

- void [testas](#) ()
Paleidžia visus [Studentas](#) klasės testus.
- void [testDefaultKonstruktorius](#) ()
Testuoja numatytąjį konstruktorių.
- void [testStreamKonstruktorius](#) ()
Testuoja konstruktorių su srautu.
- void [testCopyKonstruktorius](#) ()
Testuoja kopijavimo konstruktorių.
- void [testCopyAssignmentOperator](#) ()
Testuoja kopijavimo priskyrimo operatorių.
- void [testMoveKonstruktorius](#) ()
Testuoja perkėlimo konstruktorių.
- void [testMoveAssignmentOperator](#) ()
Testuoja perkėlimo priskyrimo operatorių.
- void [testDestructor](#) ()
Testuoja destruktorių.
- void [testInputOperator](#) ()
Testuoja įvesties operatorių (>>).
- void [testOutputOperator](#) ()
Testuoja išvesties operatorių (<<).
- void [testoRezultatas](#) (const std::string &testoPavadinimas, bool passed)
Išveda testo rezultatą į konsolę.

8.36.1 Funkcijos Dokumentacija

8.36.1.1 [testas\(\)](#)

```
void testas ()
```

Paleidžia visus [Studentas](#) klasės testus.

Apibrėžimas failo [test.cpp](#) eilutėje 4.

8.36.1.2 [testCopyAssignmentOperator\(\)](#)

```
void testCopyAssignmentOperator ()
```

Testuoja kopijavimo priskyrimo operatorių.

Apibrėžimas failo [test.cpp](#) eilutėje 42.

8.36.1.3 testCopyKonstruktorius()

```
void testCopyKonstruktorius ()
```

Testuoja kopijavimo konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 34.

8.36.1.4 testDefaultKonstruktorius()

```
void testDefaultKonstruktorius ()
```

Testuoja numatytąjį konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 18.

8.36.1.5 testDestructor()

```
void testDestructor ()
```

Testuoja destruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 68.

8.36.1.6 testInputOperator()

```
void testInputOperator ()
```

Testuoja įvesties operatorių (>>).

Apibrėžimas failo [test.cpp](#) eilutėje 78.

8.36.1.7 testMoveAssignmentOperator()

```
void testMoveAssignmentOperator ()
```

Testuoja perkėlimo priskyrimo operatorių.

Apibrėžimas failo [test.cpp](#) eilutėje 59.

8.36.1.8 testMoveKonstruktorius()

```
void testMoveKonstruktorius ()
```

Testuoja perkėlimo konstruktorių.

Apibrėžimas failo [test.cpp](#) eilutėje 51.

8.36.1.9 testoRezultatas()

```
void testoRezultatas (  
    const std::string & testName,  
    bool passed)
```

Išveda testo rezultatą į konsolę.

Parametrai

<i>testName</i>	Testo pavadinimas.
-----------------	--------------------

<i>passed</i>	Ar testas pavyko (true) ar ne (false).
---------------	--

Apibrėžimas failo [test.cpp](#) eilutėje 98.

8.36.1.10 testOutputOperator()

```
void testOutputOperator ()
```

Testuoja išvesties operatorių (<<).

Apibrėžimas failo [test.cpp](#) eilutėje 88.

8.36.1.11 testStreamKonstruktorius()

```
void testStreamKonstruktorius ()
```

Testuoja konstruktorių su srautu.

Apibrėžimas failo [test.cpp](#) eilutėje 25.

8.37 test.cpp

Eiti į šio failo dokumentaciją.

```
00001 #include "test.h"
00002 #include <memory>
00003
00004 void testas()
00005 {
00006     std::cout << "Studento klasės testas\n\n";
00007     testDefaultKonstruktorius();
00008     testStreamKonstruktorius();
00009     testCopyKonstruktorius();
00010     testCopyAssignmentOperator();
00011     testMoveKonstruktorius();
00012     testMoveAssignmentOperator();
00013     testDestructor();
00014     testInputOperator();
00015     testOutputOperator();
00016 }
00017
00018 void testDefaultKonstruktorius()
00019 {
00020     Studentas A;
00021     bool testas = (A.getVardas().empty() && A.getPavarde().empty() && A.getNd().empty() && A.getRez()
00022 == 0);
00023     testoRezultatas("Default konstruktorius", testas);
00024 }
00025 void testStreamKonstruktorius()
00026 {
00027     std::istringstream iss("Jonas Jonaitis 1 2 10");
00028     Studentas C(iss, 2);
00029     bool testas = (C.getVardas()=="Jonas" && C.getPavarde()=="Jonaitis" && C.getNd().at(0) == 1 &&
00030 C.getNd().at(1) == 2 && C.getRez() == 10);
00031     std::cout << "Turėtų išvesti 'Jonas Jonaitis 1 2 10':\n" << C << std::endl;
00032     testoRezultatas("Stream įvesties konstruktorius", testas);
00033 }
00034 void testCopyKonstruktorius()
00035 {
00036     Studentas A("A", "AAA", {1,2}, 10);
00037     Studentas B(A);
00038     bool testas = (B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
00039 B.getNd().at(1) == 2 && B.getRez() == 10);
```

```

00039     testoRezultatas("Copy konstruktorius", testas);
00040 }
00041
00042 void testCopyAssignmentOperator()
00043 {
00044     Studentas A("A", "AAA", {1,2}, 10);
00045     Studentas B;
00046     B = A;
00047     bool testas = (B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10);
00048     testoRezultatas("Copy assignment operatorius", testas);
00049 }
00050
00051 void testMoveKonstruktorius()
00052 {
00053     Studentas A("A", "AAA", {1,2}, 10);
00054     Studentas B(std::move(A));
00055     bool testas = ((B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10) && (A.getVardas().empty() && A.getPavarde().empty() &&
A.getNd().empty() && A.getRez() == 0));
00056     testoRezultatas("Move konstruktorius", testas);
00057 }
00058
00059 void testMoveAssignmentOperator()
00060 {
00061     Studentas A("A", "AAA", {1,2}, 10);
00062     Studentas B;
00063     B = std::move(A);
00064     bool testas = ((B.getVardas()=="A" && B.getPavarde()=="AAA" && B.getNd().at(0) == 1 &&
B.getNd().at(1) == 2 && B.getRez() == 10) && (A.getVardas().empty() && A.getPavarde().empty() &&
A.getNd().empty() && A.getRez() == 0));
00065     testoRezultatas("Move assignment operatorius", testas);
00066 }
00067
00068 void testDestructor()
00069 {
00070     std::weak_ptr<Studentas> weak; // "protinga" rodykle, kuri siuo atveju rodo i scope esancia shared
protinga rodykle, taciau weak rodyklei nepriklauso shared A rodykle, t.y. ji nera savininke, tiesiog
rodo i ta pati objekta kaip ir shared rodykle, t.y. i Studenta A, ir jei nerodo i nieka galima
patikrinti su .expired();
00071     {
00072         std::shared_ptr<Studentas> A = std::make_shared<Studentas>("A", "AAA", std::vector<int>{1,2},
10);
00073         weak = A;
00074     }
00075     testoRezultatas("Destruktorius", weak.expired());
00076 }
00077
00078 void testInputOperator()
00079 {
00080     Studentas A;
00081     std::istringstream iss("Jonas Jonaitis 1 2 10");
00082     iss >> A;
00083     std::cout << "Turėty išvesti 'Jonas Jonaitis 1 2 10':\n" << A << std::endl;
00084     bool testas = (A.getVardas()=="Jonas" && A.getPavarde()=="Jonaitis" && A.getNd().at(0) == 1 &&
A.getNd().at(1) == 2 && A.getRez() == 10);
00085     testoRezultatas("Išvesties operatorius", testas);
00086 }
00087
00088 void testOutputOperator()
00089 {
00090     Studentas A("A", "AAA", {1,2}, 10);
00091     std::ostringstream out;
00092     out << A;
00093     std::string isvestis = out.str();
00094     bool testas = (isvestis.find("A") != std::string::npos && isvestis.find("AAA") !=
std::string::npos && isvestis.find("1") != std::string::npos && isvestis.find("2") !=
std::string::npos && isvestis.find("10") != std::string::npos);
00095     testoRezultatas("Išvesties operatorius", testas);
00096 }
00097
00098 void testoRezultatas(const std::string& testoPavadinimas, bool passed)
00099 {
00100     std::cout << (passed ? "PASS " : "FAIL ") << testoPavadinimas << "\n";
00101 }

```

8.38 vector/test.h Failo Nuoroda

```

#include "studentas.h"
#include "investis.h"

```

Funkcijos

- void `testas` ()
Paleidžia visus `Studentas` klasės testus.
- void `testoRezultatas` (const std::string &testName, bool passed)
Išveda testo rezultatą į konsolę.
- void `testDefaultKonstruktorius` ()
Testuoja numatytąjį konstruktorių.
- void `testStreamKonstruktorius` ()
Testuoja konstruktorių su srautu.
- void `testCopyKonstruktorius` ()
Testuoja kopijavimo konstruktorių.
- void `testCopyAssignmentOperator` ()
Testuoja kopijavimo priskyrimo operatorių.
- void `testMoveKonstruktorius` ()
Testuoja perkėlimo konstruktorių.
- void `testMoveAssignmentOperator` ()
Testuoja perkėlimo priskyrimo operatorių.
- void `testDestructor` ()
Testuoja destruktorių.
- void `testInputOperator` ()
Testuoja įvesties operatorių (>>).
- void `testOutputOperator` ()
Testuoja išvesties operatorių (<<).

8.38.1 Funkcijos Dokumentacija

8.38.1.1 `testas()`

```
void testas ()
```

Paleidžia visus `Studentas` klasės testus.

Apibrėžimas failo `test.cpp` eilutėje 4.

8.38.1.2 `testCopyAssignmentOperator()`

```
void testCopyAssignmentOperator ()
```

Testuoja kopijavimo priskyrimo operatorių.

Apibrėžimas failo `test.cpp` eilutėje 42.

8.38.1.3 `testCopyKonstruktorius()`

```
void testCopyKonstruktorius ()
```

Testuoja kopijavimo konstruktorių.

Apibrėžimas failo `test.cpp` eilutėje 34.

8.38.1.4 testDefaultKonstruktorius()

```
void testDefaultKonstruktorius ()
```

Testuoja numatytąjį konstruktorių.

Apibrėžimas failo `test.cpp` eilutėje 18.

8.38.1.5 testDestructor()

```
void testDestructor ()
```

Testuoja destruktorių.

Apibrėžimas failo `test.cpp` eilutėje 68.

8.38.1.6 testInputOperator()

```
void testInputOperator ()
```

Testuoja įvesties operatorių (>>).

Apibrėžimas failo `test.cpp` eilutėje 78.

8.38.1.7 testMoveAssignmentOperator()

```
void testMoveAssignmentOperator ()
```

Testuoja perkėlimo priskyrimo operatorių.

Apibrėžimas failo `test.cpp` eilutėje 59.

8.38.1.8 testMoveKonstruktorius()

```
void testMoveKonstruktorius ()
```

Testuoja perkėlimo konstruktorių.

Apibrėžimas failo `test.cpp` eilutėje 51.

8.38.1.9 testoRezultatas()

```
void testoRezultatas (  
    const std::string & testName,  
    bool passed)
```

Išveda testo rezultatą į konsolę.

Parametrai

<code>testName</code>	Testo pavadinimas.
-----------------------	--------------------

<i>passed</i>	Ar testas pavyko (true) ar ne (false).
---------------	--

Apibrėžimas failo [test.cpp](#) eilutėje 98.

8.38.1.10 testOutputOperator()

```
void testOutputOperator ()
```

Testuoja išvesties operatorių (<<).

Apibrėžimas failo [test.cpp](#) eilutėje 88.

8.38.1.11 testStreamKonstruktorius()

```
void testStreamKonstruktorius ()
```

Testuoja konstruktorių su srautu.

Apibrėžimas failo [test.cpp](#) eilutėje 25.

8.39 test.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef TEST_H
00002 #define TEST_H
00003
00004 #include "studentas.h"
00005 #include "investis.h"
00006
00010 void testas();
00011
00017 void testoRezultatas(const std::string& testName, bool passed);
00018
00020 void testDefaultKonstruktorius();
00021
00023 void testStreamKonstruktorius();
00024
00026 void testCopyKonstruktorius();
00027
00029 void testCopyAssignmentOperator();
00030
00032 void testMoveKonstruktorius();
00033
00035 void testMoveAssignmentOperator();
00036
00038 void testDestructor();
00039
00041 void testInputOperator();
00042
00044 void testOutputOperator();
00045
00046 #endif
```

8.40 vector/Timer.h Failo Nuoroda

```
#include <chrono>
```


Klasės

- class [Timer](#)

8.41 Timer.h

Eiti į šio failo dokumentaciją.

```
00001 #ifndef TIMER_H
00002 #define TIMER_H
00003
00004 #include <chrono>
00005 class Timer {
00006     private:
00007         // panaudojame using
00008         using hrClock = std::chrono::high_resolution_clock;
00009         using durationDouble = std::chrono::duration<double>;
00010         std::chrono::time_point<hrClock> start;
00011     public:
00012         Timer() : start{ hrClock::now() } {}
00013         void reset() {
00014             start = hrClock::now();
00015         }
00016         double elapsed() const {
00017             return durationDouble (hrClock::now() - start).count();
00018         }
00019 };
00020
00021 #endif
```

8.42 vector/vector.h Failo Nuoroda

```
#include <memory>
#include <iterator>
#include <utility>
#include <stdexcept>
#include <algorithm>
#include <initializer_list>
#include <compare>
#include <cassert>
```

Klasės

- class [Vector< T, Allocator >](#)

Dinaminio masyvo konteineris, atitinkantis std::vector sąsają (C++20).

Funkcijos

- template<class T, class Alloc>
bool [operator==](#) (const [Vector](#)< T, Alloc > &lhs, const [Vector](#)< T, Alloc > &rhs)
Palygina du vektorius lygybei.
- template<class T, class Alloc>
auto [operator<=>](#) (const [Vector](#)< T, Alloc > &lhs, const [Vector](#)< T, Alloc > &rhs) -> std::compare_three_↵
_way_result_t< T >
Atlieka trijų krypčių palyginimą (C++20).

- `template<class T, class Alloc>`
`void swap (Vector< T, Alloc > &lhs, Vector< T, Alloc > &rhs) noexcept(noexcept(lhs.swap(rhs)))`
Sukeičia dviejų vektorių turinius (specializacija `std::swap`).
- `template<class T, class Alloc, class U>`
`Vector< T, Alloc >::size_type erase (Vector< T, Alloc > &c, const U &value)`
Pašalina visus elementus, lygius `value`.
- `template<class T, class Alloc, class Pred>`
`Vector< T, Alloc >::size_type erase_if (Vector< T, Alloc > &c, Pred pred)`
Pašalina visus elementus, tenkinančius predikatą `pred`.

8.42.1 Funkcijos Dokumentacija

8.42.1.1 erase()

```
template<class T, class Alloc, class U>
Vector< T, Alloc >::size_type erase (
    Vector< T, Alloc > & c,
    const U & value)
```

Pašalina visus elementus, lygius `value`.

Template Parameters

<i>T</i>	Elemento tipas.
<i>Alloc</i>	Paskirstytojas.
<i>U</i>	Palyginamos reikšmės tipas.

Parametrai

<i>c</i>	Vektorius, iš kurio šalinama.
<i>value</i>	Reikšmė, su kuria lyginama.

Gražina

Pašalintų elementų skaičius.

Apibrėžimas failo [vector.h](#) eilutėje 1004.

8.42.1.2 erase_if()

```
template<class T, class Alloc, class Pred>
Vector< T, Alloc >::size_type erase_if (
    Vector< T, Alloc > & c,
    Pred pred)
```

Pašalina visus elementus, tenkinančius predikatą `pred`.

Template Parameters

<i>T</i>	Elemento tipas.
----------	-----------------

<i>Alloc</i>	Paskirstytojas.
<i>Pred</i>	Predikato tipas.

Parametrai

<i>c</i>	Vektorius, iš kurio šalinama.
<i>pred</i>	Predikatas, grąžinantis true šalinamiems elementams.

Gražina

Pašalintų elementų skaičius.

Apibrėžimas failo [vector.h](#) eilutėje 1022.

8.42.1.3 operator<=>()

```
template<class T, class Alloc>
auto operator<=> (
    const Vector< T, Alloc > & lhs,
    const Vector< T, Alloc > & rhs) -> std::compare_three_way_result_t< T >
```

Atlieka trijų krypčių palyginimą (C++20).

Template Parameters

<i>T</i>	Elemento tipas.
<i>Alloc</i>	Paskirstytojas.

Parametrai

<i>lhs</i>	Pirmasis vektorius.
<i>rhs</i>	Antrasis vektorius.

Gražina

Rezultatas pagal <=> operatorių.

Apibrėžimas failo [vector.h](#) eilutėje 973.

8.42.1.4 operator==()

```
template<class T, class Alloc>
bool operator==(
    const Vector< T, Alloc > & lhs,
    const Vector< T, Alloc > & rhs)
```

Palygina du vektorius lygybei.

Template Parameters

<i>T</i>	Elemento tipas.
----------	-----------------

<i>Alloc</i>	Paskirstytojas.
--------------	-----------------

Parametrai

<i>lhs</i>	Pirmasis vektorius.
<i>rhs</i>	Antrasis vektorius.

Gražina

true jei abiejų dydžiai lygūs ir visi elementai lygūs, kitaip false.

Apibrėžimas failo [vector.h](#) eilutėje 956.

8.42.1.5 swap()

```
template<class T, class Alloc>
void swap (
    Vector< T, Alloc > & lhs,
    Vector< T, Alloc > & rhs) [noexcept]
```

Sukeičia dviejų vektorių turinius (specializacija std::swap).

Template Parameters

<i>T</i>	Elemento tipas.
<i>Alloc</i>	Paskirstytojas.

Parametrai

<i>lhs</i>	Pirmasis vektorius.
<i>rhs</i>	Antrasis vektorius.

Apibrėžimas failo [vector.h](#) eilutėje 989.

8.43 vector.h

[Eiti į šio failo dokumentaciją.](#)

```
00001
00005
00006 #ifndef VECTOR_H
00007 #define VECTOR_H
00008
00009 #include <memory>
00010 #include <iterator>
00011 #include <utility>
00012 #include <stdexcept>
00013 #include <algorithm>
00014 #include <initializer_list>
00015 #include <compare>
```

```

00016 #include <cassert>
00017
00024 template <class T, class Allocator = std::allocator<T>
00025 class Vector {
00026 public:
00027     // ----- tipų apibrėžimai -----
00028     using value_type = T;
00029     using allocator_type = Allocator;
00030     using size_type = std::size_t;
00031     using difference_type = std::ptrdiff_t;
00032     using reference = T&;
00033     using const_reference = const T&;
00034     using pointer = typename std::allocator_traits<Allocator>::pointer;
00035     using const_pointer = typename std::allocator_traits<Allocator>::const_pointer;
00036     using iterator = pointer;
00037     using const_iterator = const_pointer;
00038     using reverse_iterator = std::reverse_iterator<iterator>;
00039     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00040
00041     // ----- konstruktoriai -----
00042
00046     Vector() : data_(nullptr), size_(0), capacity_(0), alloc_(Allocator{}) {}
00047
00054     explicit Vector(size_type count, const Allocator& alloc = Allocator{})
00055         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00056         reserve(count);
00057     }
00058
00066     Vector(size_type count, const_reference value, const Allocator& alloc = Allocator{})
00067         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00068         reserve(count);
00069         for (size_type i = 0; i < count; ++i) {
00070             std::allocator_traits<Allocator>::construct(alloc_, data_ + i, value);
00071         }
00072         size_ = count;
00073     }
00074
00085     template <class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
00086     Vector(InputIt first, InputIt last, const Allocator& alloc = Allocator{})
00087         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00088         size_type count = static_cast<size_type>(std::distance(first, last));
00089         reserve(count);
00090         for (size_type i = 0; first != last; ++first, ++i) {
00091             std::allocator_traits<Allocator>::construct(alloc_, data_ + i, *first);
00092         }
00093         size_ = count;
00094     }
00095
00102     Vector(std::initializer_list<T> ilist, const Allocator& alloc = Allocator{})
00103         : Vector(ilist.begin(), ilist.end(), alloc) {}
00104
00112     Vector(const Vector& other)
00113         : data_(nullptr), size_(0), capacity_(0), alloc_(other.alloc_) {
00114         reserve(other.size_);
00115         for (size_type i = 0; i < other.size_; ++i) {
00116             std::allocator_traits<Allocator>::construct(alloc_, data_ + i, *(other.data_ + i));
00117         }
00118         size_ = other.size_;
00119     }
00120
00129     Vector(const Vector& other, const Allocator& alloc)
00130         : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00131         reserve(other.size_);
00132         for (size_type i = 0; i < other.size_; ++i) {
00133             std::allocator_traits<Allocator>::construct(alloc_, data_ + i, *(other.data_ + i));
00134         }
00135         size_ = other.size_;
00136     }
00137
00146     Vector& operator=(const Vector& other) {
00147         if (this != &other) {
00148             clear();
00149             if (data_) {
00150                 std::allocator_traits<Allocator>::deallocate(alloc_, data_, capacity_);
00151             }
00152             data_ = nullptr;
00153             size_ = 0;
00154             capacity_ = 0;
00155             reserve(other.size_);
00156             for (size_type i = 0; i < other.size_; ++i) {
00157                 std::allocator_traits<Allocator>::construct(alloc_, data_ + i, *(other.data_ + i));
00158             }
00159             size_ = other.size_;
00160         }
00161         return *this;
00162     }
00163

```

```

00171     Vector(Vector&& other) noexcept
00172     : data_(other.data_), size_(other.size_), capacity_(other.capacity_),
00173     alloc_(std::move(other.alloc_)) {
00174         other.data_ = nullptr;
00175         other.size_ = 0;
00176         other.capacity_ = 0;
00177     }
00187     Vector(Vector&& other, const Allocator& alloc)
00188     : data_(nullptr), size_(0), capacity_(0), alloc_(alloc) {
00189         if (alloc == other.alloc_) {
00190             data_ = other.data_;
00191             size_ = other.size_;
00192             capacity_ = other.capacity_;
00193             other.data_ = nullptr;
00194             other.size_ = 0;
00195             other.capacity_ = 0;
00196         } else {
00197             reserve(other.size_);
00198             for (size_type i = 0; i < other.size_; ++i) {
00199                 std::allocator_traits<Allocator>::construct(alloc_, data_ + i, std::move(*(other.data_
00200 + i)));
00201             }
00202             size_ = other.size_;
00203             other.clear();
00204         }
00205     }
00214     Vector& operator=(std::initializer_list<T> ilist) {
00215         assign(ilist.begin(), ilist.end());
00216         return *this;
00217     }
00218
00227     Vector& operator=(Vector&& other) noexcept {
00228         if (this != &other) {
00229             clear();
00230             if (data_) {
00231                 std::allocator_traits<Allocator>::deallocate(alloc_, data_, capacity_);
00232             }
00233             data_ = other.data_;
00234             size_ = other.size_;
00235             capacity_ = other.capacity_;
00236             alloc_ = std::move(other.alloc_);
00237             other.data_ = nullptr;
00238             other.size_ = 0;
00239             other.capacity_ = 0;
00240         }
00241         return *this;
00242     }
00243
00249     ~Vector() {
00250         clear();
00251         if (data_) {
00252             std::allocator_traits<Allocator>::deallocate(alloc_, data_, capacity_);
00253         }
00254     }
00255
00256     // ----- priskyrimo metodai -----
00257
00267     void assign(size_type count, const_reference value) {
00268         if (count > capacity_) reserve(count);
00269         if (count <= size_) {
00270             std::fill_n(data_, count, value);
00271             for (size_type i = count; i < size_; ++i) {
00272                 std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00273             }
00274         } else {
00275             std::fill_n(data_, size_, value);
00276             size_type i = size_;
00277             try {
00278                 for (; i < count; ++i) {
00279                     std::allocator_traits<Allocator>::construct(alloc_, data_ + i, value);
00280                 }
00281             } catch (...) {
00282                 for (size_type j = size_; j < i; ++j) {
00283                     std::allocator_traits<Allocator>::destroy(alloc_, data_ + j);
00284                 }
00285                 throw;
00286             }
00287             size_ = count;
00288         }
00289     }
00290
00298     template <class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
00299     void assign(InputIt first, InputIt last) {
00300         size_type new_size = static_cast<size_type>(std::distance(first, last));
00301         if (new_size > capacity_) reserve(new_size);

```

```

00302         size_type i = 0;
00303         for (; i < size_ && first != last; ++i, ++first) {
00304             *(data_ + i) = *first;
00305         }
00306         size_type constructed = i;
00307         try {
00308             for (; first != last; ++first, ++i) {
00309                 std::allocator_traits<Allocator>::construct(alloc_, data_ + i, *first);
00310             }
00311         } catch (...) {
00312             for (size_type j = constructed; j < i; ++j) {
00313                 std::allocator_traits<Allocator>::destroy(alloc_, data_ + j);
00314             }
00315             size_ = constructed;
00316             throw;
00317         }
00318         for (size_type j = new_size; j < size_; ++j) {
00319             std::allocator_traits<Allocator>::destroy(alloc_, data_ + j);
00320         }
00321         size_ = new_size;
00322     }
00323
00329     void assign(std::initializer_list<T> ilist) {
00330         assign(ilist.begin(), ilist.end());
00331     }
00332
00338     allocator_type get_allocator() const {
00339         return alloc_;
00340     }
00341
00342     // ----- elementų prieiga -----
00343
00351     reference at(size_type pos) {
00352         if (pos >= size_) throw std::out_of_range("Vector::at");
00353         return *(data_ + pos);
00354     }
00355
00363     const_reference at(size_type pos) const {
00364         if (pos >= size_) throw std::out_of_range("Vector::at");
00365         return data_[pos];
00366     }
00367
00374     reference operator[](size_type pos) {
00375         return *(data_ + pos);
00376     }
00377
00384     const_reference operator[](size_type pos) const {
00385         return *(data_ + pos);
00386     }
00387
00393     reference front() {
00394         return *data_;
00395     }
00396
00402     const_reference front() const {
00403         return *data_;
00404     }
00405
00411     reference back() {
00412         return *(data_ + (size_ - 1));
00413     }
00414
00420     const_reference back() const {
00421         return *(data_ + (size_ - 1));
00422     }
00423
00429     pointer data() {
00430         return data_;
00431     }
00432
00438     const_pointer data() const {
00439         return data_;
00440     }
00441
00442     // ----- iteratorių palaikymas -----
00443
00448     iterator begin() {
00449         return data_;
00450     }
00451
00456     const_iterator begin() const {
00457         return data_;
00458     }
00459
00464     const_iterator cbegin() const noexcept {
00465         return data_;
00466     }

```

```

00467
00472     iterator end() noexcept {
00473         return data_ + size_;
00474     }
00475
00480     const_iterator end() const noexcept {
00481         return data_ + size_;
00482     }
00483
00488     const_iterator cend() const noexcept {
00489         return data_ + size_;
00490     }
00491
00496     reverse_iterator rbegin() {
00497         return reverse_iterator(end());
00498     }
00499
00504     const_reverse_iterator rbegin() const {
00505         return reverse_iterator(end());
00506     }
00507
00512     const_reverse_iterator crbegin() const noexcept {
00513         return const_reverse_iterator(cend());
00514     }
00515
00520     reverse_iterator rend() {
00521         return reverse_iterator(begin());
00522     }
00523
00528     const_reverse_iterator rend() const {
00529         return reverse_iterator(begin());
00530     }
00531
00536     const_reverse_iterator crend() const noexcept {
00537         return const_reverse_iterator(cbegin());
00538     }
00539
00540     // ----- talpos metodai -----
00541
00546     bool empty() const {
00547         return size_ == 0;
00548     }
00549
00554     size_type size() const {
00555         return size_;
00556     }
00557
00562     size_type max_size() const {
00563         return std::allocator_traits<Allocator>::max_size(alloc_);
00564     }
00565
00574     void reserve(size_type new_cap) {
00575         if (new_cap > max_size()) throw std::length_error("new capacity larger than maximum allowed
size");
00576         if (new_cap <= capacity_) return;
00577         reallocate(new_cap);
00578     }
00579
00584     size_type capacity() const {
00585         return capacity_;
00586     }
00587
00591     void shrink_to_fit() {
00592         if (size_ < capacity_) {
00593             if (size_ == 0) {
00594                 std::allocator_traits<Allocator>::deallocate(alloc_, data_, capacity_);
00595                 data_ = nullptr;
00596                 capacity_ = 0;
00597             } else {
00598                 reallocate(size_);
00599             }
00600         }
00601     }
00602
00603     // ----- modifikatoriai -----
00604
00608     void clear() {
00609         for (size_type i = 0; i < size_; i++) {
00610             std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00611         }
00612         size_ = 0;
00613     }
00614
00622     iterator insert(const_iterator pos, const_reference value) {
00623         size_type index = static_cast<size_type>(pos - cbegin());
00624         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00625         shift_right(index, 1);

```



```

00626         std::allocator_traits<Allocator>::construct(alloc_, data_ + index, value);
00627         ++size_;
00628         return begin() + index;
00629     }
00630
00631     iterator insert(const_iterator pos, T&& value) {
00632         size_type index = static_cast<size_type>(pos - cbegin());
00633         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00634         shift_right(index, 1);
00635         std::allocator_traits<Allocator>::construct(alloc_, data_ + index, std::move(value));
00636         ++size_;
00637         return begin() + index;
00638     }
00639
00640     iterator insert(const_iterator pos, size_type count, const_reference value) {
00641         if (count == 0) return begin() + (pos - cbegin());
00642         size_type index = static_cast<size_type>(pos - cbegin());
00643         if (size_ + count > capacity_) reserve(size_ + count);
00644         shift_right(index, count);
00645         for (size_type i = 0; i < count; ++i)
00646             std::allocator_traits<Allocator>::construct(alloc_, data_ + index + i, value);
00647         size_ += count;
00648         return begin() + index;
00649     }
00650
00651     template <class InputIt, class = typename std::enable_if<!std::is_integral<InputIt>::value>::type>
00652     iterator insert(const_iterator pos, InputIt first, InputIt last) {
00653         size_type index = static_cast<size_type>(pos - cbegin());
00654         size_type count = static_cast<size_type>(std::distance(first, last));
00655         if (count == 0) return begin() + index;
00656         if (size_ + count > capacity_) reserve(size_ + count);
00657         shift_right(index, count);
00658         size_type i = 0;
00659         try {
00660             for (; first != last; ++first, ++i)
00661                 std::allocator_traits<Allocator>::construct(alloc_, data_ + index + i, *first);
00662         } catch (...) {
00663             for (size_type j = 0; j < i; ++j)
00664                 std::allocator_traits<Allocator>::destroy(alloc_, data_ + index + j);
00665             shift_left(index, count);
00666             throw;
00667         }
00668         size_ += count;
00669         return begin() + index;
00670     }
00671
00672     iterator insert(const_iterator pos, std::initializer_list<T> ilist) {
00673         return insert(pos, ilist.begin(), ilist.end());
00674     }
00675
00676     template <class... Args>
00677     iterator emplace(const_iterator pos, Args&&... args) {
00678         size_type index = static_cast<size_type>(pos - cbegin());
00679         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00680         shift_right(index, 1);
00681         std::allocator_traits<Allocator>::construct(alloc_, data_ + index,
00682             std::forward<Args>(args)...);
00683         ++size_;
00684         return begin() + index;
00685     }
00686
00687     iterator erase(const_iterator pos) {
00688         size_type index = static_cast<size_type>(pos - cbegin());
00689         std::allocator_traits<Allocator>::destroy(alloc_, data_ + index);
00690         shift_left(index + 1, 1);
00691         --size_;
00692         return begin() + index;
00693     }
00694
00695     iterator erase(const_iterator first, const_iterator last) {
00696         size_type index_first = static_cast<size_type>(first - cbegin());
00697         size_type index_last = static_cast<size_type>(last - cbegin());
00698         size_type count = index_last - index_first;
00699         for (size_type i = index_first; i < index_last; ++i)
00700             std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00701         shift_left(index_last, count);
00702         size_ -= count;
00703         return begin() + index_first;
00704     }
00705
00706     void push_back(const_reference value) {
00707         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00708         std::allocator_traits<Allocator>::construct(alloc_, data_ + size_, value);
00709         ++size_;
00710     }
00711
00712     void push_back(value_type&& value) {

```

```

00776         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00777         std::allocator_traits<Allocator>::construct(alloc_, data_ + size_, std::move(value));
00778         ++size_;
00779     }
00780
00781     template <class... Args>
00782     reference emplace_back(Args&&... args) {
00783         if (size_ >= capacity_) reserve(capacity_ == 0 ? 1 : capacity_ * 2);
00784         std::allocator_traits<Allocator>::construct(alloc_, data_ + size_,
00785             std::forward<Args>(args)...);
00786         ++size_;
00787         return back();
00788     }
00789
00790     void pop_back() {
00791         assert(size_ > 0 && "pop_back called on empty Vector");
00792         --size_;
00793         std::allocator_traits<Allocator>::destroy(alloc_, data_ + size_);
00794     }
00795
00796     void resize(size_type count) {
00797         if (count < size_) {
00798             for (size_type i = count; i < size_; ++i) {
00799                 std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00800             }
00801             size_ = count;
00802         } else if (count > size_) {
00803             if (count > capacity_) {
00804                 size_type new_cap = capacity_;
00805                 if (new_cap == 0) new_cap = 1;
00806                 while (new_cap < count) new_cap *= 2;
00807                 reserve(new_cap);
00808             }
00809             for (size_type i = size_; i < count; ++i) {
00810                 std::allocator_traits<Allocator>::construct(alloc_, data_ + i);
00811             }
00812             size_ = count;
00813         }
00814     }
00815
00816     void resize(size_type count, const_reference value) {
00817         if (count < size_) {
00818             for (size_type i = count; i < size_; ++i) {
00819                 std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00820             }
00821             size_ = count;
00822         } else if (count > size_) {
00823             if (count > capacity_) {
00824                 size_type new_cap = capacity_;
00825                 if (new_cap == 0) new_cap = 1;
00826                 while (new_cap < count) new_cap *= 2;
00827                 reserve(new_cap);
00828             }
00829             for (size_type i = size_; i < count; ++i) {
00830                 std::allocator_traits<Allocator>::construct(alloc_, data_ + i, value);
00831             }
00832             size_ = count;
00833         }
00834     }
00835
00836     void swap(Vector& other) noexcept {
00837         std::swap(data_, other.data_);
00838         std::swap(size_, other.size_);
00839         std::swap(capacity_, other.capacity_);
00840         std::swap(alloc_, other.alloc_);
00841     }
00842
00843 private:
00844     pointer data_ = nullptr;
00845     size_type size_ = 0;
00846     size_type capacity_ = 0;
00847     allocator_type alloc_;
00848
00849     void reallocate(size_type new_cap) {
00850         pointer new_data = std::allocator_traits<Allocator>::allocate(alloc_, new_cap);
00851         size_type i = 0;
00852         try {
00853             for (; i < size_; ++i) {
00854                 std::allocator_traits<Allocator>::construct(alloc_, new_data + i,
00855                     std::move_if_noexcept(*(data_ + i)));
00856             }
00857         } catch (...) {
00858             for (size_type j = 0; j < i; ++j) {
00859                 std::allocator_traits<Allocator>::destroy(alloc_, new_data + j);
00860             }
00861             std::allocator_traits<Allocator>::deallocate(alloc_, new_data, new_cap);
00862             throw;
00863         }
00864     }

```

```

00900     }
00901     for (size_type i = 0; i < size_; ++i) {
00902         std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00903     }
00904     if (data_) {
00905         std::allocator_traits<Allocator>::deallocate(alloc_, data_, capacity_);
00906     }
00907     data_ = new_data;
00908     capacity_ = new_cap;
00909 }
00910
00919 void shift_right(size_type index, size_type count) {
00920     for (size_type i = size_; i > index; --i) {
00921         std::allocator_traits<Allocator>::construct(alloc_, data_ + i + count - 1,
00922                                                     std::move_if_noexcept(*(data_ + i - 1)));
00923         std::allocator_traits<Allocator>::destroy(alloc_, data_ + i - 1);
00924     }
00925 }
00926
00935 void shift_left(size_type index, size_type count) {
00936     for (size_type i = index; i < size_; ++i) {
00937         std::allocator_traits<Allocator>::construct(alloc_, data_ + i - count,
00938                                                     std::move_if_noexcept(data_[i]));
00939         std::allocator_traits<Allocator>::destroy(alloc_, data_ + i);
00940     }
00941 }
00942 };
00943
00944 // ----- ne narių funkcijos -----
00945
00955 template <class T, class Alloc>
00956 bool operator==(const Vector<T, Alloc>& lhs, const Vector<T, Alloc>& rhs) {
00957     if (lhs.size() != rhs.size()) return false;
00958     for (typename Vector<T, Alloc>::size_type i = 0; i < lhs.size(); ++i)
00959         if (lhs[i] != rhs[i]) return false;
00960     return true;
00961 }
00962
00972 template <class T, class Alloc>
00973 auto operator<=>(const Vector<T, Alloc>& lhs, const Vector<T, Alloc>& rhs)
00974 -> std::compare_three_way_result_t<T> {
00975     for (typename Vector<T, Alloc>::size_type i = 0; i < lhs.size() && i < rhs.size(); ++i)
00976         if (auto cmp = lhs[i] <=> rhs[i]; cmp != 0) return cmp;
00977     return static_cast<std::compare_three_way_result_t<T>>(lhs.size() <=> rhs.size());
00978 }
00979
00988 template <class T, class Alloc>
00989 void swap(Vector<T, Alloc>& lhs, Vector<T, Alloc>& rhs) noexcept(noexcept(lhs.swap(rhs))) {
00990     lhs.swap(rhs);
00991 }
00992
01003 template <class T, class Alloc, class U>
01004 typename Vector<T, Alloc>::size_type erase(Vector<T, Alloc>& c, const U& value) {
01005     auto it = std::remove(c.begin(), c.end(), value);
01006     auto count = static_cast<typename Vector<T, Alloc>::size_type>(c.end() - it);
01007     c.erase(it, c.end());
01008     return count;
01009 }
01010
01021 template <class T, class Alloc, class Pred>
01022 typename Vector<T, Alloc>::size_type erase_if(Vector<T, Alloc>& c, Pred pred) {
01023     auto it = std::remove_if(c.begin(), c.end(), pred);
01024     auto count = static_cast<typename Vector<T, Alloc>::size_type>(c.end() - it);
01025     c.erase(it, c.end());
01026     return count;
01027 }
01028
01029 #endif // VECTOR_H

```

8.44 vector/vector_compare.cpp Failo Nuoroda

```
#include "vector_compare.h"
```

Funkcijos

- void `vector_compare()`
Palygina STL vektorių su nuosavu vektoriu.

8.44.1 Funkcijos Dokumentacija

8.44.1.1 vector_compare()

```
void vector_compare ()
```

Palygina STL vektorių su nuosavu vektoriu.

Apibrėžimas failo [vector_compare.cpp](#) eilutėje 3.

8.45 vector_compare.cpp

[Eiti į šio failo dokumentaciją.](#)

```
00001 #include "vector_compare.h"
00002
00003 void vector_compare() {
00004     std::vector<unsigned int> sz{10000, 100000, 1000000, 10000000, 100000000};
00005     for(auto x : sz){
00006         int perskirstymai = 0;
00007         Timer t;
00008         std::vector<int> v1;
00009
00010         for(int i = 1; i <= x; ++i)
00011         {
00012             v1.push_back(i);
00013             if(v1.size() == v1.capacity()) perskirstymai++;
00014         }
00015         std::cout << "Vektoriaus dydis - " << x << " Užpildymo laikas - " << t.elapsed() << "s." << "
Perskirstymų kiekis - " << perskirstymai << "\n";
00016         perskirstymai = 0;
00017         t.reset();
00018         Vector<int> v2;
00019         for(int i = 1; i <= x; ++i)
00020         {
00021             v2.push_back(i);
00022             if(v2.size() == v2.capacity()) perskirstymai++;
00023         }
00024
00025         std::cout << "Vektoriaus dydis - " << x << " Užpildymo laikas - " << t.elapsed() << "s." << "
Perskirstymų kiekis - " << perskirstymai << "\n";
00026     }
00027 }
```

8.46 vector/vector_compare.h Failo Nuoroda

```
#include "vector.h"
#include <vector>
#include "Timer.h"
#include <iostream>
```

Funkcijos

- void [vector_compare](#) ()

Palygina STL vektorių su nuosavu vektoriu.

8.46.1 Funkcijos Dokumentacija

8.46.1.1 vector_compare()

```
void vector_compare ()
```

Palygina STL vektorių su nuosavu vektoriu.

Apibrėžimas failo [vector_compare.cpp](#) eilutėje 3.

8.47 vector_compare.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef VECTOR_COMPARE_H
00002 #define VECTOR_COMPARE_H
00003
00004 #include "vector.h"
00005 #include <vector>
00006 #include "Timer.h"
00007 #include <iostream>
00008
00010 void vector_compare();
00011
00012 #endif
```

8.48 vector/zmogus.cpp Failo Nuoroda

```
#include "zmogus.h"
```

8.49 zmogus.cpp

[Eiti į šio failo dokumentaciją.](#)

```
00001 #include "zmogus.h"
00002
00003 //copy constructor
00004 Zmogus::Zmogus(const Zmogus &kitas)
00005     : vardas(kitas.vardas),
00006       pavarde(kitas.pavarde)
00007 {}
00008
00009 // copy assignment operator
00010 Zmogus& Zmogus::operator=(const Zmogus& kitas)
00011 {
00012     if(this != &kitas)
00013     {
00014         vardas = kitas.vardas;
00015         pavarde = kitas.pavarde;
00016     }
00017     return *this;
00018 }
00019
00020 // move constructor
00021 Zmogus::Zmogus(Zmogus&& kitas) noexcept
00022     : vardas(std::move(kitas.vardas)),
00023       pavarde(std::move(kitas.pavarde))
00024 {}
00025
00026 // move assignment operator
00027 Zmogus& Zmogus::operator=(Zmogus&& kitas) noexcept
00028 {
00029     if(this != &kitas)
```

```

00030     {
00031         vardas = std::move(kitas.vardas);
00032         pavarde = std::move(kitas.pavarde);
00033     }
00034     return *this;
00035 }
00036
00037 Zmogus::~Zmogus() {
00038     vardas.clear();
00039     pavarde.clear();
00040 };

```

8.50 vector/zmogus.h Failo Nuoroda

```
#include <string>
```

Klasės

- class [Zmogus](#)

Abstrakti bazinė klasė, turinti vardą ir pavardę.

8.51 zmogus.h

[Eiti į šio failo dokumentaciją.](#)

```

00001 #ifndef ZMOGUS_H
00002 #define ZMOGUS_H
00003
00004 #include <string>
00005
00013 class Zmogus {
00014     protected:
00015         std::string vardas;
00016         std::string pavarde;
00017
00018     public:
00020         Zmogus() : vardas(""), pavarde("") {};
00021
00027         Zmogus(const std::string v, const std::string p)
00028             : vardas(std::move(v)), pavarde(std::move(p)) {};
00029
00031         Zmogus(const Zmogus &kitas);
00032
00034         Zmogus& operator=(const Zmogus&);
00035
00037         Zmogus(Zmogus&&) noexcept;
00038
00040         Zmogus& operator=(Zmogus&&) noexcept;
00041
00043         const std::string& getVardas() const { return vardas; }
00044
00046         const std::string& getPavarde() const { return pavarde; }
00047
00052         void setVardas(std::string v) { vardas = std::move(v); }
00053
00058         void setPavarde(std::string p) { pavarde = std::move(p); }
00059
00063         virtual ~Zmogus() = 0;
00064 };
00065
00066 #endif

```

Rodyklė

- ~Studentas
 - Studentas, [21](#)
- ~Vector
 - Vector< T, Allocator >, [37](#)
- ~Zmogus
 - Zmogus, [57](#)
- addNd
 - Studentas, [21](#)
- alloc_
 - Vector< T, Allocator >, [55](#)
- allocator_type
 - Vector< T, Allocator >, [32](#)
- apdorojimas.cpp
 - failoGeneravimas, [64](#)
 - setw, [64](#)
- apdorojimas.h
 - failoGeneravimas, [65](#)
 - setw, [65](#)
- apdorojimas.tpp
 - rusiavimasPagal, [68](#)
 - rusiavimasSkirstymas, [68](#)
 - skirstymas, [68](#)
 - skirstymasStrat1, [69](#)
 - skirstymasStrat2, [70](#)
 - skirstymasStrat3, [70](#)
- arTikSkaicius
 - investis.cpp, [85](#)
 - investis.h, [88](#)
- assign
 - Vector< T, Allocator >, [37](#), [38](#)
- at
 - Vector< T, Allocator >, [38](#), [39](#)
- back
 - Vector< T, Allocator >, [39](#)
- begin
 - Vector< T, Allocator >, [40](#)
- capacity
 - Vector< T, Allocator >, [40](#)
- capacity_
 - Vector< T, Allocator >, [55](#)
- cbegin
 - Vector< T, Allocator >, [40](#)
- cend
 - Vector< T, Allocator >, [41](#)
- cinEOFgaudymas
 - investis.cpp, [85](#)
 - investis.h, [88](#)
- clear
 - Vector< T, Allocator >, [41](#)
- const_iterator
 - Vector< T, Allocator >, [32](#)
- const_pointer
 - Vector< T, Allocator >, [32](#)
- const_reference
 - Vector< T, Allocator >, [32](#)
- const_reverse_iterator
 - Vector< T, Allocator >, [33](#)
- crbegin
 - Vector< T, Allocator >, [41](#)
- crend
 - Vector< T, Allocator >, [41](#)
- data
 - Vector< T, Allocator >, [42](#)
- data_
 - Vector< T, Allocator >, [55](#)
- difference_type
 - Vector< T, Allocator >, [33](#)
- durationDouble
 - Timer, [27](#)
- egzaminoRezultatolvestis
 - investis.h, [89](#)
 - Studentas, [21](#)
- egzRandom
 - Studentas, [21](#)
- elapsed
 - Timer, [28](#)
- emplace
 - Vector< T, Allocator >, [42](#)
- emplace_back
 - Vector< T, Allocator >, [43](#)
- empty
 - Vector< T, Allocator >, [43](#)
- end
 - Vector< T, Allocator >, [43](#), [44](#)
- erase
 - Vector< T, Allocator >, [44](#)
 - vector.h, [118](#)
- erase_if
 - vector.h, [118](#)
- failoApdorojimas
 - main.cpp, [94](#)
 - main.h, [97](#)
- failoGeneravimas
 - apdorojimas.cpp, [64](#)

- apdorojimas.h, 65
- failoPasirinkimas
 - isvestis.cpp, 73
 - isvestis.h, 78
- failoTestavimas
 - main.tpp, 98
- failoUzklausa
 - isvestis.cpp, 73
 - isvestis.h, 79
- front
 - Vector< T, Allocator >, 45
- galutinis
 - Studentas, 21
- gautiPatvirtinima
 - investis.cpp, 85
 - investis.h, 89
- gautiSkaiciu
 - investis.cpp, 85
 - investis.h, 89
- gautiTipoPasirinkima
 - isvestis.cpp, 73
 - isvestis.h, 79
- genVardaPavarde
 - Studentas, 22
- get_allocator
 - Vector< T, Allocator >, 45
- getNd
 - Studentas, 22
- getPavarde
 - Zmogus, 57
- getRez
 - Studentas, 22
- getVardas
 - Zmogus, 57
- gtest.cpp, 61
 - TEST, 61, 62
- hrClock
 - Timer, 27
- insert
 - Vector< T, Allocator >, 46, 47
- isvestis
 - isvestis.tpp, 82
- isvestis.cpp
 - failoPasirinkimas, 73
 - failoUzklausa, 73
 - gautiTipoPasirinkima, 73
 - lietuviskosRaides, 74
 - medianosUzklausa, 74
 - menu, 74
 - ndPasirinkimas, 74
 - rusiavimoPasirinkimas, 75
 - strategijosPasirinkimas, 75
 - studentuPasirinkimas, 75
 - testavimoPasirinkimas, 75
- isvestis.h
 - failoPasirinkimas, 78
 - failoUzklausa, 79
 - gautiTipoPasirinkima, 79
 - lietuviskosRaides, 79
 - medianosUzklausa, 79
 - menu, 80
 - ndPasirinkimas, 80
 - rusiavimoPasirinkimas, 80
 - strategijosPasirinkimas, 80
 - studentuPasirinkimas, 81
 - testavimoPasirinkimas, 81
- isvestis.tpp
 - isvestis, 82
- iterator
 - Vector< T, Allocator >, 33
- investis.cpp
 - arTikSkaicius, 85
 - cinEOFgaudymas, 85
 - gautiPatvirtinima, 85
 - gautiSkaiciu, 85
 - investiStudentus, 86
- investis.h
 - arTikSkaicius, 88
 - cinEOFgaudymas, 88
 - egzaminoRezultatolvestis, 89
 - gautiPatvirtinima, 89
 - gautiSkaiciu, 89
 - investiStudentus, 90
 - namuDarbuRezultatulvestis, 90
 - skaitymas, 90
 - skaitymasIsFailo, 90
 - studentoVardoPavardesIvestis, 91
- investis.tpp
 - skaitymasIsFailo, 92
- investiStudentus
 - investis.cpp, 86
 - investis.h, 90
- investiStudentusRandom
 - random.cpp, 102
 - random.h, 103
- lietuviskosRaides
 - isvestis.cpp, 74
 - isvestis.h, 79
- main
 - main.cpp, 94
- main.cpp
 - failoApdorojimas, 94
 - main, 94
- main.h
 - failoApdorojimas, 97
- main.tpp
 - failoTestavimas, 98
 - skirstymoPasirinkimas, 99
- max_size
 - Vector< T, Allocator >, 48
- maxNdKiekis
 - studentas.h, 108
- medianosUzklausa

- isvestis.cpp, [74](#)
- isvestis.h, [79](#)
- menu
 - isvestis.cpp, [74](#)
 - isvestis.h, [80](#)
- namuDarbuRezultatulvestis
 - ivestis.h, [90](#)
 - Studentas, [22](#)
- nd
 - Studentas, [26](#)
- ndPasirinkimas
 - isvestis.cpp, [74](#)
 - isvestis.h, [80](#)
- ndRandom
 - Studentas, [22](#)
- operator<<
 - Studentas, [26](#)
 - studentas.cpp, [104](#)
- operator<=>
 - vector.h, [119](#)
- operator>>
 - Studentas, [26](#)
 - studentas.cpp, [104](#)
- operator=
 - Studentas, [23](#)
 - Vector< T, Allocator >, [48](#), [49](#)
 - Zmogus, [58](#)
- operator==
 - vector.h, [119](#)
- operator[]
 - Vector< T, Allocator >, [49](#)
- pavarde
 - Zmogus, [59](#)
- pointer
 - Vector< T, Allocator >, [33](#)
- pop_back
 - Vector< T, Allocator >, [50](#)
- Programos naudojimas:, [1](#)
- push_back
 - Vector< T, Allocator >, [50](#)
- random.cpp
 - ivestiStudentusRandom, [102](#)
 - rng, [102](#)
- random.h
 - ivestiStudentusRandom, [103](#)
 - rng, [103](#)
- rbegin
 - Vector< T, Allocator >, [51](#)
- read
 - Studentas, [23](#)
- README.md, [63](#)
- readRandom
 - Studentas, [23](#)
- readSemiRandom
 - Studentas, [24](#)
- readStudentConsole
 - Studentas, [24](#)
- readStudentStream
 - Studentas, [24](#)
- reallocate
 - Vector< T, Allocator >, [51](#)
- reference
 - Vector< T, Allocator >, [33](#)
- rend
 - Vector< T, Allocator >, [51](#), [52](#)
- reserve
 - Vector< T, Allocator >, [52](#)
- reset
 - Timer, [28](#)
- resize
 - Vector< T, Allocator >, [52](#), [53](#)
- resizeNd
 - Studentas, [25](#)
- reverse_iterator
 - Vector< T, Allocator >, [33](#)
- rez
 - Studentas, [26](#)
- rng
 - random.cpp, [102](#)
 - random.h, [103](#)
- rusiavimasPagal
 - apdorojimas.tpp, [68](#)
- rusiavimasSkirstymas
 - apdorojimas.tpp, [68](#)
- rusiavimoPasirinkimas
 - isvestis.cpp, [75](#)
 - isvestis.h, [80](#)
- setNd
 - Studentas, [25](#)
- setPavarde
 - Zmogus, [58](#)
- setRez
 - Studentas, [25](#)
- setVardas
 - Zmogus, [58](#)
- setw
 - apdorojimas.cpp, [64](#)
 - apdorojimas.h, [65](#)
- shift_left
 - Vector< T, Allocator >, [53](#)
- shift_right
 - Vector< T, Allocator >, [53](#)
- shrink_to_fit
 - Vector< T, Allocator >, [54](#)
- size
 - Vector< T, Allocator >, [54](#)
- size_
 - Vector< T, Allocator >, [55](#)
- size_type
 - Vector< T, Allocator >, [33](#)
- skaitymas
 - ivestis.h, [90](#)
- skaitymasIsFailo

- investis.h, 90
- investis.tpp, 92
- skirstymas
 - apdorojimas.tpp, 68
- skirstymasStrat1
 - apdorojimas.tpp, 69
- skirstymasStrat2
 - apdorojimas.tpp, 70
- skirstymasStrat3
 - apdorojimas.tpp, 70
- skirstymoPasirinkimas
 - main.tpp, 99
- start
 - Timer, 28
- strategijosPasirinkimas
 - isvestis.cpp, 75
 - isvestis.h, 80
- Studentas, 17
 - ~Studentas, 21
 - addNd, 21
 - egzaminoRezultatolvestis, 21
 - egzRandom, 21
 - galutinis, 21
 - genVardaPavarde, 22
 - getNd, 22
 - getRez, 22
 - namuDarbuRezultatulvestis, 22
 - nd, 26
 - ndRandom, 22
 - operator<<, 26
 - operator>>, 26
 - operator=, 23
 - read, 23
 - readRandom, 23
 - readSemiRandom, 24
 - readStudentConsole, 24
 - readStudentStream, 24
 - resizeNd, 25
 - rez, 26
 - setNd, 25
 - setRez, 25
 - Studentas, 20
 - studentoVardoPavardesIvestis, 25
- studentas.cpp
 - operator<<, 104
 - operator>>, 104
- studentas.h
 - maxNdKiekis, 108
- studentoVardoPavardesIvestis
 - investis.h, 91
 - Studentas, 25
- studentuPasirinkimas
 - isvestis.cpp, 75
 - isvestis.h, 81
- swap
 - Vector< T, Allocator >, 54
 - vector.h, 120

TEST

- gtest.cpp, 61, 62
- test.cpp
 - testas, 110
 - testCopyAssignmentOperator, 110
 - testCopyKonstruktorius, 110
 - testDefaultKonstruktorius, 111
 - testDestructor, 111
 - testInputOperator, 111
 - testMoveAssignmentOperator, 111
 - testMoveKonstruktorius, 111
 - testoRezultatas, 111
 - testOutputOperator, 112
 - testStreamKonstruktorius, 112
- test.h
 - testas, 114
 - testCopyAssignmentOperator, 114
 - testCopyKonstruktorius, 114
 - testDefaultKonstruktorius, 114
 - testDestructor, 115
 - testInputOperator, 115
 - testMoveAssignmentOperator, 115
 - testMoveKonstruktorius, 115
 - testoRezultatas, 115
 - testOutputOperator, 116
 - testStreamKonstruktorius, 116
- testas
 - test.cpp, 110
 - test.h, 114
- testavimoPasirinkimas
 - isvestis.cpp, 75
 - isvestis.h, 81
- testCopyAssignmentOperator
 - test.cpp, 110
 - test.h, 114
- testCopyKonstruktorius
 - test.cpp, 110
 - test.h, 114
- testDefaultKonstruktorius
 - test.cpp, 111
 - test.h, 114
- testDestructor
 - test.cpp, 111
 - test.h, 115
- testInputOperator
 - test.cpp, 111
 - test.h, 115
- testMoveAssignmentOperator
 - test.cpp, 111
 - test.h, 115
- testMoveKonstruktorius
 - test.cpp, 111
 - test.h, 115
- testoRezultatas
 - test.cpp, 111
 - test.h, 115
- testOutputOperator
 - test.cpp, 112
 - test.h, 116

- testStreamKonstruktorius
 - test.cpp, 112
 - test.h, 116
- Timer, 27
 - durationDouble, 27
 - elapsed, 28
 - hrClock, 27
 - reset, 28
 - start, 28
 - Timer, 28
- value_type
 - Vector< T, Allocator >, 34
- vardas
 - Zmogus, 59
- Vector
 - Vector< T, Allocator >, 34–36
- vector Direktorijos aprašas, 15
- Vector< T, Allocator >, 28
 - ~Vector, 37
 - alloc_, 55
 - allocator_type, 32
 - assign, 37, 38
 - at, 38, 39
 - back, 39
 - begin, 40
 - capacity, 40
 - capacity_, 55
 - cbegin, 40
 - cend, 41
 - clear, 41
 - const_iterator, 32
 - const_pointer, 32
 - const_reference, 32
 - const_reverse_iterator, 33
 - crbegin, 41
 - crend, 41
 - data, 42
 - data_, 55
 - difference_type, 33
 - emplace, 42
 - emplace_back, 43
 - empty, 43
 - end, 43, 44
 - erase, 44
 - front, 45
 - get_allocator, 45
 - insert, 46, 47
 - iterator, 33
 - max_size, 48
 - operator=, 48, 49
 - operator[], 49
 - pointer, 33
 - pop_back, 50
 - push_back, 50
 - rbegin, 51
 - reallocate, 51
 - reference, 33
 - rend, 51, 52
 - reserve, 52
 - resize, 52, 53
 - reverse_iterator, 33
 - shift_left, 53
 - shift_right, 53
 - shrink_to_fit, 54
 - size, 54
 - size_, 55
 - size_type, 33
 - swap, 54
 - value_type, 34
 - Vector, 34–36
- vector.h
 - erase, 118
 - erase_if, 118
 - operator<=>, 119
 - operator==, 119
 - swap, 120
- vector/apdorojimas.cpp, 63, 64
- vector/apdorojimas.h, 65, 67
- vector/apdorojimas.tpp, 67, 71
- vector/isvestis.cpp, 72, 76
- vector/isvestis.h, 78, 82
- vector/isvestis.tpp, 82, 84
- vector/ivestis.cpp, 84, 86
- vector/ivestis.h, 88, 92
- vector/ivestis.tpp, 92, 93
- vector/main.cpp, 93, 94
- vector/main.h, 97, 98
- vector/main.tpp, 98, 100
- vector/random.cpp, 101, 102
- vector/random.h, 103, 104
- vector/studentas.cpp, 104, 105
- vector/studentas.h, 108, 109
- vector/test.cpp, 110, 112
- vector/test.h, 113, 116
- vector/Timer.h, 116, 117
- vector/vector.h, 117, 120
- vector/vector_compare.cpp, 127, 128
- vector/vector_compare.h, 128, 129
- vector/zmogus.cpp, 129
- vector/zmogus.h, 130
- vector_compare
 - vector_compare.cpp, 128
 - vector_compare.h, 129
- vector_compare.cpp
 - vector_compare, 128
- vector_compare.h
 - vector_compare, 129
- Zmogus, 55
 - ~Zmogus, 57
 - getPavarde, 57
 - getVardas, 57
 - operator=, 58
 - pavarde, 59
 - setPavarde, 58
 - setVardas, 58
 - vardas, 59

Zmogus, [56](#), [57](#)